COLEGIO UNIVERSITARIO IES
Analista en Sistemas de Información · Córdoba

SISTEMA DE GESTIÓN INTEGRAL PARA EMPRESA AVÍCOLA

TILCOR S.A.S. — Colonia Tirolesa, Córdoba

DESARROLLO COMPLETO DEL PROYECTO

PPIII · Instancia Evaluativa 2

AUTORES	REFERENCIAS
Carnielli, Matías Bacari, Fernando	Demo: tilcorvercel.vercel.app Código: github.com/Mcarnielli/Ingenieria-en-SFT-Tilcor

Córdoba, noviembre de 2026

ÍNDICE GENERAL

PARTE 1 — Relevamiento y propuesta (PP2) 7

Resumen ejecutivo 8

1 Introducción 9

1.1 Carta de presentación (opcional) 9

1.2 Pedido del usuario 9

1.2.1 Identificación del Cliente 9

1.2.2 Solicitud Original del Cliente 9

1.2.3 Necesidades Identificadas por el Cliente 10

1.2.4 Restricciones y Condiciones del Cliente 10

1.3 Descripción general del proyecto 10

1.3.1 Descripción del Proyecto 10

1.3.2 Justificación 11

1.3.3 Objetivos 11

1.3.4 Alcance 11

2 Relevamiento 13

2.1 Contextual y del negocio 13

2.1.1 Historia y Contexto de la Organización 13

2.1.2 Misión y Visión 13

2.1.3 Datos Productivos Registrados 13

2.1.4 Resumen de Indicadores por Raza 14

2.1.5 Curvas de Producción y Viabilidad 15

2.1.6 Variables Ambientales Relevadas 16

2.2 Estructura de la organización 16

2.2.1 Estructura Organizaciónal — Organigrama 16

2.2.2 Descripción de los Puestos 16

2.2.3 Cultura Organizaciónal 17

2.2.4 Descripción de las Instalaciones Físicas 17

2.2.5 Equipamiento Principal 18

2.2.6 Red de Sensores IoT Instalados 18

2.3 Procesos de negocio 19

2.3.1 Proceso 1: Producción avícola diaria 21

2.3.2 Proceso 2: Gestión comercial y facturación 24

2.3.3 Proceso 3: Monitoreo ambiental IoT 27

2.3.4 Proceso 4: Control sanitario 29

2.4 Tecnología 31

2.4.1 Tecnología existente en la empresa 31

2.4.2 Tecnología existente en el mercado (aplicable) 35

3 Propuesta 36

3.1 Diagnóstico y propuesta 36

3.1.1 Resumen del Diagnóstico 36

3.1.2 Problemas Críticos Identificados 36

3.1.3 Cuadro Comparativo de Mejoras 36

3.1.4 Conclusión del Diagnóstico 37

3.1.5 Situación propuesta de los procesos 38

3.1.6 Diagnóstico de la propuesta 42

3.2 Propuesta general 44

3.2.1 Visión General 44

3.2.2 Módulos del Sistema 44

3.2.3 Roles del Sistema 44
3.2.4 Tecnologías Propuestas 44
3.2.5 Plan de Implementación 45
3.2.6 Requerimientos preliminares del relevamiento 45

PARTE 2 — Modelado y desarrollo del sistema (PP3) 47

4 Objetivos, límites y alcance 48
4.1 Objetivo del sistema 48
4.1.1 Objetivo General 48
4.1.2 Objetivos Específicos 48
4.2 Límite del sistema 48
4.2.1 Definición del Límite 48
4.2.2 Dentro del límite del sistema 48
4.2.3 Fuera del límite del sistema 49
4.3 Alcance del sistema 49
4.3.1 Alcance Funciónal 49
4.3.2 Alcance Técnico 50
5 Especificación de requerimientos 51
5.1 Requerimientos Funciónales — Sistema Ambiental IoT 51
5.2 Requerimientos Funciónales — Producción y Sanidad 51
5.3 Requerimientos No Funciónales 52
6 Análisis y diseño del producto 53
6.1 Modelado ambiental 53
6.1.1 Descripción General 53
6.1.2 Actores del Sistema 53
6.1.3 Diagrama de Caso de Uso 53
6.1.4 Especificaciones Detalladas de Casos de Uso Clave 54
6.1.5 Relaciones entre casos de uso 56
6.1.6 Articulación con el relevamiento y la propuesta (Parte 1) 57
6.2 Modelado de paquetes 58
6.2.1 Visión General 58
6.2.2 Stack Tecnológico 59
6.2.3 Patrones de Diseño Aplicados 59
6.2.4 Modelo de datos 60
6.3 Modelado de los casos de uso 66
6.3.1 Módulo 1 — Producción avícola 67
6.3.1.1 CU01 — Registrar producción diaria de huevos 67
6.3.1.2 CU02 — Registrar mortalidad diaria de aves 69
6.3.1.3 CU04 — Ver historial de producción por fecha 71
6.3.1.4 CU09 — Ver curva de postura real vs tabla genética 73
6.3.2 Módulo 2 — Monitoreo IoT y alertas 77
6.3.2.1 CU03 — Consultar estado actual de sensores IoT 77
6.3.2.2 CU05 — Configurar umbrales de alerta por sensor 79
6.3.2.3 CU06 — Recibir alerta Warning 81
6.3.2.4 CU07 — Recibir alerta Crítico 84
6.3.2.5 CU08 — Consultar dashboard IoT en tiempo real 86
6.3.2.6 CU19-22 — Flujo automático IoT (CU19 a CU22) 89
6.3.3 Módulo 3 — Sanidad y mortalidad 92
6.3.3.1 CU14 — Ver viabilidad acumulada de los lotes 92
6.3.3.2 CU16 — Recibir informe semanal de mortalidad 94
6.3.3.3 CU17 — Consultar historial sanitario del lote 97

6.3.3.4 CU18 — Ver curva de mortalidad histórica 99
6.3.4 Módulo 4 — Dashboard, reportes y plan de implementación 102
6.3.4.1 CU10 — Editar progreso del plan de implementación 102
6.3.4.2 CU12 — Ver KPIs generales en dashboard principal 104
6.3.4.3 CU13 — Recibir reporte diario automatizado 108
6.3.5 Módulo 5 — Usuarios y roles 111
6.3.5.1 CU15 — Iniciar sesión con rol diferenciado 111
6.3.6 Módulo 6 — Gestión comercial (Don Gestión) 113
6.3.6.1 CU11 — Aprobar pedido que supera límite de crédito 113
6.3.6.2 CU23 — Actualizar datos comerciales (Don Gestión) 115
6.3.6.3 CU24 — Emitir factura electrónica ARCA/AFIP 117
7 Implementación del software (desarrollo) 120
7.1 Módulo 1 — Producción avícola 121
7.1.1 CU01 — Registrar producción diaria de huevos 121
7.1.2 CU02 — Registrar mortalidad diaria de aves 121
7.1.3 CU04 — Ver historial de producción por fecha 121
7.1.4 CU09 — Ver curva de postura real vs tabla genética 122
7.1.5 Diseño y código de referencia (Tesis) 122
7.2 Módulo 2 — Monitoreo IoT y alertas 128
7.2.1 CU03 — Consultar estado actual de sensores IoT 128
7.2.2 CU05 — Configurar umbrales de alerta por sensor 128
7.2.3 CU06 — Recibir alerta Warning 128
7.2.4 CU07 — Recibir alerta Crítico 129
7.2.5 CU08 — Consultar dashboard IoT en tiempo real 129
7.2.6 CU19-22 — Flujo automático IoT (CU19 a CU22) 129
7.2.7 Diseño y código de referencia (Tesis) 131
7.3 Módulo 3 — Sanidad y mortalidad 136
7.3.1 CU14 — Ver viabilidad acumulada de los lotes 136
7.3.2 CU16 — Recibir informe semanal de mortalidad 136
7.3.3 CU17 — Consultar historial sanitario del lote 136
7.3.4 CU18 — Ver curva de mortalidad histórica 136
7.4 Módulo 4 — Dashboard, reportes y plan de implementación 137
7.4.1 CU10 — Editar progreso del plan de implementación 137
7.4.2 CU12 — Ver KPIs generales en dashboard principal 137
7.4.3 CU13 — Recibir reporte diario automatizado 137
7.4.4 Diseño y código de referencia (Tesis) 137
7.5 Módulo 5 — Usuarios y roles 139
7.5.1 CU15 — Iniciar sesión con rol diferenciado 139
7.5.2 Diseño y código de referencia (Tesis) 140
7.6 Módulo 6 — Gestión comercial (Don Gestión) 142
7.6.1 CU11 — Aprobar pedido que supera límite de crédito 142
7.6.2 CU23 — Actualizar datos comerciales (Don Gestión) 142
7.6.3 CU24 — Emitir factura electrónica ARCA/AFIP 142
7.6.4 Diseño y código de referencia (Tesis) 142
8 Integración y Testing 146
8.1 Alcance y entorno de prueba 146
8.2 Estrategia de integración 146
8.3 Pruebas unitarias y de integración del módulo IoT 146
8.4 Pruebas de la interfaz (demo funcional) 147
8.5 Defectos detectados y corregidos 148

8.6 Pruebas de aceptación con el cliente 148
8.7 Pruebas con hardware real en TILCOR 148
8.8 Limitaciones y trabajo futuro 149
8.9 Plan de pruebas del sistema completo (diseño) 150
8.9.1 Estrategia 150
8.9.2 Pruebas Unitarias e Integración 150
8.9.3 Cobertura de Pruebas 152
9 Especificaciones del Sistema 153
9.1 Despliegue 153
9.1.1 Arquitectura de despliegue 153
9.1.2 Contenedores y puertos 153
9.1.3 Procedimiento de despliegue 153
9.2 Hardware mínimo y óptimo 154
9.2.1 Servidor 154
9.2.2 Dimensionamiento de datos 154
9.2.3 Dispositivos de campo y usuario 154
9.3 Software de base y aplicación 155
9.4 Seguridad y auditoría 155
9.4.1 Autenticación y control de acceso 155
9.4.2 Protección de datos en tránsito y en reposo 155
9.4.3 Auditoría 156
9.4.4 Riesgos de seguridad y medidas 156
9.5 Configuración de despliegue de referencia 157
9.5.1 Configuración Docker Compose 157
9.5.2 Variables de Entorno (.env) 158
9.5.3 Estrategia de Backup 159

CIERRE — Conclusiones, glosario y referencias 160
Conclusiones 160
Logros Técnicos 161
Desafíos y Soluciones 161
Trabajos Futuros 161
Glosario técnico 162
Referencias bibliográficas 163

GUÍA DE LECTURA Y ESTADO DE IMPLEMENTACIÓN

Este documento reúne todo el desarrollo del proyecto en un único archivo, organizado según el **Checklist de Avances** de la materia, en dos partes con numeración continua de capítulos del 1 al 9.

Parte	Capítulos	Contenido
1 · PP2	1 a 3	Introducción (carta y pedido del usuario), relevamiento (contexto, estructura, procesos con cursogramas, tecnología existente y de mercado) y propuesta (diagnóstico y propuesta general).
2 · PP3	4 a 6	Objetivos, límites y alcance; especificación de requerimientos; análisis y diseño: modelado ambiental, de paquetes y de los 24 casos de uso por módulo (requerimientos, prototipos, ficha, vista estática y dinámica).
2 · PP3	7 a 9	Implementación por módulo y caso de uso (código real del repositorio y de la demo); integración y testing con resultados reales; especificaciones del sistema (despliegue, hardware, software, seguridad y auditoría).
Cierre	—	Conclusiones, glosario técnico y referencias bibliográficas.

Estado de implementación (a la fecha de entrega)

Componente	Estado	Detalle
Demo funcional (PWA)	Implementado	En línea en tilcorvercel.vercel.app. Perfiles Administrador y Empleado; inicio, producción, IoT, sanidad y ventas (Don Gestión por iframe). Datos de ejemplo en el navegador.
Módulo IoT (MQTT → PostgreSQL)	Implementado y probado	Ingesta, validación, motor de alertas, simulador de sensores y auditoría por triggers. 29 pruebas automáticas aprobadas (cap. 8).
Prueba con sensores reales	Pendiente	Se realiza en una visita a TILCOR; resultados a completar en la tabla de la sección 8.7.
Backend completo (API REST, JWT, clientes, facturación AFIP, reportes)	Diseñado	Especificado en los capítulos 4 a 7; su construcción corresponde a las fases 3 a 5 del plan de implementación (sección 3.2.5).
Prototipos de CU05, CU11, CU13, CU16 y CU23	Bocetos	Las pantallas de estos casos de uso no están en la demo; se presentan como bocetos rotulados en la sección 6.3.

Nota: los capítulos 1 a 3 conservan el texto del relevamiento original, redactado antes de la demo; las cifras y el estado de los sensores allí descriptos corresponden a la fecha del relevamiento.

PARTE 1

Relevamiento y propuesta (PP2)

1 Introducción
2 Relevamiento
3 Propuesta

Resumen ejecutivo

El presente documento corresponde al Trabajo Final de Tesis de la carrera Analista en Sistemas de Informacion y documenta el relevamiento, analisis y propuesta de un Sistema de Gestion Integral (SGI) para una empresa avicola productora de huevos ubicada en la provincia de Cordoba, Argentina.

La empresa opera con gallinas ponedoras de las razas Lohmann Brown-Classic y Lohmann LSL-Classic en un galpon principal con capacidad para mas de 4.500 aves por lote. El relevamiento evidencio tres problemas criticos: registro y gestion completamente manual fragmentada en multiples herramientas sin conexion entre si; ausencia total de alertas automaticas que resulto en la no deteccion oportuna de una caida de viabilidad del lote LSL-Classic de 26,3 puntos porcentuales respecto de la tabla del fabricante; y un proceso comercial y de facturacion desintegrado con doble carga de datos y demoras de hasta 48 horas por pedido.

La propuesta consiste en un sistema web integral que centraliza la gestion de clientes, el control de produccion avicola, el monitoreo IoT en tiempo real, la facturacion electronica con AFIP y la generacion de reportes automaticos, con una reduccion estimada del 85% en el tiempo de registro de produccion y la automatizacion del 61% de las actividades administrativas actuales.

1 Introducción

1.1 Carta de presentación (opcional)

Córdoba, noviembre de 2026

A la Dirección de **TILCOR S.A.S.**
Colonia Tirolesa, Córdoba

De nuestra mayor consideración:

Nos dirigimos a ustedes, estudiantes de la carrera de Analista en Sistemas de Información del Colegio Universitario IES (Córdoba), para presentarles el trabajo de relevamiento, diseño y desarrollo del **Sistema de Gestión Integral Avícola** que realizamos tomando como caso de estudio a la empresa.

El documento que sigue recoge lo observado en las visitas a los galpones y en las conversaciones con el personal, el diagnóstico de la situación actual, la propuesta de sistema y su especificación, junto con una demostración funcional y un módulo de ingesta de datos de sensores ya probado con datos simulados.

Agradecemos la predisposición del personal de TILCOR para mostrarnos los procesos, las planillas y los sistemas que utilizan (Gallinero y Don Gestión), y quedamos a disposición para revisar este material y ajustarlo a las necesidades reales de la empresa.

Atentamente,

_____ _____

Carnielli, Matías Bacari, Fernando

1.2 Pedido del usuario

1.2.1 Identificación del Cliente

Campo	Detalle
Organizacion	Empresa Avicola (Galpon 1 — Lote F1)
Rubro	Produccion avicola intensiva de postura — Huevos comerciales
Ubicacion	Provincia de Cordoba, Argentina
Tamano	Pequeña-mediana empresa (PyME) — 3.500 a 4.500 aves activas por lote
Interlocutor principal	Propietario / Gerente General
Usuarios del sistema	Propietario, Supervisor Productivo, Operarios de Galpon, Vendedor, Veterinario (externo)

1.2.2 Solicitud Original del Cliente

> *El cliente solicita el desarrollo de un sistema informatico que permita: (1) registrar diariamente la produccion de huevos por galpon y lote desde un dispositivo movil en el galpon mismo, sin necesidad de trasladarse a una oficina; (2) visualizar en tiempo real el estado de los sensores ambientales del galpon (temperatura, humedad, calidad del aire y agua) y recibir alertas automaticas cuando algun valor salga del rango normal; (3) gestionar los clientes compradores de huevos, sus pedidos, remitos y*

facturas electronicas de manera integrada; y (4) obtener reportes automaticos de produccion sin necesidad de armar planillas manualmente cada semana.

1.2.3 Necesidades Identificadas por el Cliente

- Eliminar el registro manual en papel y Excel que genera errores y consume tiempo del supervisor.
- Recibir alertas en el celular cuando la temperatura, la humedad o el flujo de agua salgan del rango normal.
- Tener un registro centralizado de todos los clientes con su saldo de cuenta corriente actualizado.
- Emitir facturas electronicas directamente desde el sistema sin usar un programa separado.
- Ver de manera grafica si la produccion del lote esta por encima o por debajo de la curva estandar de la raza.
- Saber cuantas aves mueren por semana y recibir una alerta si la mortalidad supera lo normal.

1.2.4 Restricciones y Condiciones del Cliente

Restriccion	Descripcion
Presupuesto	Desarrollo a cargo de los autores como proyecto academico — sin costo para el cliente
Infraestructura existente	Sensores IoT ya instalados y funcionando con plataforma 'Gallinero' — deben integrarse
Dispositivos	El operario de galpon usa smartphone — la interfaz de registro debe ser mobile-first
Conectividad	La zona puede tener cortes de internet — el sistema debe tolerar modo offline con sincronizacion
Usuarios no tecnicos	Los operarios del galpon tienen bajo nivel de experiencia informatica — interfaz ultra-simple
Obligacion fiscal	La empresa debe emitir facturas electronicas segun normativa AFIP vigente

1.3 Descripción general del proyecto

1.3.1 Descripción del Proyecto

El presente proyecto consiste en el analisis, diseno e implementacion de un Sistema de Gestion Integral (SGI) completo para una empresa avicola productora de huevos en Cordoba. La empresa se especializa en la cria intensiva de gallinas ponedoras con instalaciones de jaulas en bateria, sistema de recoleccion automatica de huevos marca Instalaciones Cattaneo, y una red de sensores IoT para monitoreo ambiental continuo del galpon.

Este trabajo fue realizado en el marco del Trabajo Final de Tesis de la carrera Analista en Sistemas de Informacion, con el objetivo de aplicar las metodologias, herramientas y conocimientos adquiridos durante la carrera al desarrollo de una solucion informatica real para una organizacion real.

1.3.2 Justificación

La produccion avicola intensiva requiere el control preciso y simultaneo de variables biologicas, ambientales, sanitarias y comerciales. La ausencia de un sistema integrado genera fragmentacion de la informacion, dificultad en la trazabilidad, errores en la facturacion y — lo mas grave — demoras en la deteccion de problemas productivos que representan perdidas economicas directas.

El caso mas elocuente relevado en esta empresa es el lote Lohmann LSL-Classic, cuya viabilidad real de 67,3% se encuentra 26,3 puntos porcentuales por debajo del estandar de tabla del fabricante (93,6%), sin que existiera un mecanismo de alerta temprana que permitiera intervencion veterinaria oportuna.

1.3.3 Objetivos

1.3.3.1 Objetivo General

Desarrollar un Sistema de Gestion Integral para una empresa avicola que centralice la administracion de clientes, el control productivo, el monitoreo ambiental IoT y la gestion comercial, mejorando la eficiencia operativa, la trazabilidad y la capacidad de respuesta ante eventos criticos.

1.3.3.2 Objetivos Específicos

1. Relevar y documentar los procesos actuales de gestion de la empresa avicola.
2. Realizar el relevamiento organizacional, estructural, funcional y tecnologico de la organizacion.
3. Identificar los problemas criticos mediante cursogramas analiticos y elaborar el diagnostico de situacion.
4. Documentar el pedido del usuario y los requerimientos del sistema.
5. Disenar la arquitectura del sistema integrando modulos de clientes, produccion, sensores IoT y facturacion.
6. Implementar el sistema y validarlo con datos reales de la empresa.

1.3.4 Alcance

El sistema abarca: gestion de clientes y ventas, control de produccion avicola por galpon y lote, monitoreo IoT ambiental en tiempo real, administracion de lotes y razas con tabla genetica del fabricante, gestion de mortalidad y viabilidad, facturacion electronica AFIP-WSFE, cuenta corriente de clientes, y reportes automatizados. Queda fuera del alcance: integracion con

sistemas contables externos, modulo de recursos humanos y gestion de proveedores de alimento balanceado.

2 Relevamiento

2.1 Contextual y del negocio

2.1.1 Historia y Contexto de la Organización

La empresa es una unidad productiva familiar dedicada a la produccion comercial de huevos de gallina en la provincia de Cordoba. Cuenta con mas de 10 anos de trayectoria en el sector avicola, habiendo crecido progresivamente en capacidad instalada y tecnificacion de sus procesos. Actualmente opera con dos razas geneticas de alta produccion: Lohmann Brown-Classic y Lohmann LSL-Classic, comercializando sus productos principalmente en el mercado local cordobes.

Los lotes relevados (semanas 87-89 al momento del relevamiento) evidencian un ciclo productivo consolidado de mas de un ano y medio, con datos de produccion que en el caso del lote Brown-Classic superan la tabla de referencia del fabricante en 21,2 huevos por ave alojada.

2.1.2 Misión y Visión

	Descripcion
Mision	Producir huevos de calidad con el mayor bienestar animal posible, brindando a nuestros clientes un producto fresco, trazable y entregado en tiempo y forma, a traves de una gestion eficiente de los recursos disponibles.
Vision	Consolidarse como un proveedor avicola de referencia en la provincia de Cordoba, incorporando tecnologia que permita mejorar continuamente la eficiencia productiva, la trazabilidad y la gestion comercial.

2.1.3 Datos Productivos Registrados

El relevamiento de los datos productivos se realizo a partir de las planillas Excel y de papel del Galpon 1, Lote F1. Se relevaron las semanas 87 a 89 del ciclo productivo para ambas razas, junto con las curvas de produccion y viabilidad acumulada comparadas contra la tabla genetica del fabricante Lohmann Tierzucht GmbH.

2.1.3.1 Planillas de Producción Relevadas

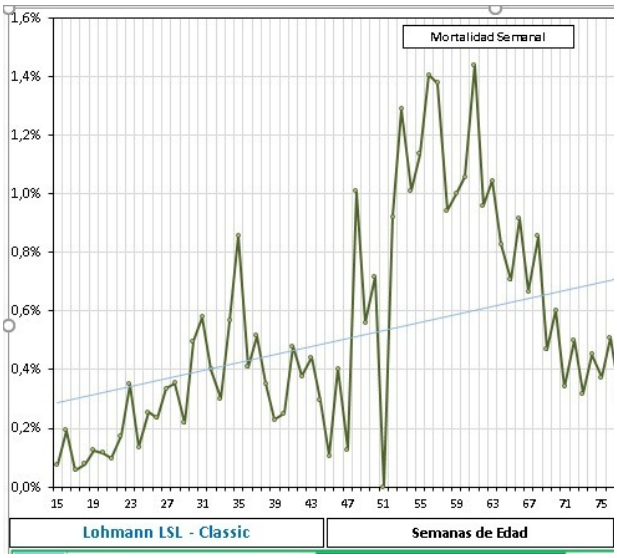


Fig. 9: Planilla Lohmann Brown-Classic — semanas 87-88

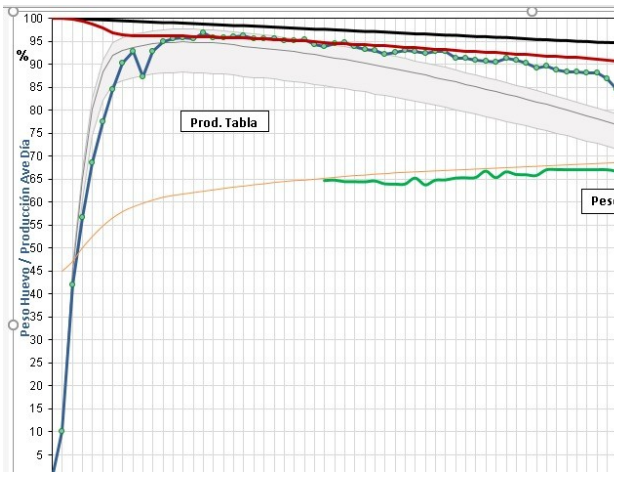


Fig. 10: Planilla Lohmann Brown-Classic — semana 88 detallada

Galpón 1			Lote F1		Lohmann LSL - Classic			
					Fecha actual:	17/5/2024		
Edad	Día	Huevos	Cajones	Maples	Muertas	Existencia	% Mort Sem	% Postura
	9/5/2024	2.940		98	7	3.526	0,20	83,39
	Totales	**20.880**	**58**	**0**	**33**	**3.526**	**0,94**	**84,61**
88	10/5/2024	2.940		98	3	3.523	0,09	83,46
	11/5/2024	2.880		96	1	3.522	0,03	81,78
	12/5/2024	2.940		98		3.522	0,00	83,49
	13/5/2024	2.940		98	7	3.515	0,20	83,65
	14/5/2024	2.550		85	7	3.508	0,20	72,70
	15/5/2024	3.000		100	3	3.505	0,09	85,60
	16/5/2024	2.820		94	6	3.499	0,17	80,61
	Totales	**20.070**	**55**	**9**	**27**	**3.499**	**0,77**	**81,95**
89	17/5/2024				2			

Fig. 11: Planilla Lohmann LSL-Classic — semana 88

2.1.4 Resumen de Indicadores por Raza

Indicador	L. Brown — Tabla	L. Brown — Real	L. LSL — Tabla	L. LSL — Real
Huevos AA acumulados	393,5	414,7 (+21,2)	350,6	366,7 (+16,1)
Viabilidad acumulada	93,1%	90,2%	93,6%	67,3% (-26,3pp)
% Postura sem. 88	82,0%	81,95%	82,0%	84,61%
Mortalidad sem. 88	< 0,5%	0,77%	< 0,5%	0,94%

HALLAZGO CRITICO: La viabilidad del lote LSL-Classic (67,3%) es 26,3 puntos porcentuales inferior a la tabla del fabricante (93,6%). Esto representa la perdida acumulada de aproximadamente 870 aves sobre el inicio del ciclo, sin que existiera un mecanismo automatico de alerta durante todo el periodo.

2.1.5 Curvas de Producción y Viabilidad

Galpón 1			Lote F1			
Edad	Día	Huevos	Cajones	Maples	Muertas	
	Totales	25.080	69	8	8	
87	3/5/2024	3.570		119	1	
	4/5/2024	3.570		119	2	
	5/5/2024	3.510		117		
	6/5/2024	3.630		121	1	
	7/5/2024	3.510		117	3	
	8/5/2024	3.480		116		
	9/5/2024	3.360		112	3	
	Totales	24.630	68	5	10	
88	10/5/2024	3.510		117	4	

Fig. 12: Curva Lohmann Brown-Classic — real vs tabla

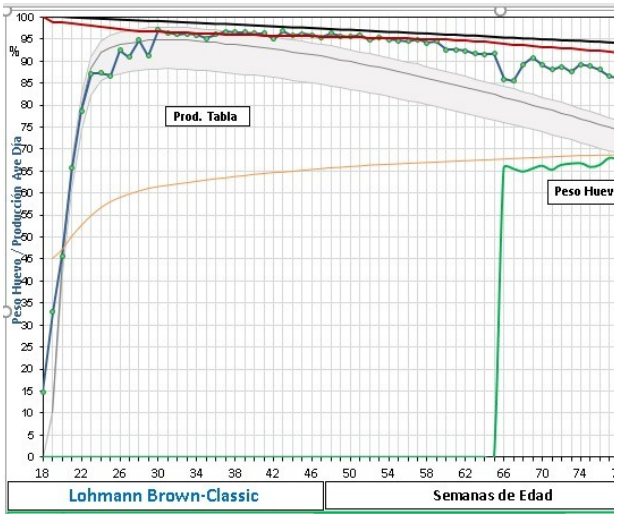


Fig. 13: Curva Lohmann LSL-Classic — real vs tabla

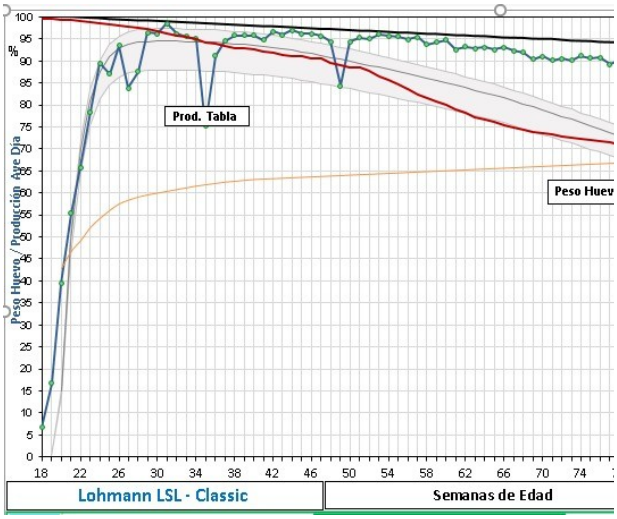


Fig. 14: Curva comparativa complementaria

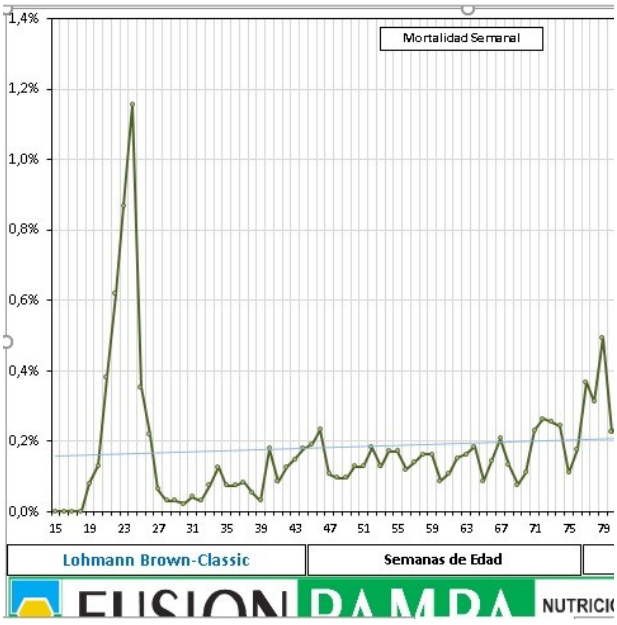


Fig. 15: Mortalidad semanal LSL-Classic — tendencia ascendente

2.1.6 Variables Ambientales Relevadas

Variable	Minimo	Maximo	Ultimo	Alerta detectable	Ocurrio sin alerta
Temp. 1 (°C)	23,1	30,0	28,6	Si > 30°C	Si — 11/12 08:27
Temp. 2 (°C)	22,6	28,9	27,4	Si > 30°C	No
Hum. 1 (%)	43,7	67,4	48,4	Si < 45%	Si — 04/12 10:11
Hum. 2 (%)	47,6	72,7	51,1	Si < 45% o > 75%	No
Calidad Aire 2 (ppm)	190,6	232,4	205,0	Si > 3000 ppm	No
Flujo Agua (L)	0,0	98,0	55,0	Si = 0 por > 2hs	Si — 11/12 02:21
Temp. Agua (°C)	20,75	25,75	25,6	Si > 30°C	No

2.2 Estructura de la organización

2.2.1 Estructura Organizaciónal — Organigrama

La empresa presenta una estructura organizacional simple y plana, tipica de las PyMEs agropecuarias. El propietario concentra la toma de decisiones estrategicas y supervisa directamente las tres areas operativas: produccion, comercializacion y sanidad animal.



Figura: Organigrama de la empresa avicola

2.2.2 Descripción de los Puestos

Puesto	Cantidad	Funciones principales	Interaccion con el sistema
Propietario / Gerente General	1	Toma de decisiones, relacion con proveedores, control general	Dashboard KPIs, reportes automaticos, alertas criticas
Supervisor de Galpon	1	Control de produccion, seguimiento de indicadores, coordinacion operativa	Modulo produccion, curvas, alertas, reportes semanales
Operario de Galpon	2-3	Recoleccion de huevos, registro de produccion, limpieza y mantenimiento	Formulario de registro diario (app movil — 5 campos)

Puesto	Cantidad	Funciones principales	Interaccion con el sistema
Vendedor / Administrativo	1	Atencion a clientes, gestion de pedidos, facturacion, cobros	Modulo clientes, pedidos, facturacion, cta. corriente
Repartidor	1	Distribucion y entrega de pedidos a clientes	Confirmacion digital de entrega en app movil
Veterinario (externo)	1	Visitas periodicas, diagnostico sanitario, recetas	Reporte de mortalidad PDF (envio automatico semanal)

2.2.3 Cultura Organizaciónal

La organizacion presenta una cultura tipica de empresa familiar: alta flexibilidad, comunicacion directa e informal entre los integrantes, toma de decisiones centralizada en el propietario y baja formalizacion de los procesos. Esta caracteristica implica que la adopcion del sistema debe ser gradual, con capacitacion personalizada y una interfaz de usuario extremadamente simple para los operarios del galpon, que son quienes tienen menor experiencia con herramientas informaticas.

2.2.4 Descripción de las Instalaciones Físicas

El establecimiento avicola cuenta con un galpon principal (Galpon 1) de estructura metalica con techo de chapa, iluminacion LED con control automatico de fotoperiodo, y piso de cemento con desague. El galpon esta dividido en dos sectores longitudinales de jaulas en bateria, con pasillos de servicio centrales y laterales para el acceso del personal.



Figura 1: Interior del Galpon 1 — Sistema de jaulas en bateria y cinta recolectora de huevos (Instalaciones Cattaneo)

2.2.5 Equipamiento Principal

Equipamiento	Marca/Tipo	Estado	Funcion
Sistema de jaulas en bateria	Bateria convencional	Bueno	Alojamiento de gallinas ponedoras
Cinta recolectora de huevos	Instalaciones Cattaneo	Bueno	Recoleccion automatica diaria de huevos
Sistema de alimentacion automatica	Comedero en linea	Bueno	Provision de alimento balanceado
Bebederos automaticos	Niple con sensor de flujo	Bueno	Provision de agua de bebida
Iluminacion LED	Control de fotoperiodo	Bueno	Estimulo lumínico para la postura
Red de sensores IoT	Multimarca	Funcionando	Monitoreo ambiental en tiempo real
Smartphone operario	Android	Activo	Acceso a plataforma Gallinero y sistema propuesto
Computadora de escritorio	PC convencional	Activa	Gestion administrativa y comercial

2.2.6 Red de Sensores IoT Instalados

Sensor	Cantidad	Variable medida	Unidad	Ubicacion
Temperatura ambiente	2	Temperatura del galpon	°C	Sector Norte y Sector Sur del galpon
Humedad relativa	2	Humedad relativa del aire	Porcentaje	Sector Norte y Sector Sur del galpon
Calidad del aire / CO2	2	Concentracion de gases	ppm	Altura media del galpon
Flujo de agua	1	Consumo de agua de bebida	Litros	Linea principal de bebederos
Temperatura del agua	1	Temperatura del agua de bebida	°C	Deposito principal

2.3 Procesos de negocio

Esta sección describe los cuatro procesos relevados. Para cada uno se presenta la descripción funcional y, a continuación, el cursograma de la situación actual. Los cursogramas de la situación propuesta se encuentran en la sección 3.1.

Descripción general de los procesos

La operacion de la empresa se organiza en cuatro procesos principales: produccion avicola diaria, monitoreo ambiental del galpon, gestion comercial y administrativa, y control sanitario del lote. Cada proceso fue relevado mediante observacion directa y entrevistas con los responsables, documentando el flujo de actividades, los tiempos involucrados, los responsables y los problemas identificados en la situacion actual.

Simbología utilizada en los cursogramas

Los cursogramas de este documento utilizan la simbologia ANSI estandar para diagramas de flujo de procesos administrativos. Cada simbolo representa un tipo de actividad distinta dentro del flujo, permitiendo identificar visualmente las operaciones, transportes, demoras, inspecciones y archivos que componen cada proceso.

Simb.	Tipo	Descripcion	Uso en este documento
◉	OPERACION	Actividad que produce, modifica o transforma un elemento del proceso	*Ver cursogramas Caps. 1-4*
→	TRANSPORTE	Traslado fisico o digital de informacion, producto o documento	*Ver cursogramas Caps. 1-4*
▽	DEMORA	Espera o inactividad entre pasos del proceso	*Ver cursogramas Caps. 1-4*
□	ARCHIVO	Almacenamiento o guarda de informacion o producto	*Ver cursogramas Caps. 1-4*
◇	INSPECCION	Verificacion, control de calidad o punto de decision	*Ver cursogramas Caps. 1-4*
✱	COMBINADA	Operacion e inspeccion realizadas de manera simultanea	*Ver cursogramas Caps. 1-4*

2.3.1 Proceso 1: Producción avícola diaria

El proceso de produccion es el nucleo operativo de la empresa. Se inicia cada manana con la puesta en marcha de la cinta recolectora automatica que concentra los huevos producidos durante la noche y la manana. El operario realiza el conteo manual, clasifica por calidad, anota en planilla de papel y luego traslada la informacion a la oficina para que el supervisor la transcribar al Excel semanal.

Paso	Actividad	Responsable	Tiempo actual	Problema identificado
1	Activar cinta recolectora automatica Cattaneo	Operario	5 min	Sin problemas
2	Contar huevos manualmente por bandeja	Operario	12 min	Error estimado +-50 huevos/jornada
3	Anotar produccion en planilla de papel	Operario	5 min	Riesgo de extravío o deterioro
4	Contar y retirar aves muertas del dia	Operario	5 min	Sin comparacion con umbrales
5	Trasladar planilla a la oficina	Operario	10 min	Demora innecesaria, riesgo extravío
6	Acumular datos durante toda la semana	Supervisor	5-7 dias	Informacion desactualizada
7	Transcribir datos a Excel semanal	Supervisor	20 min	Doble carga, riesgo de errores
8	Calcular % postura manualmente	Supervisor	15 min	Solo 1 vez/semana — errores aritmeticos
9	Comparar con tabla genetica del fabricante	Supervisor	20 min	Manual, 1 vez/semana — muy tarde

2.3.1.1 Cursograma de la situación actual

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
PROCESO: Registro Diario de Produccion | SITUACION ACTUAL | Operario de Galpon + Supervisor					
1	◉	Finalizacion de recoleccion diaria (cinta automatica Cattaneo) *Proceso mecanico, sin problemas*	Operario	8 min	
2	◉	Conteo MANUAL de huevos por bandeja *Error estimado: +-50 huevos por jornada*	Operario	12 min	
3	□	Anotacion en planilla de PAPEL *Riesgo de extravío o deterioro del registro*	Operario	5 min	
4	◉	Conteo de aves muertas del dia *Sin comparacion con umbrales de alerta*	Operario	3 min	
5	→	Traslado fisico de planilla a oficina *Demora innecesaria, riesgo de extravío*	Operario	10 min	
6	▽	**Acumulacion y espera de datos hasta fin de semana** *Informacion desactualizada 5-7 dias*	—	5-7 dias	
7	◉	**Transcripcion manual a planilla Excel** *Doble carga de datos — riesgo de errores de tipeo*	Supervisor	20 min	
8	✱	**Calculo manual del % de postura** *Solo 1 vez/semana, puede contener errores aritmeticos*	Supervisor	15 min	
9	◇	**Comparacion manual con tabla genetica del fabricante** *Se realiza 1 vez/semana — muy tarde para actuar*	Supervisor	20 min	

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
10	☐	Archivo fisico en bibliorato *Busqueda de historicos: 10-20 minutos*	Supervisor	5 min	
TOTAL SITUACION ACTUAL: ~98 minutos/semana | 4 actividades manuales redundantes | Frecuencia: diaria + revision semanal					

Cursograma 2.1 — Registro diario de produccion: situacion actual

2.3.2 Proceso 2: Gestión comercial y facturación

La gestion comercial se inicia con la solicitud de pedido del cliente, generalmente por telefono o WhatsApp. El vendedor anota el pedido en Excel, consulta manualmente el saldo de cuenta corriente del cliente en otra hoja, prepara el remito en papel, realiza la entrega y luego el contador emite la factura electronica en un sistema separado. Todo el proceso puede demorar hasta 48 horas desde el pedido hasta la factura.

Paso	Actividad	Responsable	Tiempo actual	Problema identificado
1	Recibir pedido del cliente (telefono/WhatsApp)	Vendedor	5 min	Sin registro formal del pedido
2	Anotar pedido en Excel	Vendedor	5 min	Sin validacion de credito ni stock
3	Consultar saldo de cta. corriente en Excel	Vendedor	10 min	Archivo desactualizado, busqueda manual
4	Confirmar pedido al cliente verbalmente	Vendedor	5 min	Sin constancia escrita
5	Preparar y cargar pedido en camion	Deposito	20 min	Sin lista de carga digital
6	Entregar y obtener firma en remito de papel	Repartidor	15 min	Remito puede extraviarse
7	Trasladar remito firmado a oficina	Repartidor	30 min	Demora hasta 24hs para facturar
8	Emitir factura electronica en sistema externo	Contador	20 min	Doble carga — sistema separado
9	Registrar en Excel de cta. corriente	Contador	10 min	Tercer sistema separado — inconsistencias

2.3.2.1 Cursograma de la situación actual

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
PROCESO: Venta y Facturacion | SITUACION ACTUAL | Vendedor + Supervisor + Contador					
1	◉	Cliente solicita pedido (telefono / WhatsApp) *Canal informal, sin registro sistematico*	Vendedor	5 min	
2	□	**Vendedor anota pedido en Excel o papel** *Sin validacion de stock ni limite de credito*	Vendedor	5 min	
3	◉	**Consulta manual del saldo en Excel de cta. corriente** *Busqueda manual, propenso a errores de version*	Vendedor	10 min	
4	→	**Llamado al supervisor para consulta si supera limite** *Demora operativa, depende de disponibilidad*	Vendedor	15 min	
5	◉	Confirmacion verbal del pedido al cliente *Sin constancia escrita del pedido confirmado*	Vendedor	5 min	
6	◉	Preparacion y carga del pedido en el camion *Sin lista de carga digital*	Deposito	20 min	
7	✱	Entrega al cliente y firma de remito MANUAL en papel *Remito puede extraviarse antes de facturar*	Repartidor	15 min	
8	→	**Traslado del remito firmado a la oficina** *Demora hasta 24 hs para procesar la factura*	Repartidor	30 min	
9	◉	**Emision manual de factura electronica (sistema AFIP separado)** *Doble carga de datos — sistema externo independiente*	Contador	20 min	
10	□	**Registro manual en Excel**	Contador	10 min	

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
		de cuenta corriente *Excel separado, riesgo de inconsistencia de saldos*			
11	→	Envio manual de factura PDF por email al cliente *Proceso manual — puede olvidarse o demorar*	Contador	5 min	
TOTAL SITUACION ACTUAL: ~140 minutos/pedido | 4 personas involucradas | Demora total: hasta 48 hs					

Cursograma 3.1 — Gestion de venta y facturacion: situacion actual

2.3.3 Proceso 3: Monitoreo ambiental IoT

Los sensores IoT del galpon envian datos continuamente a la plataforma movil 'Gallinero'. Sin embargo, el acceso a esa informacion requiere que el responsable abra la app manualmente para verificar el estado de cada variable. No existen alertas automaticas configuradas, lo que significa que anomalias criticas pueden pasar horas o incluso dias sin ser detectadas.

Variable	Valor critico relevado	Fecha	Estado de alerta	Consecuencia sin alerta
Temperatura Sensor 1	30,0°C (limite critico)	11/12	Sin alerta	Estres termico de las aves
Humedad Sensor 1	43,7% (bajo minimo 45%)	04/12	Sin alerta	Afecta confort y postura
Flujo de agua	0 litros (corte total)	11/12 02:21	Sin alerta	Deshidratacion de aves
Viabilidad LSL-Classic	67,3% (tabla: 93,6%)	Ciclo completo	Sin alerta	Perdida de 870+ aves

2.3.3.1 Cursograma de la situación actual

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
PROCESO: Monitoreo Ambiental IoT | SITUACION ACTUAL | Operario / Supervisor					
1	◉	Sensor IoT mide variable ambiental cada 5 minutos *Sensores funcionando correctamente*	Sensor	Continuo	
2	→	**Dato enviado a plataforma 'Gallinero' (app separada)** *Informacion aislada, no integrada al sistema de gestion*	Sistema	< 1 seg	
3	◇	**Responsable abre la app MANUALMENTE para consultar** *Sin alertas automaticas — deteccion por consulta periodica*	Operario	5-10 min	
4	◉	**Deteccion visual de anomalia en grafica de la app** *Puede pasar horas o dias sin deteccion*	Operario	Variable	
5	→	Llamado verbal al supervisor para informar la anomalia *Comunicacion informal, sin registro del evento*	Operario	10 min	
6	◉	Supervisor toma accion correctiva *Sin protocolo definido — accion discrecional*	Supervisor	Variable	
7	▽	**Dato ambiental NO se cruza con datos de produccion** *Informacion valiosa desperdiciada sin correlacion*	—	—	
8	□	**Sin historico unificado de variables ambientales** *Datos se pierden, no son comparables entre periodos*	—	—	
CASO CRITICO REAL: Lote LSL-Classic — Viabilidad 67,3% vs tabla 93,6% — NO detectado a tiempo por falta de alertas					

Cursograma 4.1 — Monitoreo ambiental IoT: situacion actual

2.3.4 Proceso 4: Control sanitario

El control sanitario es realizado por un veterinario externo que visita el establecimiento de manera periodica (mensual o ante llamado del propietario). No existe un sistema automatico de seguimiento de la mortalidad acumulada ni de comparacion con curvas de viabilidad estandar, lo que implica que la deteccion de problemas sanitarios graves depende de la observacion visual del operario y de la frecuencia de las visitas del veterinario.

2.3.4.1 Cursograma de la situación actual

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
PROCESO: Control de Mortalidad y Viabilidad | SITUACION ACTUAL | Operario + Supervisor					
1	◉	Operario retira aves muertas del dia *Proceso fisico obligatorio, correcto*	Operario	5 min	
2	□	Anotacion del numero de bajas en planilla *Solo numero, sin calculo de % de mortalidad*	Operario	2 min	
3	▽	**Acumulacion de datos durante TODA la semana** *El problema puede crecer 7 dias sin que nadie lo detecte*	—	7 dias	
4	✱	**Calculo manual de mortalidad semanal en Excel** *Solo si el supervisor recuerda realizarlo*	Supervisor	15 min	
5	◇	**Comparacion visual manual con curvas del fabricante** *Manual, basada en graficos del software Lohmann externo*	Supervisor	20 min	
6	▽	**Sin protocolo de escalada ante mortalidad elevada** *No existe flujo definido de respuesta ante alertas*	—	—	
7	→	Informe al veterinario (mensual y si se recuerda) *Informe manual con datos incompletos*	Supervisor	30 min	
8	□	Sin proyeccion del momento optimo de salida del lote *No se anticipa el fin de ciclo productivo*	—	—	
CASO REAL LSL-CLASSIC: Viabilidad 67,3% vs tabla 93,6% — Diferencia: -26,3pp — No detectada a tiempo					

Cursograma 5.1 — Control de mortalidad y viabilidad: situacion actual

2.4 Tecnología

2.4.1 Tecnología existente en la empresa

2.4.1.1 Infraestructura Tecnológica Actual

Componente	Descripcion	Estado	Integracion actual
Red de sensores IoT	Sensores de temperatura, humedad, CO2 y flujo de agua instalados en el galpon	Operativo	Solo con app Gallinero — aislada
Plataforma Gallinero	App movil que muestra datos de sensores en graficas de serie temporal	Operativo	No integrada con gestion
Smartphone operario	Android — usado para consultar app Gallinero	Activo	Solo lectura de datos IoT
PC de escritorio	Windows — usada para Excel y sistema de facturacion	Activo	Excel + facturacion separada
Sistema de facturacion	Software externo para emision de comprobantes AFIP	Activo	Sin integracion con produccion
Planillas Excel	Registro de produccion semanal y cuenta corriente de clientes	En uso	Sin conexion entre ellas
Planillas de papel	Registro diario de produccion en galpon	En uso	Transcripcion manual semanal
Conexion a internet	Disponible con posibles cortes en zona rural	Variable	Cada sistema por separado

2.4.1.2 Problemas Tecnológicos Identificados

- Cuatro herramientas completamente desconectadas entre si: planilla de papel, Excel de produccion, sistema de facturacion externo y app Gallinero. No existe flujo automatico de informacion entre ellas.
- Los datos de los sensores IoT (app Gallinero) nunca se cruzaron con los datos de produccion, impidiendo identificar correlaciones entre condiciones ambientales y variaciones en la postura.
- El sistema de facturacion externo obliga a reingresar datos que ya existen en el Excel de pedidos, generando doble carga y riesgo de inconsistencias.
- No existe backup automatico de ninguna de las herramientas actuales: si el Excel se corrompe o se pierde, se pierden meses de datos de produccion y cuenta corriente.
- La plataforma Gallinero no tiene configuracion de alertas automaticas, por lo que el responsable debe revisar activamente la app para detectar anomalias.

2.4.1.3 Plataforma 'Gallinero' — Capturas del Sistema Relevado

A continuacion se presentan las capturas del sistema IoT 'Gallinero' tomadas durante el relevamiento en dos fechas distintas (4 y 11 de diciembre), que muestran el comportamiento de las variables ambientales y los eventos criticos que ocurrieron sin generar alertas automaticas.

2.4.1.3.1 Relevamiento del 4 de diciembre

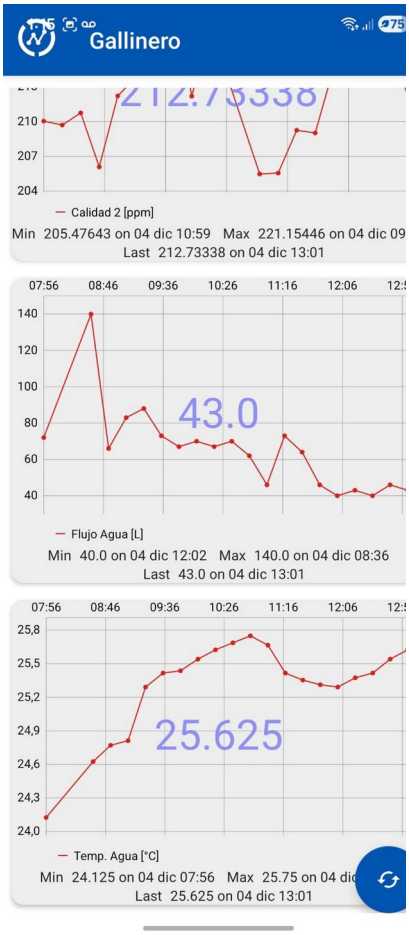


Fig. 2: Temp.2 / Hum.2 / Calidad 2 (04/12)

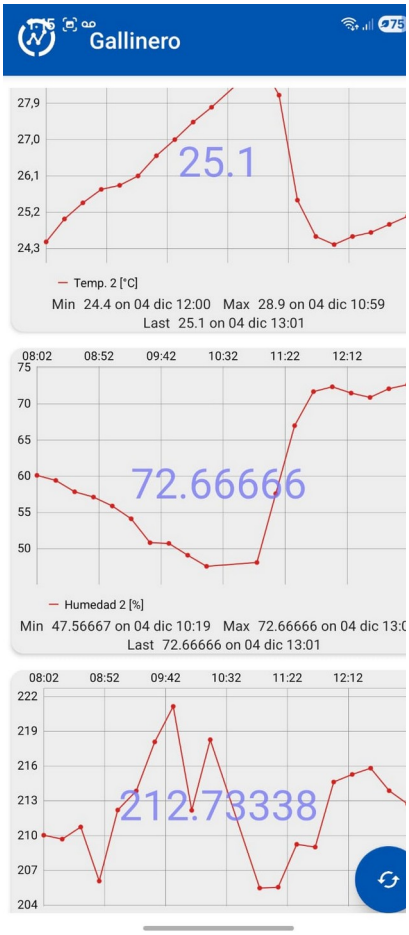


Fig. 3: Temp.1 / Hum.1 (04/12)

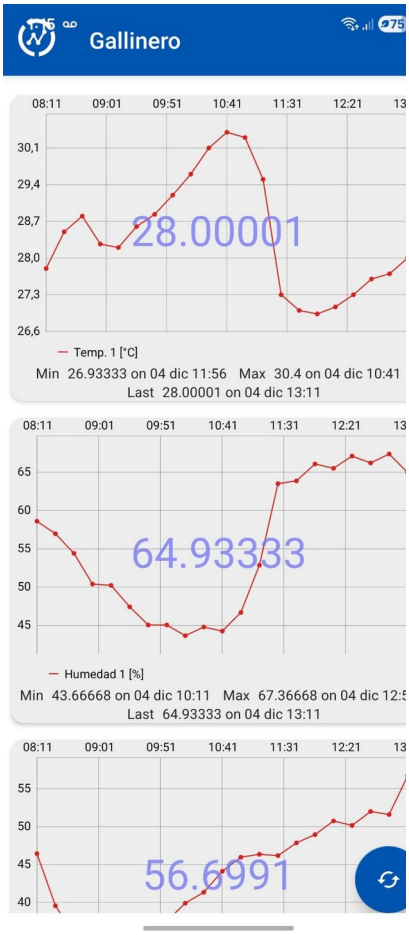


Fig. 4: Calidad 2 / Flujo Agua (04/12)

2.4.1.3.2 Relevamiento del 11 de diciembre



Fig. 5: Vista combinada (04/12)

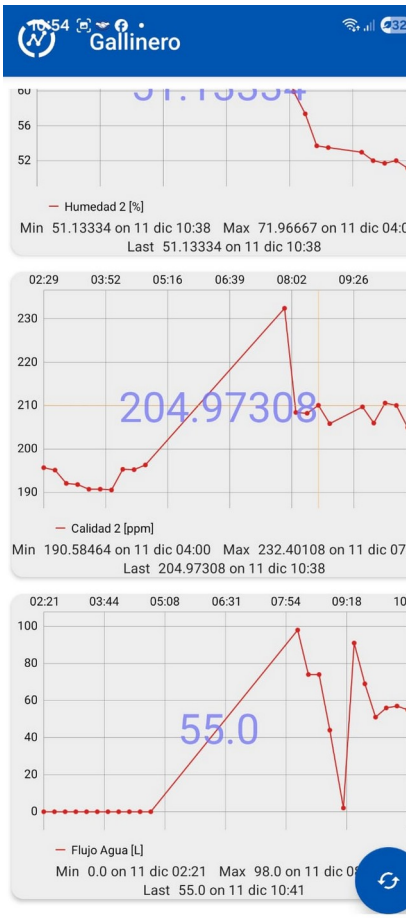


Fig. 6: Calidad 1 / Temp.2 / Hum.2 (11/12)



Fig. 7: Temp.1 / Hum.1 (11/12)

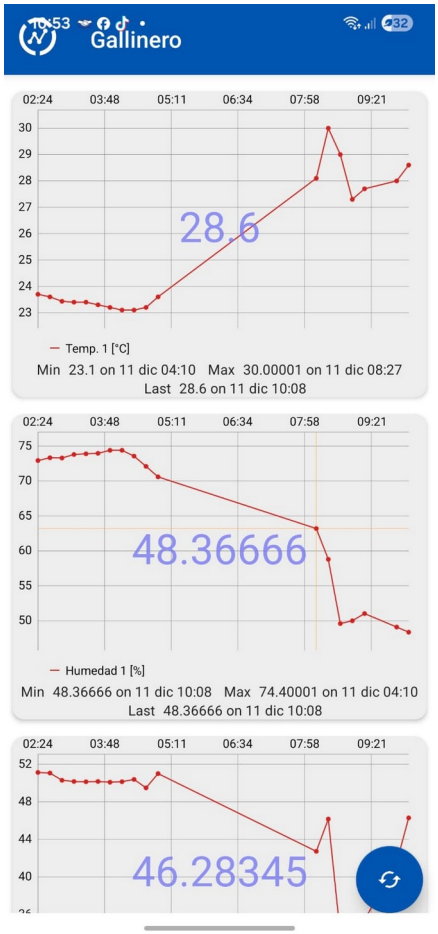


Fig. 8: Hum.2 / Calidad 2 / Flujo Agua (11/12)

2.4.2 Tecnología existente en el mercado (aplicable)

Se relevaron soluciones comerciales de gestión avícola que resuelven problemas similares a los de TILCOR, además de las dos herramientas que la empresa ya utiliza. El objetivo es ubicar la propuesta frente a lo que ya existe y justificar el desarrollo a medida.

Solución	Origen	Qué ofrece (según el fabricante)	Relación con TILCOR
BigFarmNet	Big Dutchman	Software integral de gestión avícola: clima del galpón, datos de producción, resúmenes en gráficos y listas, registro de alarmas e historial. Acceso desde la computadora del galpón, una PC o un dispositivo móvil, y continúa operando si cae la red.	Solución de mercado de referencia. Está pensada para integrarse con equipamiento del mismo fabricante.
Amino	MTech Systems	Software de gestión de gallinas ponedoras: seguimiento unificado de costos y producción (de recría a postura), métricas como costo por huevo y conversión alimenticia, tableros de tendencias, predicciones asistidas por IA y registros de salud con datos de sensores IoT.	Solución de mercado de referencia. Enfocada en costos y optimización de la producción.
Gallinero	Plataforma existente en TILCOR	Plataforma donde hoy se visualizan las variables ambientales de los sensores instalados (capturas relevadas en 2.4.1.3).	Se mantiene como fuente de datos; el nuevo sistema consume el flujo de sensores y agrega alertas y correlación con la producción.
Don Gestión	Sistema existente en TILCOR	Sistema de gestión comercial de la empresa: ventas, clientes, cuentas y facturación.	No se reemplaza: se integra por iframe (CU23) y se aprovecha para facturación (CU24).

Análisis y alcance de este relevamiento

- Las soluciones de mercado cubren funciones similares a algunas de las que se proponen (monitoreo, registro de producción, alarmas), pero son productos cerrados de fabricantes de equipamiento o plataformas de gestión de grandes operaciones.
- El sistema propuesto se diferencia porque está **hecho a medida del flujo de TILCOR**: carga de producción de 5 campos desde el celular, comparación contra la tabla genética Lohmann, alertas de mortalidad con umbrales propios, integración con Gallinero y Don Gestión y facturación electrónica ARCA/AFIP.
- En esta instancia **no se evaluaron costos de licencia ni disponibilidad comercial en Argentina** de las soluciones citadas; queda como trabajo futuro consultar a distribuidores locales y cotizar para completar la comparación.

Fuentes consultadas

Big Dutchman. BigFarmNet — controles y software ambiental. https://www.bigdutchmanusa.com/egg-production/products/environmental/controls-and-software/bigfarmnet/

MTech Systems. 6 Direct Benefits of Poultry Layer Management Software (Amino). https://test.mtech-systems.com/?p=20248

3 Propuesta

3.1 Diagnóstico y propuesta

3.1.1 Resumen del Diagnóstico

A partir del relevamiento organizacional, estructural, funcional, tecnologico y del negocio, se elabora el siguiente diagnostico de la situacion actual de la empresa avicola. El diagnostico identifica los problemas criticos que justifican el desarrollo del sistema propuesto y cuantifica el impacto de cada uno sobre la operacion.

3.1.2 Problemas Críticos Identificados

1 — FRAGMENTACION DE LA INFORMACION

Causa: Cuatro herramientas desconectadas (planilla papel, Excel produccion, Excel cta. corriente, sistema facturacion, app Gallinero) sin flujo de datos entre ellas.

Impacto: Doble y triple carga de datos, inconsistencias entre sistemas, imposibilidad de correlacionar datos ambientales con produccion.

2 — AUSENCIA DE ALERTAS AUTOMATICAS

Causa: La plataforma Gallinero no tiene alertas configuradas. El monitoreo IoT depende de que el responsable abra la app manualmente.

Impacto CRITICO: Temperatura 30°C, humedad 44,2% (bajo minimo) y corte de agua sin notificacion. Viabilidad LSL en 67,3% (-26,3pp vs tabla) no detectada durante todo el ciclo.

3 — TIEMPO OPERATIVO CONSUMIDO EN TAREAS MANUALES DE BAJO VALOR

Causa: Registro en papel, transcripcion a Excel, calculo manual de indicadores, busqueda de saldos y emision de facturas en sistema separado.

Impacto: El supervisor pierde 6-8 horas semanales en tareas administrativas que no agregan valor y que el sistema puede automatizar completamente.

3.1.3 Cuadro Comparativo de Mejoras

Proceso	Situacion actual	Sistema propuesto	Ahorro estimado
Registro produccion diaria	98 min/semana manual	11 min/dia app movil	85%
Calculo % postura	10-15 min manual/semana	Automatico al guardar	100%
Deteccion alerta IoT	Horas o dias (o nunca)	Menos de 2 minutos	99%
Gestion de pedido	140 min/pedido — 4 personas	30 min/pedido — 2 personas	79%
Emision factura electronica	20 min manual — sistema externo	3-4 min automatico AFIP	80%
Reporte semanal de	2-3 hs manual	Automatico cada lunes	100%

Proceso	Situacion actual	Sistema propuesto	Ahorro estimado
produccion			
Control mortalidad/viabilidad	Semanal sin alerta	Diario con umbrales automaticos	Critico

3.1.4 Conclusión del Diagnóstico

El diagnostico confirma que la empresa avicola cuenta con un proceso productivo de alto rendimiento (produccion real superior a tabla en ambas razas) pero carece de las herramientas de informacion necesarias para proteger ese rendimiento, detectar problemas sanitarios y ambientales a tiempo, y gestionar eficientemente la operacion comercial.

La situacion mas critica es la del lote LSL-Classic, cuya viabilidad del 67,3% representa la materializacion mas concreta del riesgo de operar sin alertas automaticas. El sistema propuesto hubiera detectado la caida de mortalidad semanal desde la semana 25 del ciclo, cuando supero el umbral del 0,5%, permitiendo intervencion veterinaria con mas de 60 semanas de anticipacion al cierre del ciclo.

En funcion de este diagnostico, se justifica plenamente el desarrollo del Sistema de Gestion Integral propuesto, cuya implementacion en 5 fases de 26 semanas generara mejoras medibles en todos los procesos analizados.

3.1.5 Situación propuesta de los procesos

Cursogramas de cada proceso con el sistema nuevo, a comparar con los cursogramas de la situación actual de la sección 2.3.

3.1.5.1 Proceso 1: Producción avícola diaria — situación propuesta

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
PROCESO: Registro Diario de Produccion | SISTEMA PROPUESTO | Operario via App Movil					
1	◉	Finalizacion de recoleccion diaria (cinta automatica Cattaneo) *Sin cambios en proceso fisico*	Operario	8 min	
2	◉	Ingreso de datos en app movil — formulario de 5 campos *Validaciones automaticas, no acepta datos invalidos*	Operario	2 min	
3	✱	Sistema calcula % postura automaticamente (GENERATED ALWAYS AS) *Sin intervencion humana — precision 100%*	Sistema	< 1 seg	
4	◇	Sistema compara con tabla genetica Lohmann (diario) *Comparacion cada vez que se guarda un registro*	Sistema	< 1 seg	
5	◉	Si desvio > 5pp: alerta push automatica al supervisor *Notificacion inmediata — accion preventiva posible*	Sistema	< 1 seg	
6	□	Datos en dashboard y historico en tiempo real *Acceso desde cualquier dispositivo*	Sistema	Continuo	
TOTAL SISTEMA PROPUESTO: ~11 minutos/dia | AHORRO: 85% | 4 actividades manuales eliminadas					

Cursograma 2.2 — Registro diario de produccion: sistema propuesto

3.1.5.2 Proceso 2: Gestión comercial y facturación — situación propuesta

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
PROCESO: Venta y Facturacion | SISTEMA PROPUESTO | Vendedor + Sistema Integrado					

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
1	◉	Cliente solicita pedido — vendedor carga en sistema *Registro inmediato, trazable y auditable*	Vendedor	3 min	
2	◇	Sistema valida credito del cliente automaticamente *Consulta v_saldos_clientes en tiempo real*	Sistema	< 1 seg	
3	◉	Si supera limite: alerta push al supervisor *Aprobacion con un click desde cualquier dispositivo*	Sistema	< 1 seg	
4	◉	Sistema genera remito PDF automaticamente *Remito imprimible con codigo QR de verificacion*	Sistema	< 5 seg	
5	◉	Preparacion y entrega con lista de carga digital *Lista impresa desde el sistema, sin transcripcion*	Deposito	20 min	
6	✱	Confirmacion digital de entrega en app movil *Estado del pedido actualizado en tiempo real*	Repartidor	1 min	
7	◉	Sistema emite factura electronica via AFIP-WSFE *CAE obtenido y guardado automaticamente*	Sistema	< 5 seg	
8	□	Sistema actualiza cuenta corriente del cliente *Saldo actualizado sin intervencion humana*	Sistema	< 1 seg	
9	→	Envio automatico de factura PDF por email *Tarea Celery: envio con PDF adjunto*	Sistema	< 5 seg	
TOTAL SISTEMA PROPUESTO: ~30 minutos/pedido | AHORRO: 79% | 5 actividades manuales eliminadas					

Cursograma 3.2 — Gestion de venta y facturacion: sistema propuesto

3.1.5.3 Proceso 3: Monitoreo ambiental IoT — situación propuesta

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
PROCESO: Monitoreo Ambiental IoT | SISTEMA PROPUESTO | Gateway MQTT + Django Channels					
1	◉	Sensor publica dato via protocolo MQTT (QoS 1) *Entrega garantizada al broker Mosquitto*	Sensor	Continuo	
2	→	Gateway MQTT recibe y persiste en TimescaleDB *Hypertable con compresion automatica a 7 dias*	Sistema	< 100 ms	
3	◇	Motor de alertas evalua umbrales configurados *Reglas configurables por galpon y tipo de sensor*	Sistema	< 100 ms	
4	◉	Si hay desvio: clasifica nivel Warning o Critico *Warning: >umbral | Critico: >110% del umbral*	Sistema	< 100 ms	
5	→	Notificacion push / WhatsApp al responsable *Alerta en menos de 2 minutos del evento*	Sistema	< 2 min	
6	◉	Broadcast WebSocket al dashboard en tiempo real *Semaforo verde/amarillo/rojo por sensor*	Sistema	< 1 seg	
7	✱	Correlacion automatica ambiental vs produccion *Job diario: cruza temperatura y humedad vs % postura*	Sistema	Diario	
8	□	Historico completo disponible para analisis *Graficas por rango, exportables a Excel/PDF*	Sistema	Siempre	
TIEMPO DE RESPUESTA PROPUESTO: < 2 minutos desde el evento | AHORRO: 99% en tiempo de deteccion					

Cursograma 4.2 — Monitoreo ambiental IoT: sistema propuesto

3.1.5.4 Proceso 4: Control sanitario — situación propuesta

N°	Simb.	Actividad / Paso	Responsable	Tiempo	Observaciones
PROCESO: Control de Mortalidad y Viabilidad | SISTEMA PROPUESTO					
1	◉	Operario registra aves muertas en app (campo del formulario diario) *Integrado al mismo formulario de produccion*	Operario	3 min	
2	◉	Sistema actualiza existencia y viabilidad acumulada *Campo calculado existencia_actual() — automatico*	Sistema	< 1 seg	
3	✱	Sistema calcula % mortalidad diaria y semanal *Comparacion diaria y semanal automatica*	Sistema	< 1 seg	
4	◇	Si mortalidad semanal > 0,5%: alerta Warning *Push al supervisor: revisar causa de muerte*	Sistema	< 1 seg	
5	◇	Si mortalidad semanal > 1,0%: alerta Critico *Push + WhatsApp: llamar al veterinario de inmediato*	Sistema	< 1 seg	
6	◉	Curva de viabilidad actualizada en dashboard diariamente *Comparacion visual real vs tabla Lohmann*	Sistema	Continuo	
7	→	Reporte PDF semanal automatico enviado al veterinario *Celery: genera y envia cada lunes 08:00 AM*	Sistema	Semanal	
8	◉	Proyeccion del fin de ciclo con tendencia historica *Modelo de regresion: semana optima de salida del lote*	Sistema	Diario	
MEJORA CRITICA: Con el sistema, la caida del lote LSL hubiera generado alerta desde semana 25 — 60+ semanas antes del cierre					

Cursograma 5.2 — Control de mortalidad y viabilidad: sistema propuesto

3.1.6 Diagnóstico de la propuesta

El diagnostico de propuesta consolida el analisis comparativo entre la situacion actual relevada y el sistema propuesto, identificando los problemas criticos detectados, las mejoras cuantificables y el impacto esperado en cada proceso de la empresa avicola.

3.1.6.1 Problemas Identificados en el Relevamiento

A partir del relevamiento de campo, las planillas de produccion, las capturas del sistema IoT 'Gallinero' y los registros comerciales, se identificaron los siguientes problemas criticos que motivan el desarrollo del sistema:

- Registro manual fragmentado: planillas de papel, Excel de produccion, sistema de facturacion y app Gallinero operan de manera completamente aislada, sin flujo de informacion entre si.
- Deteccion tardia de anomalias productivas: el lote LSL-Classic alcanzo una viabilidad del 67,3% vs tabla 93,6% (-26,3pp) sin ser detectado a tiempo por ausencia de alertas automaticas.
- Alertas ambientales inexistentes: eventos criticos reales como temperatura de 30°C, humedad de 44,2% (bajo umbral de 45%) y corte de flujo de agua no generaron ninguna notificacion automatica.
- Doble carga de datos en facturacion: el proceso de emision de facturas electronicas se realiza en un sistema separado, requiriendo re-ingresar datos ya existentes en Excel.
- Sin control de credito en tiempo real: los vendedores no tienen visibilidad inmediata del saldo de cuenta corriente de cada cliente al momento de tomar un pedido.
- Tiempo operativo no productivo: el calculo de indicadores, transcripcion de datos, seguimiento de cuentas y emision de facturas consumen aproximadamente 6-8 horas semanales del supervisor.

3.1.6.2 Comparativo por Proceso

Proceso	Situacion Actual	Sistema Propuesto	Ahorro estimado	Impacto
Registro produccion	15-20 min/dia manual	2-3 min/dia app movil	85%	Alto
Calculo % postura	10-15 min/semana manual	Automatico al guardar	100%	Alto
Deteccion alerta IoT	Horas / No detectado	< 2 minutos automatico	99%	Critico
Gestion de pedido	20-30 min/pedido	5-7 min/pedido	75%	Alto
Emision factura electronica	15-20 min/factura	3-4 min/factura	80%	Alto
Reporte semanal produccion	2-3 hs/semana manual	Automatico cada lunes	100%	Alto
Consulta saldo cliente	5-10 min busqueda manual	Instantaneo en ficha	100%	Medio
Control mortalidad/viabilidad	Semanal sin protocolo	Diario con umbrales	Evita caso LSL	Critico

3.1.6.3 Actividades Eliminadas y Automatizadas por Módulo

Modulo	Pasos actuales	Pasos propuestos	Eliminados	Automatizados
Registro produccion	10	6	-4 (40%)	3 (50%)
Venta y facturacion	11	9	-4 (36%)	5 (56%)
Monitoreo IoT	8	8	0	6 (75%)
Control mortalidad	8	8	0	5 (63%)
TOTAL DEL SISTEMA	37	31	-8 (22%)	19 (61%)

3.1.6.4 Impacto en los Datos Reales Relevados

Aplicando el sistema propuesto sobre los datos concretos relevados de la empresa:

- Lote Brown-Classic (sem. 87-89): produccion acumulada real 414,7 huevos/AA vs tabla 393,5 (+21,2 unidades). El sistema agrega valor automatizando el seguimiento de esta ventaja competitiva y alertando ante cualquier desvio hacia abajo.
- Lote LSL-Classic: viabilidad 67,3% vs tabla 93,6%. Con el sistema propuesto, la mortalidad semanal elevada hubiera disparado la alerta Warning desde la semana 25, cuando supero el umbral del 0,5% semanal, permitiendo intervencion veterinaria 60+ semanas antes del fin del ciclo.
- Datos IoT del 04 y 11 de diciembre: temperatura pico 30°C (limite critico), humedad Sensor 1 en 44,2% (bajo umbral de 45%) y corte de flujo de agua a 0 litros durante la jornada del 11/12. Los tres eventos hubieran generado alertas automaticas en menos de 2 minutos con el sistema propuesto.
- Planilla comercial semana 88: 20.070 huevos en 55 cajones con mortalidad semanal 0,77%. El sistema integra este dato con el stock disponible para comprometer ventas sin exceder la produccion real.

3.1.6.5 Conclusión del Diagnóstico

El diagnostico confirma que la empresa avicola cuenta con un proceso productivo de alto rendimiento (produccion real superior a tabla en ambas razas) pero carece de las herramientas de informacion necesarias para proteger ese rendimiento, detectar problemas a tiempo y gestionar eficientemente la operacion comercial.

El Sistema de Gestion Integral propuesto resuelve los tres problemas criticos identificados: la fragmentacion de la informacion, la ausencia de alertas automaticas y el tiempo operativo dedicado a tareas de transcripcion y calculo manual. La implementacion en 5 fases de 26 semanas garantiza una transicion ordenada y sin impacto en la operacion diaria del galpon.

Autor del Sistema
Analista en Sistemas de Informacion

Director / Tutor de Tesis
Carrera Analista en Sistemas

3.2 Propuesta general

3.2.1 Visión General

El Sistema de Gestion Integral propuesto centraliza en una unica plataforma web todos los procesos operativos, productivos y comerciales de la empresa avicola. La arquitectura adopta un diseno N-Tier con componentes especializados para el manejo de datos IoT en tiempo real.

3.2.2 Módulos del Sistema

Modulo	Descripcion	Usuarios
Dashboard Principal	KPIs en tiempo real: produccion, postura, aves, ventas, estado IoT, alertas activas	Todos
Gestion de Clientes	ABMC clientes, historial pedidos, cta. corriente, limites de credito, listas de precios	Vendedor / Admin
Pedidos y Remitos	Creacion de pedidos con validacion de credito, remitos PDF con QR, confirmacion de entrega	Vendedor / Repartidor
Facturacion AFIP	Emision automatica Fact. A/B/C via WSFE, obtencion CAE, actualizacion cta. corriente	Sistema / Admin
Registro de Produccion	Formulario movil 5 campos, calculo % postura, comparacion tabla genetica, alertas de desvio	Operario / Supervisor
Monitoreo IoT	Dashboard semaforo tiempo real, motor de alertas, historico sensoriales, correlacion con produccion	Supervisor / Propietario
Reportes Automaticos	Diario al propietario, semanal al supervisor, mortalidad al veterinario — PDF y Excel	Sistema

3.2.3 Roles del Sistema

Rol	Modulos accesibles	Permisos
Administrador	Todos	ABMC completo, configuracion, reportes globales
Supervisor Productivo	Produccion, IoT, Reportes	Editar produccion, configurar alertas, ver curvas
Operario de Galpon	Solo registro produccion	Formulario simplificado de su galpon
Vendedor Comercial	Clientes, Pedidos, Facturacion	ABMC clientes, crear pedidos, ver saldos
Veterinario	Produccion, Mortalidad (lectura)	Solo consulta — reportes de salud

3.2.4 Tecnologías Propuestas

Capa	Tecnologia	Version	Justificacion
Backend API	Python / Django REST Framework	4.x	ORM robusto, ecosistema maduro para IoT

Capa	Tecnologia	Version	Justificacion
Frontend Web	React.js + Tailwind CSS	18 / 3.x	SPA responsiva, mobile-first
Base de Datos	PostgreSQL + TimescaleDB	15 / 2.x	Relacional + series temporales IoT
Broker IoT	MQTT (Eclipse Mosquitto)	2.x	Protocolo ligero para sensores
Tiempo real	Django Channels + Redis	4.x / 7.x	WebSocket para dashboard IoT
Tareas async	Celery + Redis	5.x	Reportes, alertas y notificaciones automaticas
Facturacion	AFIP WSFE (SOAP)	Vigente	Integracion legal para comprobantes electronicos
Contenedores	Docker + Docker Compose	24.x	Despliegue reproducible y portable

3.2.5 Plan de Implementación

Fase	Nombre	Semanas	Entregables
1	Fundacion e infraestructura	1 - 4	Docker, DDL PostgreSQL+TimescaleDB, JWT, ABMC galpones y razas
2	Nucleo productivo	5 - 10	Registro produccion, tablas geneticas, dashboard curvas, alertas desvio
3	Clientes y facturacion	11 - 16	ABMC clientes, pedidos, remitos PDF, integracion AFIP, cta. corriente
4	Integracion IoT	17 - 21	Gateway MQTT, motor alertas, dashboard tiempo real (WebSocket)
5	Reportes y cierre	22 - 26	Reportes automaticos, pruebas de aceptacion, capacitacion, produccion

3.2.6 Requerimientos preliminares del relevamiento

3.2.6.1 Requerimientos Funciónales

3.2.6.1.1 Módulo de Clientes

- RF01: Alta, baja logica y modificacion de clientes (razon social, CUIT, condicion IVA, categoria, limite de credito).
- RF02: Historial completo de pedidos y compras por cliente.
- RF03: Listas de precios diferenciadas por categoria de cliente.
- RF04: Emision de remitos PDF y facturas electronicas integradas con AFIP-WSFE.
- RF05: Cuenta corriente con saldo en tiempo real y alerta de limite de credito.

3.2.6.1.2 Módulo de Producción

- RF06: Registro diario de huevos por galpon y lote desde app movil — formulario de 5 campos.
- RF07: Calculo automatico del % de postura al guardar cada registro.

- RF08: Registro de mortalidad diaria y calculo de viabilidad acumulada del lote.
- RF09: Comparacion automatica vs tabla genetica del fabricante con alerta si desvio > 5pp.
- RF10: Alerta Warning si mortalidad semanal > 0,5% y alerta Critico si > 1,0%.

3.2.6.1.3 Módulo de Monitoreo IoT

- RF11: Integracion con sensores existentes via protocolo MQTT.
- RF12: Alertas automaticas push y WhatsApp en menos de 2 minutos ante desviaciones.
- RF13: Historico de datos sensoriales con graficas de serie temporal.
- RF14: Dashboard con semaforo visual en tiempo real (verde/amarillo/rojo).

3.2.6.1.4 Módulo de Reportes

- RF15: Reporte diario automatico enviado al propietario al cierre del dia.
- RF16: Reporte semanal de produccion real vs tabla genetica.
- RF17: Exportacion a Excel y PDF de todos los modulos.

3.2.6.2 Requerimientos No Funciónales

ID	Requerimiento	Descripcion	Nivel
RNF01	Disponibilidad	Modulo IoT disponible 24/7	Critico
RNF02	Usabilidad	Interfaz mobile-first para operarios de galpon	Alto
RNF03	Seguridad	JWT con roles diferenciados por modulo y galpon	Alto
RNF04	Performance	Dashboard IoT actualizado en < 2 segundos	Alto
RNF05	Escalabilidad	Hasta 10 galpones y 50.000 aves	Medio
RNF06	Trazabilidad	Auditoria de toda operacion con usuario y timestamp	Alto
RNF07	Backup	Respaldo automatico diario en la nube	Alto
RNF08	Portabilidad	Funcionamiento en Chrome, Firefox y Edge actuales	Medio

PARTE 2

Modelado y desarrollo del sistema (PP3)

4 Objetivos, límites y alcance
5 Especificación de requerimientos
6 Análisis y diseño del producto
7 Implementación del software
8 Integración y testing
9 Especificaciones del sistema

4 Objetivos, límites y alcance

4.1 Objetivo del sistema

4.1.1 Objetivo General

Desarrollar e implementar un Sistema de Gestión Integral Avícola para la empresa TILCOR S.A.S., con sede en Colonia Tirolesa, Córdoba, que centralice el monitoreo ambiental del galpón mediante sensores IoT en tiempo real, el registro y control de la producción diaria de huevos, el seguimiento de mortalidad y viabilidad de los lotes, y la integración con el sistema de gestión comercial Don Gestión ya existente en la empresa.

4.1.2 Objetivos Específicos

1. Integrar los sensores IoT existentes en el galpón de TILCOR S.A.S. al sistema central mediante el protocolo MQTT, permitiendo la recepción, persistencia y visualización de datos ambientales en tiempo real.
2. Implementar un motor de alertas automáticas que notifique al personal responsable en menos de 2 minutos ante cualquier desvío en los parámetros ambientales configurados (temperatura, humedad, calidad del aire y flujo de agua).
3. Desarrollar un módulo de registro diario de producción avícola con cálculo automático del porcentaje de postura y comparación en tiempo real contra la tabla genética del fabricante Lohmann Tierzucht GmbH.
4. Implementar el control de mortalidad semanal con alertas automáticas por nivel (Warning >0,5% y Crítico >1,0%) y seguimiento de la viabilidad acumulada del lote contra los estándares del fabricante.
5. Desarrollar una Progressive Web App (PWA) instalable en dispositivos móviles para el personal operativo del galpón, con soporte offline.
6. Integrar el sistema con Don Gestión mediante iframe directo para el acceso unificado a ventas, clientes, productos, cuentas y reportes comerciales desde una sola interfaz.
7. Implementar un sistema de autenticación con roles diferenciados (Admin y Empleado) que controle el acceso a los módulos del sistema según el perfil de cada usuario.

4.2 Límite del sistema

4.2.1 Definición del Límite

El límite del sistema define la frontera entre lo que el Sistema de Gestión Integral TILCOR gestiona directamente y lo que pertenece a sistemas externos con los cuales se integra o interactúa.

4.2.2 Dentro del límite del sistema

Módulo	Descripción	Responsable
Dashboard principal	KPIs en tiempo real, alertas activas, viabilidad,	Sistema TILCOR

Módulo	Descripción	Responsable
	resumen comercial, plan de implementación	
Dashboard IoT	Visualización en tiempo real de sensores ambientales con semáforo visual	Sistema TILCOR
Motor de alertas	Evaluación de umbrales y envío de notificaciones push/WhatsApp en < 2 minutos	Sistema TILCOR
Registro de producción	Formulario móvil diario de carga de huevos, cajones, maples, muertas y existencia	Sistema TILCOR
Cálculo de indicadores	% postura, desvío vs tabla genética, mortalidad semanal, viabilidad acumulada	Sistema TILCOR
Control de mortalidad	Seguimiento de bajas diarias, curva histórica y alertas sanitarias	Sistema TILCOR
PWA móvil	Aplicación instalable con soporte offline y Service Worker	Sistema TILCOR
Persistencia IoT	Almacenamiento de series temporales de sensores en TimescaleDB	Sistema TILCOR
Autenticación y roles	Login con roles Admin/Empleado y control de acceso por módulo	Sistema TILCOR
Reportes automáticos	Generación y envío automático de reportes diarios y semanales	Sistema TILCOR

4.2.3 Fuera del límite del sistema

Elemento	Sistema responsable	Integración
Gestión de ventas y pedidos	Don Gestión (ERP externo)	Iframe embebido
Facturación electrónica ARCA/AFIP	Don Gestión + ARCA	Sin integración directa
Cuentas corrientes y cobros	Don Gestión	Iframe embebido
Hardware de sensores IoT	Proveedor externo	MQTT (entrada de datos)
Gestión de RRHH y proveedores	Fuera del proyecto	No aplica

Don Gestión sigue siendo el ERP principal de TILCOR S.A.S. para toda la gestión comercial. El sistema desarrollado en esta tesis lo complementa cubriendo los procesos avícolas y ambientales que Don Gestión no contempla.

4.3 Alcance del sistema

4.3.1 Alcance Funciónal

4.3.1.1 Módulo de Monitoreo Ambiental IoT

- Recepción de datos de sensores via protocolo MQTT con broker Eclipse Mosquitto (temperatura x2, humedad x2, CO2 x2, flujo de agua).

- Persistencia de series temporales en TimescaleDB con compresión automática a los 7 días.
- Dashboard en tiempo real con semáforo visual actualizado vía WebSocket en menos de 2 segundos.
- Motor de alertas con clasificación Warning / Crítico y notificación push en < 2 minutos.
- Historial de datos sensoriales con gráficas por rango de fechas.

4.3.1.2 Módulo de Registro de Producción

- Formulario de carga diaria optimizado para celular: fecha, semana, huevos, cajones, maples, muertas, existencia.
- Cálculo automático del % de postura y comparación contra tabla genética Lohmann.
- Alerta automática al supervisor cuando el desvío supera los 5pp por debajo de la tabla.
- Historial de los últimos 7 registros con indicador de desvío en puntos porcentuales.

4.3.1.3 Módulo de Sanidad y Mortalidad

- Cálculo automático de mortalidad semanal acumulada con umbrales configurables.
- Alerta Warning > 0,5% semanal y alerta Crítico > 1,0% semanal.
- Gráfica histórica de mortalidad semanal (semanas 76-89) con línea de tendencia.
- Envío automático de informe PDF semanal al veterinario externo.

4.3.1.4 Módulo de Autenticación y Roles

- Login con usuario y contraseña. Dos roles: Admin (acceso completo) y Empleado (sin acceso al módulo Ventas).
- Sesión persistida en sessionStorage durante la sesión activa.
- Botón de cierre de sesión con menú desplegable desde el avatar del topbar.

4.3.2 Alcance Técnico

Dimensión	Alcance
Plataforma	Aplicación web responsive + PWA instalable en Android e iOS
Dispositivos	Desktop (Chrome, Firefox), Android (Chrome), iOS (Safari)
Conectividad	Online con soporte offline vía Service Worker
Galpones	Galpón 1 — Lote F1 (escalable a múltiples galpones)
Razas	Lohmann Brown-Classic y Lohmann LSL-Classic
Despliegue	Vercel (frontend) + servidor Linux con Docker (backend)
Retención IoT	Ilimitada con compresión automática en TimescaleDB

5 Especificación de requerimientos

5.1 Requerimientos Funciónales — Sistema Ambiental IoT

ID	Requerimiento	Descripción	Prioridad
RF-IOT-01	Recepción MQTT	Recibir datos de sensores via MQTT con QoS 1 a través de Eclipse Mosquitto.	Crítica
RF-IOT-02	Persistencia TimescaleDB	Almacenar cada dato en hypertable particionada por timestamp con compresión a 7 días.	Crítica
RF-IOT-03	Dashboard tiempo real	Actualizar dashboard en < 2 segundos via WebSocket (Django Channels).	Alta
RF-IOT-04	Semáforo visual	Mostrar verde/amarillo/rojo por sensor según su estado respecto a los umbrales.	Alta
RF-IOT-05	Motor de alertas	Evaluar cada dato contra umbrales y clasificar como Warning o Crítico automáticamente.	Crítica
RF-IOT-06	Notificación < 2 min	Enviar notificación push al responsable en máximo 2 minutos desde el evento.	Crítica
RF-IOT-07	Config. umbrales	Permitir configurar min/max de alerta por sensor desde el panel de administración.	Alta
RF-IOT-08	Histórico	Consultar historial de cualquier sensor en cualquier rango de fechas con gráfica.	Media
RF-IOT-09	Log de alertas	Registrar todas las alertas con timestamp, sensor, valor, umbral, nivel y estado.	Alta
RF-IOT-10	Correlación amb-prod	Proceso diario que cruza datos ambientales con el % de postura del día.	Media

5.2 Requerimientos Funciónales — Producción y Sanidad

ID	Requerimiento	Descripción	Prioridad
RF-PROD-01	Registro diario móvil	Formulario de 5 campos con teclado numérico optimizado para celular.	Crítica
RF-PROD-02	Cálculo % postura	Calcular automáticamente (huevos / existencia × 100) al guardar cada registro.	Crítica
RF-PROD-03	Comparación tabla	Comparar % de postura real vs tabla genética Lohmann para la semana de edad.	Alta
RF-PROD-04	Alerta desvío >5pp	Generar alerta al supervisor si el % de postura real cae más de 5pp bajo la tabla.	Alta
RF-PROD-05	Control mortalidad	Calcular mortalidad semanal y alertar: Warning >0,5%, Crítico >1,0%.	Crítica
RF-PROD-06	Viabilidad acumulada	Mantener actualizada la viabilidad del lote vs tabla del fabricante.	Alta
RF-PROD-	Informe al veterinario	Enviar automáticamente cada lunes a las	Media

ID	Requerimiento	Descripción	Prioridad
07		08:00 informe PDF de mortalidad.	
RF-PROD-08	Historial 7 días	Mostrar últimos 7 registros con fecha, indicadores y desvío en pp.	Alta

5.3 Requerimientos No Funciónales

ID	Categoría	Requerimiento	Criterio de aceptación
RNF-01	Disponibilidad	Módulo IoT disponible 24/7	Uptime ≥ 99,5% mensual
RNF-02	Performance	Dashboard IoT actualizado en < 2s	Tiempo ≤ 2s en red 4G
RNF-03	Usabilidad	Interfaz operable con una sola mano en celular	Todos los elementos accesibles con pulgar
RNF-04	Offline	Funcionar sin conexión con datos de última sesión	Service Worker con caché activa
RNF-05	Seguridad	Autenticación con roles y transmisión cifrada	JWT + HTTPS obligatorio
RNF-06	Escalabilidad	Soportar hasta 10 galpones sin rediseño	Arquitectura modular verificada
RNF-07	Portabilidad	PWA instalable en Android e iOS	Verificación en Android e iOS pendiente
RNF-08	Backup	Respaldo automático diario	Backup 03:00 AM con retención 30 días
RNF-09	Trazabilidad	Audit log de toda operación de escritura	Log con usuario, timestamp, datos anteriores
RNF-10	Alertas	Notificación en < 2 min desde el evento	Tiempo ≤ 120 segundos en pruebas

6 Análisis y diseño del producto

6.1 Modelado ambiental

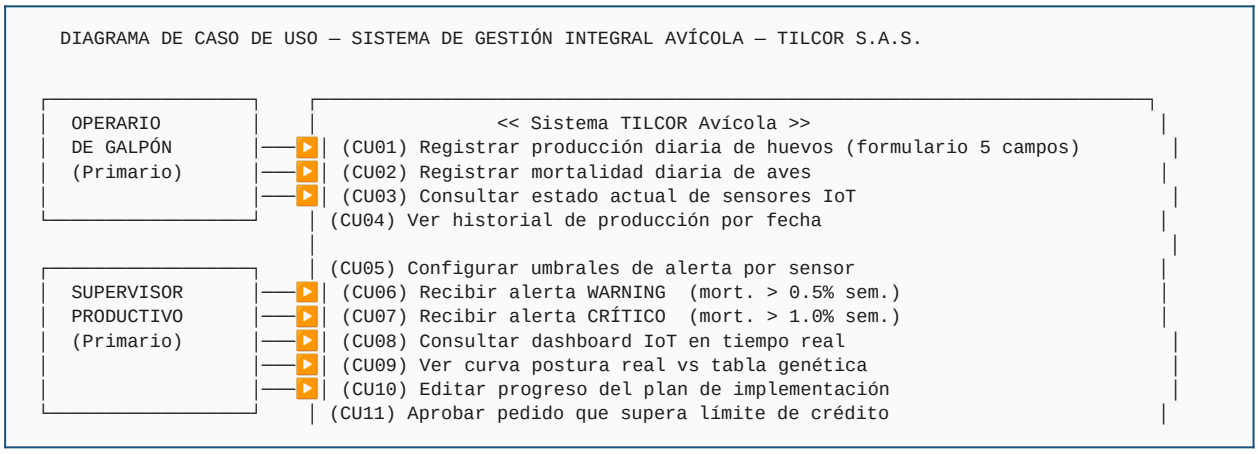
6.1.1 Descripción General

El diagrama de caso de uso ambiental describe las interacciones entre los actores del sistema y las funcionalidades del Sistema de Gestión Integral Avícola de TILCOR S.A.S. Se identifican 5 actores (Operario de Galpón, Supervisor Productivo, Propietario, Veterinario Externo y Sensores IoT/Don Gestión como sistemas automáticos) y 24 casos de uso distribuidos en 4 módulos funcionales.

6.1.2 Actores del Sistema

Actor	Tipo	Rol en el sistema	CU asociados
Operario de Galpón	Primario — Humano	Carga el registro diario desde el celular en el galpón. Consulta estado de sensores.	CU01-CU04
Supervisor Productivo	Primario — Humano	Configura alertas, analiza indicadores, toma decisiones operativas.	CU05-CU11
Propietario	Secundario — Humano	Consulta KPIs ejecutivos y recibe reportes automáticos diarios.	CU12-CU15
Veterinario Externo	Secundario — Humano	Recibe informe semanal automatizado y consulta historial sanitario.	CU16-CU18
Sensores IoT (SA)	Primario — Sistema Auto.	Publican datos ambientales cada 5 min via MQTT.	CU19-CU22
Don Gestión (SE)	Secundario — Sistema Ext.	ERP de TILCOR para gestión comercial y facturación.	CU23-CU24

6.1.3 Diagrama de Caso de Uso



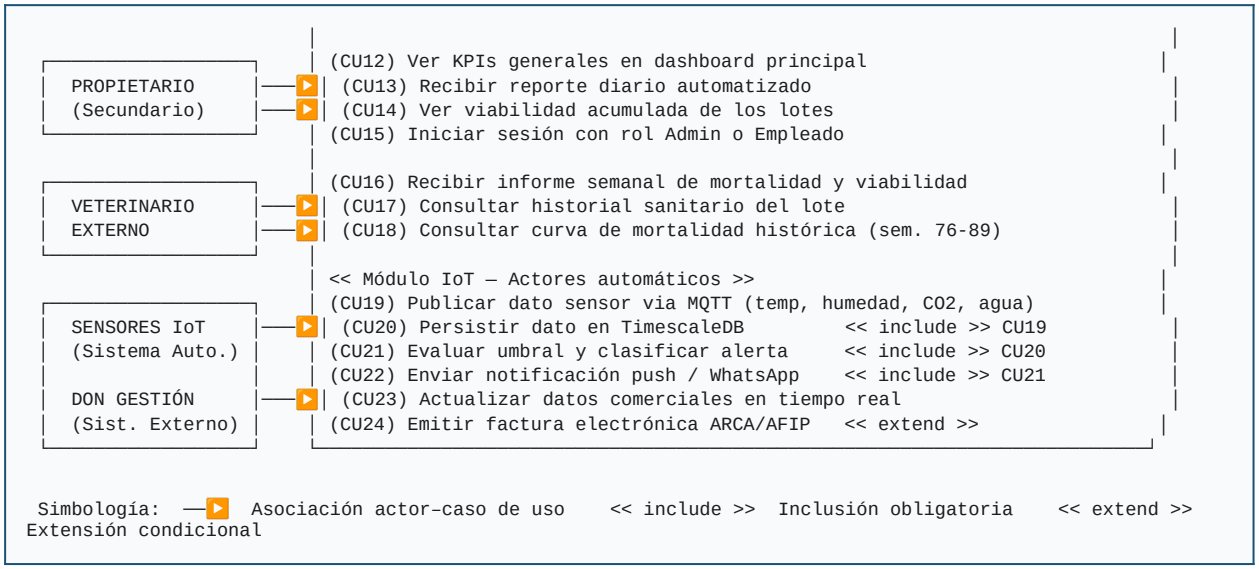


Diagrama 5.1 — Caso de uso del Sistema de Gestión Integral Avícola TILCOR S.A.S. (24 CU · 5 actores)

6.1.4 Especificaciones Detalladas de Casos de Uso Clave

CU01 — Registrar producción diaria

ID / Nombre	CU01 — Registrar producción diaria de huevos
Actores	Operario de Galpón
Precondición	Finalizó la recolección de huevos del día con la cinta Cattaneo. App TILCOR activa en el celular.
Flujo principal	1. El operario abre la sección Producción de la app. 2. El sistema presenta el formulario: fecha, semana, huevos, cajones, maples, muertas, existencia. 3. El operario completa los campos con teclado numérico. 4. El sistema calcula el % de postura en tiempo real mientras el operario escribe. 5. El operario toca 'Guardar registro'. 6. El sistema compara el % calculado vs tabla genética Lohmann para la semana de edad. 7. Si desvío > -5pp: modal de alerta con opción de notificar al supervisor. 8. El sistema guarda el registro y actualiza el historial de los últimos 7 días.
Postcondición	Registro guardado. Historial actualizado. Si hubo desvío crítico, supervisor notificado.
Flujo alternativo	Si existencia = 0: error de validación, no se guarda.
Prioridad	Crítica · Caso base — desencadena CU06/07 en caso de desvío

CU06/07 — Recibir alerta Warning / Crítico

ID / Nombre	CU06/07 — Generación y envío de alerta automática
Actores	Sistema TILCOR (Motor de Alertas) · Supervisor Productivo
Precondición	Se registró un dato que supera los umbrales configurados (sensor IoT o desvío de producción).
Flujo principal	1. El motor recibe el dato y los umbrales configurados. 2. Clasifica: Warning si supera umbral; Crítico si supera el 110% del umbral o mort. > 1%. 3. Registra el evento en el log: timestamp, origen, valor, umbral, nivel. 4. Genera mensaje de notificación con descripción y acción recomendada. 5. Envía notificación push al supervisor en < 2 minutos (Celery task). 6. El dashboard actualiza el semáforo del sensor/indicador afectado (amarillo/rojo). 7. El supervisor recibe la alerta y puede marcarla como resuelta.
Postcondición	Alerta registrada en log. Supervisor notificado en < 2 min. Dashboard actualizado.
Flujo alternativo	Sin conexión del supervisor: la notificación se encola y entrega al recuperar conectividad.
Prioridad	Crítica · << include >> CU22 (notificación) · << extend >> de CU01 y CU19-21

CU15 — Iniciar sesión con rol

ID / Nombre	CU15 — Iniciar sesión con rol diferenciado
Actores	Todos los actores humanos
Precondición	El usuario tiene la app TILCOR abierta en la pantalla de login.
Flujo principal	1. El usuario ingresa usuario y contraseña. 2. El sistema verifica las credenciales. 3. Si son correctas: establece sesión con rol (Admin o Empleado). 4. El rol Admin habilita todos los módulos incluyendo Ventas/Don Gestión. 5. El rol Empleado oculta el tab de Ventas y su sección. 6. El nombre del usuario aparece en el avatar del topbar con menú de cierre de sesión.
Postcondición	Sesión activa con rol. Módulos habilitados según permisos del rol.
Flujo alternativo	Credenciales incorrectas: error con animación shake y campo de contraseña limpiado.
Prioridad	Crítica · Requisito base para todos los demás CU

CU19 → CU22 — Flujo automático IoT

ID / Nombre	CU19-22 — Publicación, persistencia, evaluación y alerta IoT

Actores	Sensores IoT (SA) · Broker MQTT · Gateway Django Channels · TimescaleDB
Precondición	Sensores activos conectados a red WiFi del galpón. Broker Mosquitto corriendo. Gateway suscripto.
Flujo principal	1. El sensor mide la variable y publica en JSON via MQTT QoS1: {sensor_id, valor, unidad, timestamp}. 2. El broker distribuye el mensaje al gateway. 3. El gateway valida el payload y persiste en TimescaleDB (CU20). 4. El gateway evalúa el valor contra umbrales configurados (CU21). 5. Sin desvío: transmite al dashboard via WebSocket y finaliza. 6. Con desvío: ejecuta CU06 o CU07 según nivel, que a su vez ejecuta CU22. 7. CU22: notificación push y/o WhatsApp al supervisor en < 2 minutos.
Postcondición	Dato en TimescaleDB. Dashboard actualizado via WebSocket. Supervisor notificado si hubo desvío.
Flujo alternativo	Sin conexión MQTT del sensor: dato perdido para ese ciclo. Timeout registrado en log.
Prioridad	Crítica · CU20 << include >> CU19 · CU21 << include >> CU20 · CU22 << include >> CU21

6.1.5 Relaciones entre casos de uso

Relación	Tipo	Descripción
CU20 incluye CU19	<< include >>	Siempre que se publica un dato MQTT (CU19) se persiste en TimescaleDB (CU20). Obligatorio.
CU21 incluye CU20	<< include >>	Siempre que se persiste un dato (CU20) se evalúa contra umbrales (CU21). Obligatorio.
CU22 incluye CU21	<< include >>	Si CU21 detecta desvío, siempre se envía notificación (CU22). Obligatorio.
CU06 incluye CU22	<< include >>	La alerta Warning siempre incluye envío de notificación push.
CU07 incluye CU22	<< include >>	La alerta Crítico siempre incluye notificación push + WhatsApp.
CU24 extiende CU23	<< extend >>	La facturación ARCA/AFIP se ejecuta opcionalmente al confirmar un pedido en Don Gestión.
CU07 extiende CU06	<< extend >>	Si la alerta Warning alcanza nivel Crítico, CU07 extiende CU06 con acciones adicionales.
CU11 extiende CU01	<< extend >>	La aprobación del supervisor se activa condicionalmente si hay desvío al registrar producción.

6.1.6 Articulación con el relevamiento y la propuesta (Parte 1)

Sección del relevamiento (Parte 1)	Sección de la especificación (Parte 2)	Relación
1.3 Descripción general del proyecto	4.1 Objetivo del sistema	Detalla y formaliza los objetivos
2.1 y 2.2 Relevamiento contextual y estructural	4.2 Límite del sistema	Define la frontera con Don Gestión
3.2.6 Requerimientos preliminares	4.3 Alcance · 5 Especificación de requerimientos	Desarrolla RF y RNF con criterios de aceptación
3.2 Propuesta general	6.1 Modelado ambiental y 6.3 Casos de uso	Documenta las 24 interacciones de los actores con el sistema
3.2.5 Plan de implementación	Capítulos 4 a 9	Sustenta la planificación con especificaciones verificables

La Parte 2 de este documento debe leerse en conjunto con la Parte 1 — Sistema de Gestión Integral para Empresa Avícola TILCOR S.A.S. (Carnielli, M. y Bacari, F., 2024). Juntos conforman la documentación completa del Trabajo Final de Tesis de la carrera Analista en Sistemas de Información del Colegio Universitario IES, Córdoba.

6.2 Modelado de paquetes

El sistema se organiza en cuatro capas y seis paquetes de aplicación, uno por módulo funcional. Cada paquete agrupa los casos de uso que implementa (sección 6.3) y las tablas que usa. La arquitectura y la base de datos de referencia se detallan a continuación.

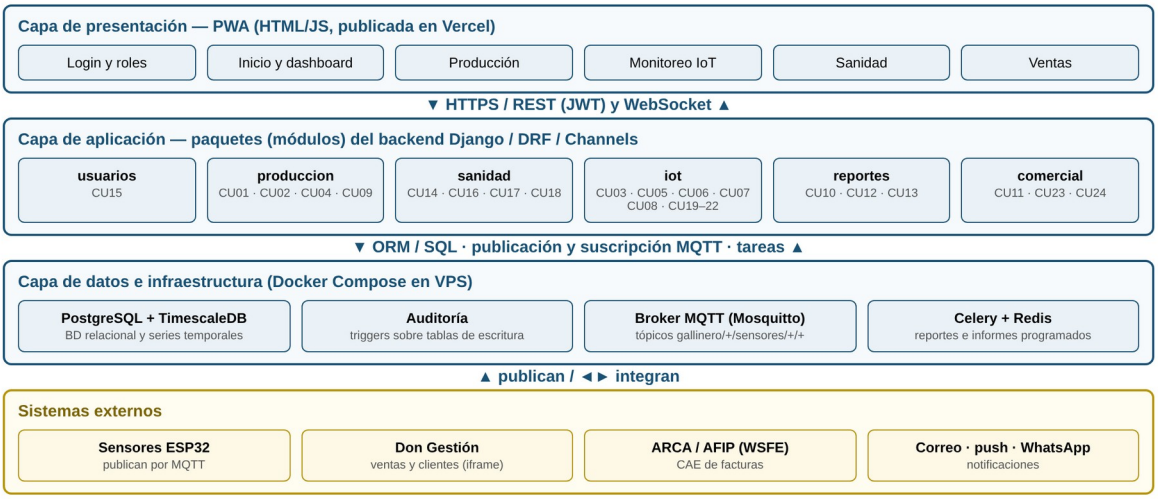


Figura 6.2-1. Diagrama de paquetes del sistema y casos de uso que contiene cada paquete.

Paquete	Responsabilidad	Casos de uso	Tablas principales
usuarios	Autenticación JWT, roles y auditoría de acceso	CU15	usuarios, auditoria
produccion	Registro diario, cálculo de postura y comparación con tabla genética	CU01, CU02, CU04, CU09	lotes, registros_produccion, tabla_genetica, alertas_productivas
sanidad	Mortalidad, viabilidad e informes al veterinario	CU14, CU16, CU17, CU18	lotes, registros_produccion
iot	Ingesta MQTT, evaluación de umbrales, alertas y dashboard en tiempo real	CU03, CU05, CU06, CU07, CU08, CU19–22	sensores, registros_sensor, alertas_iot
reportes	Dashboard principal, reportes programados y plan de implementación	CU10, CU12, CU13	consultas sobre las tablas anteriores
comercial	Clientes, pedidos, cuenta corriente y facturación	CU11, CU23, CU24	clientes, pedidos, cuenta_corriente, facturas

6.2.1 Visión General

El sistema adopta una arquitectura multicapa (N-Tier) que separa presentacion, logica de negocio, datos e integracion IoT. El despliegue se realiza con contenedores Docker sobre Ubuntu 22.04 LTS. La comunicacion entre capas usa APIs REST sobre HTTPS para operaciones de negocio y WebSocket para streaming de sensores en tiempo real.

```
========================================================================
  ARQUITECTURA GENERAL DEL SISTEMA — N-TIER + IoT
========================================================================

  ┌──────────────────────────────────────────────────────────────────┐
  │                    CAPA DE PRESENTACION                          │
```

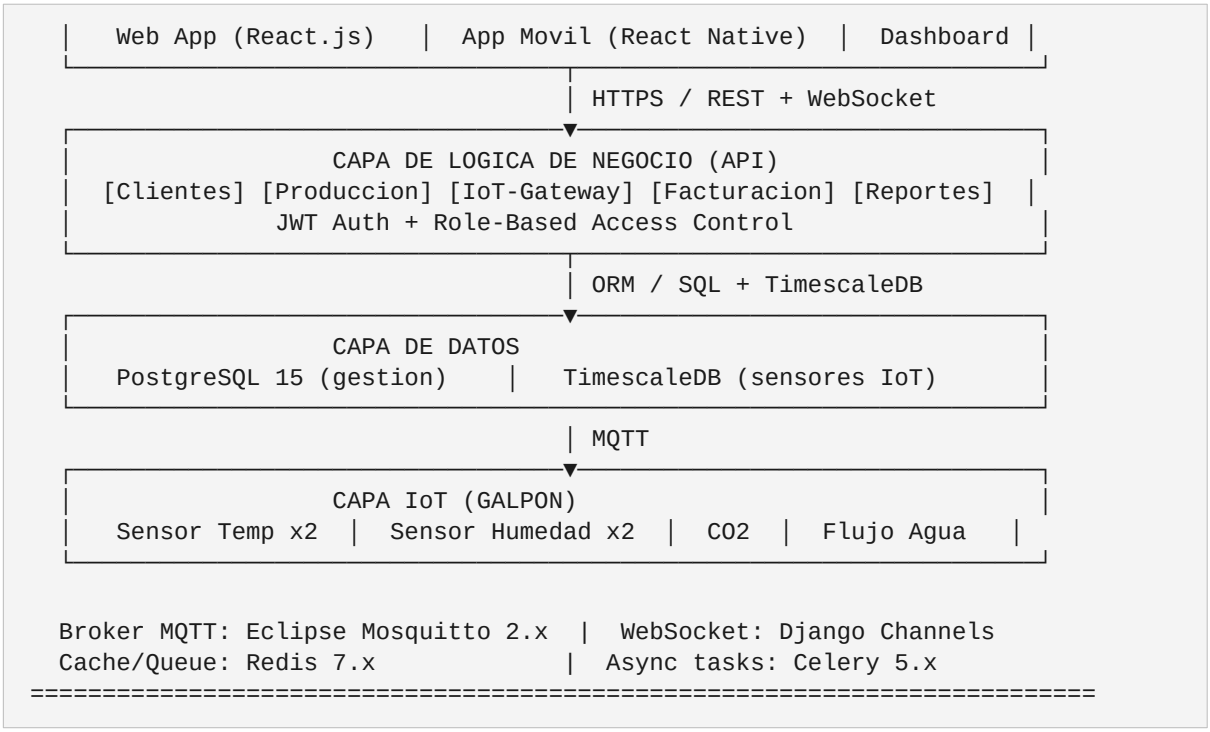


Diagrama 6.1 — Arquitectura N-Tier con capa IoT integrada

6.2.2 Stack Tecnológico

Capa	Tecnologia	Version	Justificacion
Backend API	Python / Django REST Framework	4.x	ORM, ecosistema maduro, MQTT ready
Frontend Web	React.js + Tailwind CSS	18 / 3.x	SPA responsiva, componentes reutilizables
App Movil	React Native	0.73	Codigo compartido iOS/Android
Base de Datos	PostgreSQL	15	ACID, tipos enumerados, columnas generadas
Series Temp.	TimescaleDB	2.x	Extension Postgres para IoT, compresion 90%
Broker MQTT	Eclipse Mosquitto	2.x	Protocolo ligero para sensores
WebSocket	Django Channels + Redis	4.x	Push en tiempo real al dashboard
Tareas Async	Celery + Redis	5.x	Reportes, alertas y jobs nocturnos
Contenedores	Docker + Compose	24.x	Despliegue reproducible

6.2.3 Patrones de Diseño Aplicados

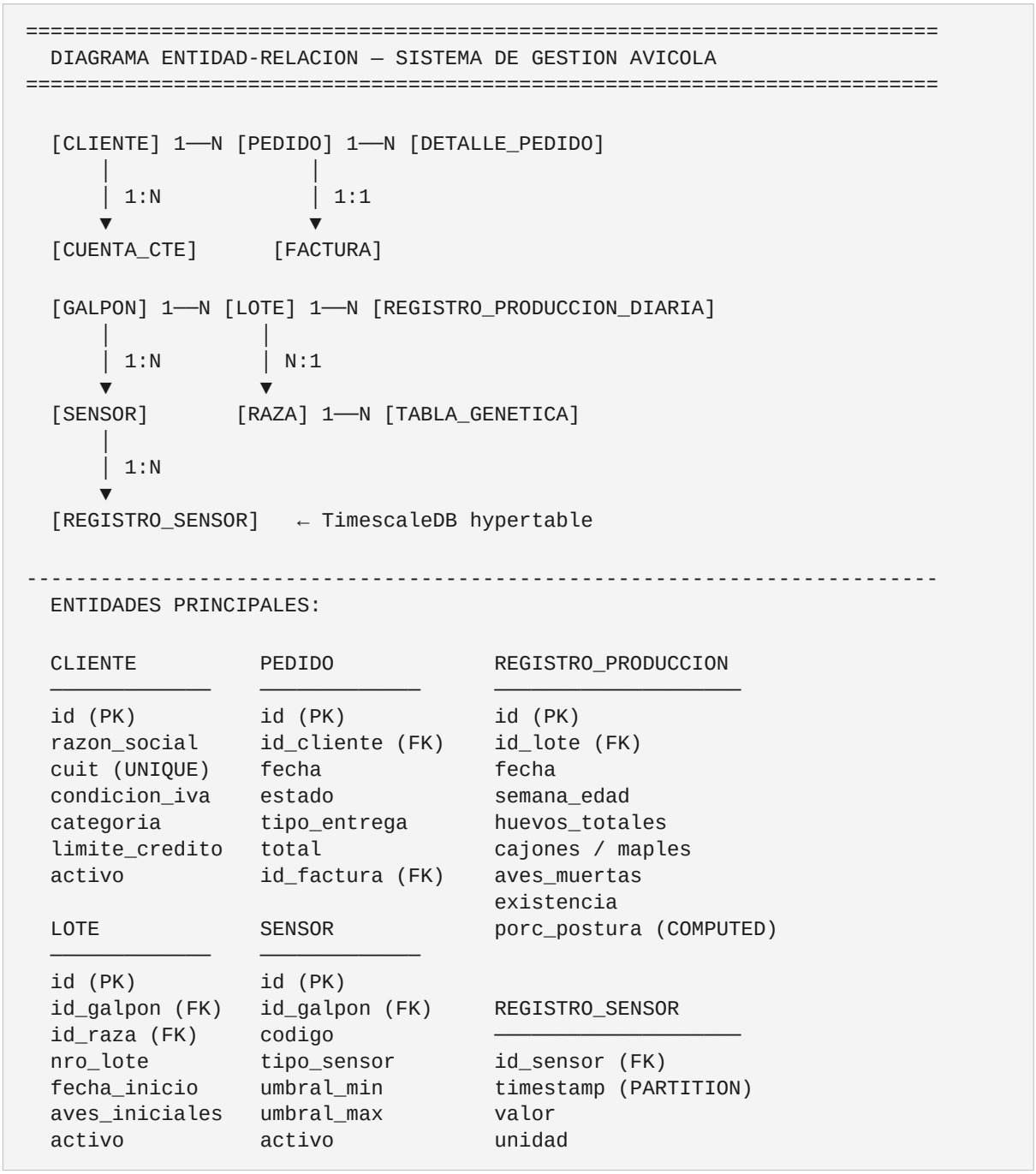
- Repository Pattern: abstraccion del acceso a datos, los servicios no conocen el ORM directamente.
- Service Layer: la logica compleja (produccion, alertas, facturacion) reside en clases de servicio desacopladas de las vistas.

- Observer / Event-driven: el motor de alertas IoT reacciona a eventos de sensores sin acoplar el gateway al sistema.
- Strategy Pattern: el tipo de factura (A/B/C) y alicuota de IVA se resuelven mediante estrategias intercambiables segun condicion impositiva del cliente.

6.2.4 Modelo de datos

6.2.4.1 Diagrama Entidad-Relación

El modelo de datos refleja las entidades del dominio avicola y sus relaciones. Se eligio PostgreSQL por su soporte de tipos enumerados nativos, columnas generadas (GENERATED ALWAYS AS STORED) y la extension TimescaleDB para series temporales de sensores.



```
========================================================================
```

Diagrama 7.1 — DER del Sistema de Gestion Avicola

6.2.4.2 Decisiones de Diseño

- Columna calculada porc_postura: definida como GENERATED ALWAYS AS STORED en registros_produccion, garantizando consistencia sin calculos en la aplicacion.
- TimescaleDB hypertable: registros_sensor se convierte en hypertable particionada por timestamp con politica de compresion automatica a los 7 dias, reduciendo hasta 90% el espacio en disco.
- Tipos ENUM nativos: condicion_iva, categoria_cliente y estado_pedido usan ENUM de PostgreSQL, mejorando integridad y legibilidad.
- Columna subtotal en detalle_pedido: tambien GENERATED AS STORED para evitar inconsistencias.
- Vistas de resumen: v_produccion_semanal y v_saldos_clientes centralizan la logica de agregacion.

6.2.4.3 Script DDL Completo

El script siguiente crea todas las tablas, indices, tipos y vistas del sistema. Requiere PostgreSQL 15+ con TimescaleDB instalado.

```
-- ================================================================
-- DDL COMPLETO: SISTEMA DE GESTION AVICOLA
-- Motor: PostgreSQL 15 + extension TimescaleDB
-- ================================================================

CREATE EXTENSION IF NOT EXISTS timescaledb;

-- Tipos enumerados
CREATE TYPE condicion_iva AS ENUM
  ('responsable_inscripto','consumidor_final','exento','monotributista');
CREATE TYPE categoria_cliente AS ENUM
  ('mayorista','minorista','distribuidor');
CREATE TYPE estado_pedido AS ENUM
  ('borrador','confirmado','en_entrega','entregado','facturado','cancelado');
CREATE TYPE tipo_sensor AS ENUM
  ('temperatura','humedad','calidad_aire','flujo_agua','temp_agua');

-- ─── clientes ──────────────────────────────────────────────────
CREATE TABLE clientes (
    id              SERIAL PRIMARY KEY,
    razon_social    VARCHAR(200) NOT NULL,
    cuit            CHAR(11) NOT NULL UNIQUE
                    CHECK (cuit ~ '^[0-9]{11}$'),
    condicion_iva   condicion_iva NOT NULL DEFAULT 'consumidor_final',
    categoria       categoria_cliente NOT NULL DEFAULT 'minorista',
    domicilio       VARCHAR(300),
    telefono        VARCHAR(30),
    email           VARCHAR(150),
```

```
    limite_credito  DECIMAL(12,2) NOT NULL DEFAULT 0,
    activo          BOOLEAN NOT NULL DEFAULT TRUE,
    fecha_alta      DATE NOT NULL DEFAULT CURRENT_DATE
);
CREATE INDEX idx_clientes_cuit   ON clientes(cuit);
CREATE INDEX idx_clientes_activo ON clientes(activo);

-- ─── razas ──────────────────────────────────────────────────────
CREATE TABLE razas (
    id          SERIAL PRIMARY KEY,
    nombre      VARCHAR(100) NOT NULL UNIQUE,
    fabricante  VARCHAR(100),
    descripcion TEXT
);

-- ─── tabla_genetica ─────────────────────────────────────────────
CREATE TABLE tabla_genetica (
    id           SERIAL PRIMARY KEY,
    id_raza      INT NOT NULL REFERENCES razas(id),
    semana_edad  INT NOT NULL CHECK (semana_edad BETWEEN 15 AND 115),
    prod_tabla   DECIMAL(5,2) NOT NULL,   -- % postura esperado
    viab_tabla   DECIMAL(5,2) NOT NULL,   -- % viabilidad esperado
    peso_huevo_g DECIMAL(5,1),
    UNIQUE (id_raza, semana_edad)
);

-- ─── galpones ───────────────────────────────────────────────────
CREATE TABLE galpones (
    id            SERIAL PRIMARY KEY,
    numero        INT NOT NULL UNIQUE,
    descripcion   VARCHAR(200),
    capacidad_max INT NOT NULL,
    activo        BOOLEAN NOT NULL DEFAULT TRUE
);

-- ─── lotes ──────────────────────────────────────────────────────
CREATE TABLE lotes (
    id             SERIAL PRIMARY KEY,
    id_galpon      INT NOT NULL REFERENCES galpones(id),
    id_raza        INT NOT NULL REFERENCES razas(id),
    nro_lote       VARCHAR(20) NOT NULL,
    fecha_inicio   DATE NOT NULL,
    fecha_fin      DATE,
    aves_iniciales INT NOT NULL,
    activo         BOOLEAN NOT NULL DEFAULT TRUE,
    UNIQUE (id_galpon, nro_lote)
);

-- ─── registros_produccion ───────────────────────────────────────
CREATE TABLE registros_produccion (
    id              SERIAL PRIMARY KEY,
    id_lote         INT NOT NULL REFERENCES lotes(id),
    fecha           DATE NOT NULL,
    semana_edad     INT NOT NULL,
    huevos_totales  INT NOT NULL DEFAULT 0,
    cajones         INT NOT NULL DEFAULT 0,
    maples          INT NOT NULL DEFAULT 0,
```

```
    aves_muertas    INT NOT NULL DEFAULT 0,
    existencia      INT NOT NULL,
    porc_postura    DECIMAL(5,2) GENERATED ALWAYS AS (
        CASE WHEN existencia > 0
             THEN ROUND((huevos_totales::DECIMAL/existencia)*100, 2)
             ELSE 0 END
    ) STORED,
    observaciones   TEXT,
    usuario_carga   VARCHAR(100),
    UNIQUE (id_lote, fecha)
);
CREATE INDEX idx_prod_lote_fecha ON registros_produccion(id_lote, fecha);

-- ─── sensores ──────────────────────────────────────────────────
CREATE TABLE sensores (
    id          SERIAL PRIMARY KEY,
    id_galpon   INT NOT NULL REFERENCES galpones(id),
    codigo      VARCHAR(50) NOT NULL UNIQUE,
    tipo        tipo_sensor NOT NULL,
    ubicacion   VARCHAR(100),
    umbral_min  DECIMAL(10,4),
    umbral_max  DECIMAL(10,4),
    activo      BOOLEAN NOT NULL DEFAULT TRUE
);

-- ─── registros_sensor (TimescaleDB hypertable) ─────────────────
CREATE TABLE registros_sensor (
    id_sensor   INT NOT NULL REFERENCES sensores(id),
    timestamp   TIMESTAMPTZ NOT NULL,
    valor       DECIMAL(10,4) NOT NULL,
    unidad      VARCHAR(20) NOT NULL
);
SELECT create_hypertable('registros_sensor', 'timestamp');
SELECT add_compression_policy('registros_sensor', INTERVAL '7 days');
CREATE INDEX idx_sensor_ts ON registros_sensor(id_sensor, timestamp DESC);

-- ─── pedidos ───────────────────────────────────────────────────
CREATE TABLE pedidos (
    id                SERIAL PRIMARY KEY,
    id_cliente        INT NOT NULL REFERENCES clientes(id),
    fecha             TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    estado            estado_pedido NOT NULL DEFAULT 'borrador',
    tipo_entrega      VARCHAR(50),
    direccion_entrega VARCHAR(300),
    observaciones     TEXT,
    total             DECIMAL(14,2),
    id_factura        INT,
    usuario_creador   VARCHAR(100)
);

-- ─── detalle_pedido ────────────────────────────────────────────
CREATE TABLE detalle_pedido (
    id              SERIAL PRIMARY KEY,
    id_pedido       INT NOT NULL REFERENCES pedidos(id) ON DELETE CASCADE,
    tipo_producto   VARCHAR(50) NOT NULL,
    descripcion     VARCHAR(200),
    cantidad        INT NOT NULL CHECK (cantidad > 0),
    precio_unitario DECIMAL(12,2) NOT NULL,
```

```
    subtotal        DECIMAL(14,2) GENERATED ALWAYS AS
                    (cantidad * precio_unitario) STORED
);

-- ─── facturas ──────────────────────────────────────────────────────
CREATE TABLE facturas (
    id               SERIAL PRIMARY KEY,
    id_pedido        INT NOT NULL REFERENCES pedidos(id),
    id_cliente       INT NOT NULL REFERENCES clientes(id),
    nro_factura      VARCHAR(20) NOT NULL UNIQUE,
    tipo_comprobante CHAR(1) NOT NULL,  -- A, B o C
    punto_venta      INT NOT NULL,
    fecha_emision    DATE NOT NULL DEFAULT CURRENT_DATE,
    cae              VARCHAR(20),
    cae_vto          DATE,
    total_neto       DECIMAL(14,2) NOT NULL,
    alicuota_iva     DECIMAL(5,2) NOT NULL DEFAULT 21.00,
    total_iva        DECIMAL(14,2) NOT NULL,
    total_factura    DECIMAL(14,2) NOT NULL,
    anulada          BOOLEAN NOT NULL DEFAULT FALSE
);

-- ─── cuenta_corriente ──────────────────────────────────────────────
CREATE TABLE cuenta_corriente (
    id              SERIAL PRIMARY KEY,
    id_cliente      INT NOT NULL REFERENCES clientes(id),
    fecha           TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    tipo_movimiento VARCHAR(30) NOT NULL,  -- factura, pago, nc, nd
    referencia      VARCHAR(50),
    importe         DECIMAL(14,2) NOT NULL,
    saldo_parcial   DECIMAL(14,2) NOT NULL
);

-- ─── usuarios ──────────────────────────────────────────────────────
CREATE TABLE usuarios (
    id              SERIAL PRIMARY KEY,
    username        VARCHAR(80) NOT NULL UNIQUE,
    password_hash   VARCHAR(255) NOT NULL,
    nombre_completo VARCHAR(200),
    rol             VARCHAR(50) NOT NULL,
    activo          BOOLEAN NOT NULL DEFAULT TRUE,
    ultimo_acceso   TIMESTAMPTZ
);

-- ─── VISTAS ────────────────────────────────────────────────────────
CREATE VIEW v_produccion_semanal AS
SELECT l.id_galpon, g.numero AS galpon, l.id AS id_lote,
       l.nro_lote, r.nombre AS raza, rp.semana_edad,
       SUM(rp.huevos_totales)       AS huevos_semana,
       SUM(rp.cajones)              AS cajones_semana,
       SUM(rp.aves_muertas)         AS muertas_semana,
       MAX(rp.existencia)           AS existencia_fin,
       ROUND(AVG(rp.porc_postura),2) AS prom_postura,
       ROUND(SUM(rp.aves_muertas)::DECIMAL /
             NULLIF(MAX(rp.existencia),0)*100, 3) AS mort_semanal_pct
FROM registros_produccion rp
JOIN lotes l   ON rp.id_lote = l.id
JOIN galpones g ON l.id_galpon = g.id
```

```
JOIN razas r    ON l.id_raza = r.id
GROUP BY l.id_galpon, g.numero, l.id, l.nro_lote, r.nombre, rp.semana_edad;

CREATE VIEW v_saldos_clientes AS
SELECT c.id, c.razon_social, c.cuit, c.limite_credito,
       COALESCE(SUM(cc.importe),0) AS saldo_actual,
       c.limite_credito - COALESCE(SUM(cc.importe),0) AS credito_disponible
FROM clientes c
LEFT JOIN cuenta_corriente cc ON c.id = cc.id_cliente
WHERE c.activo = TRUE
GROUP BY c.id, c.razon_social, c.cuit, c.limite_credito;
```

Codigo 7.1 — DDL completo: PostgreSQL 15 + TimescaleDB

6.2.4.4 Índices y Estrategia de Performance

Tabla	Indice	Tipo	Proposito
clientes	idx_clientes_cuit	B-tree	Busqueda rapida por CUIT
clientes	idx_clientes_activo	B-tree	Filtrado por clientes activos
registros_produccion	idx_prod_lote_fecha	B-tree compuesto	Consultas por lote en rango de fechas
registros_sensor	idx_sensor_ts	TimescaleDB chunk	Optimizado para rangos temporales
lotes	galpon + nro_lote	UNIQUE constraint	Unicidad de lote por galpon

6.3 Modelado de los casos de uso

Los 24 casos de uso del sistema se agrupan en seis módulos. Para cada caso de uso se presentan sus requerimientos específicos, los prototipos de interfaz, la ficha de especificación y los diagramas de vista estática (clases) y dinámica (secuencia). Los casos CU19 a CU22 forman un único flujo automático y se modelan en conjunto.

Módulo	Casos de uso	Sección
1. Producción avícola	CU01, CU02, CU04, CU09	6.3.1 / 7.1
2. Monitoreo IoT y alertas	CU03, CU05, CU06, CU07, CU08, CU19-22	6.3.2 / 7.2
3. Sanidad y mortalidad	CU14, CU16, CU17, CU18	6.3.3 / 7.3
4. Dashboard, reportes y plan de implementación	CU10, CU12, CU13	6.3.4 / 7.4
5. Usuarios y roles	CU15	6.3.5 / 7.5
6. Gestión comercial (Don Gestión)	CU11, CU23, CU24	6.3.6 / 7.6

Resumen de relaciones entre casos de uso

Relación	Tipo	Descripción
CU20 incluye CU19	*<< include >>*	Siempre que se publica un dato MQTT (CU19) se persiste en TimescaleDB (CU20). Obligatorio.
CU21 incluye CU20	*<< include >>*	Siempre que se persiste un dato (CU20) se evalúa contra umbrales (CU21). Obligatorio.
CU22 incluye CU21	*<< include >>*	Si CU21 detecta desvío, siempre se envía notificación (CU22). Obligatorio.
CU06 incluye CU22	*<< include >>*	La alerta Warning siempre incluye envío de notificación push al supervisor.
CU07 incluye CU22	*<< include >>*	La alerta Crítico incluye notificación push + WhatsApp al supervisor.
CU24 extiende CU23	*<< extend >>*	La facturación ARCA/AFIP se ejecuta opcionalmente al confirmar un pedido en Don Gestión.
CU07 extiende CU06	*<< extend >>*	Si la alerta Warning alcanza nivel Crítico, CU07 extiende CU06 con acciones adicionales.
CU11 extiende CU01	*<< extend >>*	La aprobación del supervisor se activa condicionalmente si un pedido supera el límite de crédito.
CU02 incluye CU01	*<< include >>*	El registro de mortalidad (CU02) siempre se realiza junto al registro de producción diaria (CU01).

6.3.1 Módulo 1 — Producción avícola

6.3.1.1 CU01 — Registrar producción diaria de huevos

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-PROD-01	Registro diario móvil	Formulario de 5 campos con teclado numérico optimizado para celular.	Crítica
RF-PROD-02	Cálculo % postura	Calcular automáticamente (huevos / existencia × 100) al guardar cada registro.	Crítica
RF-PROD-03	Comparación tabla	Comparar % de postura real vs tabla genética Lohmann para la semana de edad.	Alta
RF-PROD-04	Alerta desvío >5pp	Generar alerta al supervisor si el % de postura real cae más de 5pp bajo la tabla.	Alta
RF-PROD-08	Historial 7 días	Mostrar últimos 7 registros con fecha, indicadores y desvío en pp.	Alta
RF06	(Relevamiento)	Registro diario de huevos por galpon y lote desde app movil — formulario de 5 campos.	—
RF07	(Relevamiento)	Calculo automatico del % de postura al guardar cada registro.	—

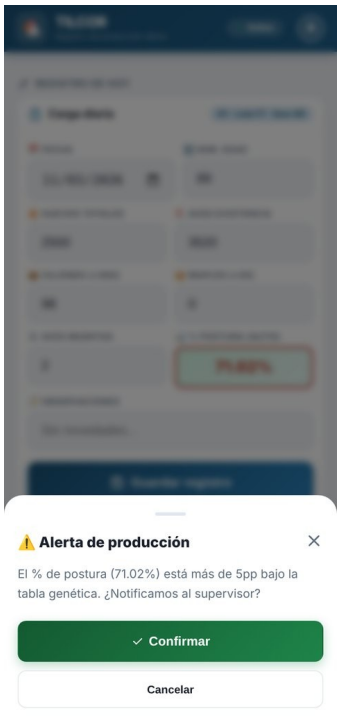
Prototipos de interfaz



Formulario de registro



Registro guardado



Alerta por postura baja

Ficha de caso de uso

FICHA DE CASO DE USO — CU01	
Nombre del CU	Registrar producción diaria de huevos
Actores	Operario de Galpón (actor primario)

Tipo	Funcional
Precondición	El operario finalizó la recolección de huevos del día. La app TILCOR está instalada y con sesión activa en su celular.
Flujo principal	1. El operario abre la app TILCOR y navega a la sección 'Producción'. 2. El sistema presenta el formulario con los campos: fecha (por defecto hoy), semana de edad, huevos totales, cajones, maples, aves muertas y existencia. 3. El operario completa los campos utilizando el teclado numérico del celular. 4. A medida que ingresa los datos de huevos y existencia, el sistema calcula el % de postura en tiempo real. 5. El operario toca el botón 'Guardar registro'. 6. El sistema compara el % de postura calculado vs la tabla genética Lohmann para la semana ingresada. 7. Si el desvío es mayor a -5pp: muestra modal de alerta y ofrece notificar al supervisor. 8. El sistema guarda el registro y actualiza el historial de los últimos 7 días en pantalla.
Postcondición	Registro guardado en la base de datos. Historial actualizado. Si hubo desvío crítico, el supervisor fue notificado.
Flujo alternativo / Excepción	Si el operario ingresa existencia = 0: el sistema muestra error de validación y no permite guardar.
Prioridad	Crítica

Vista estática (diagrama de clases)

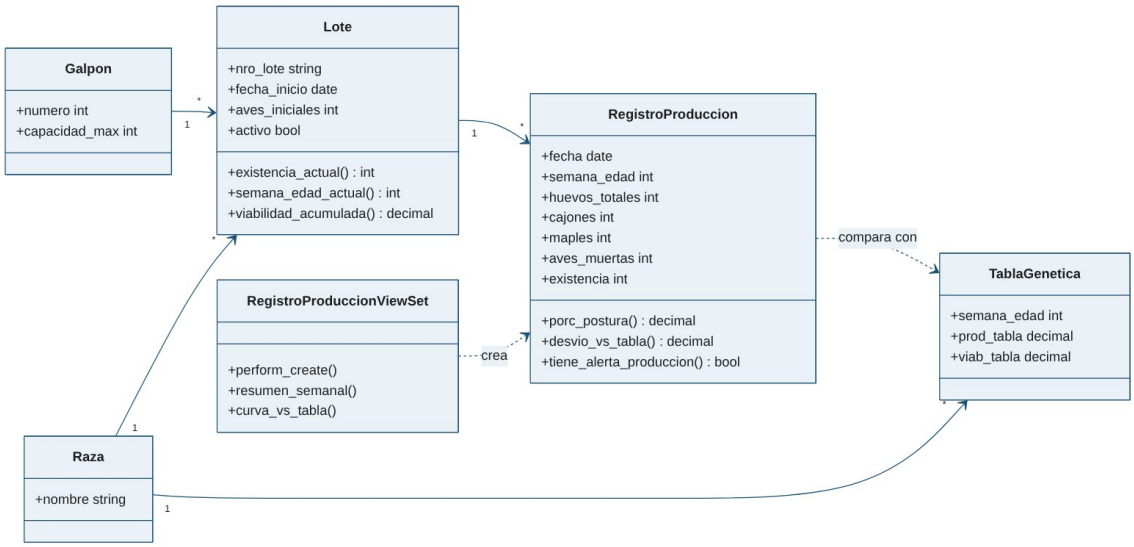


Diagrama de clases — CU01

Vista dinámica (diagrama de secuencia)

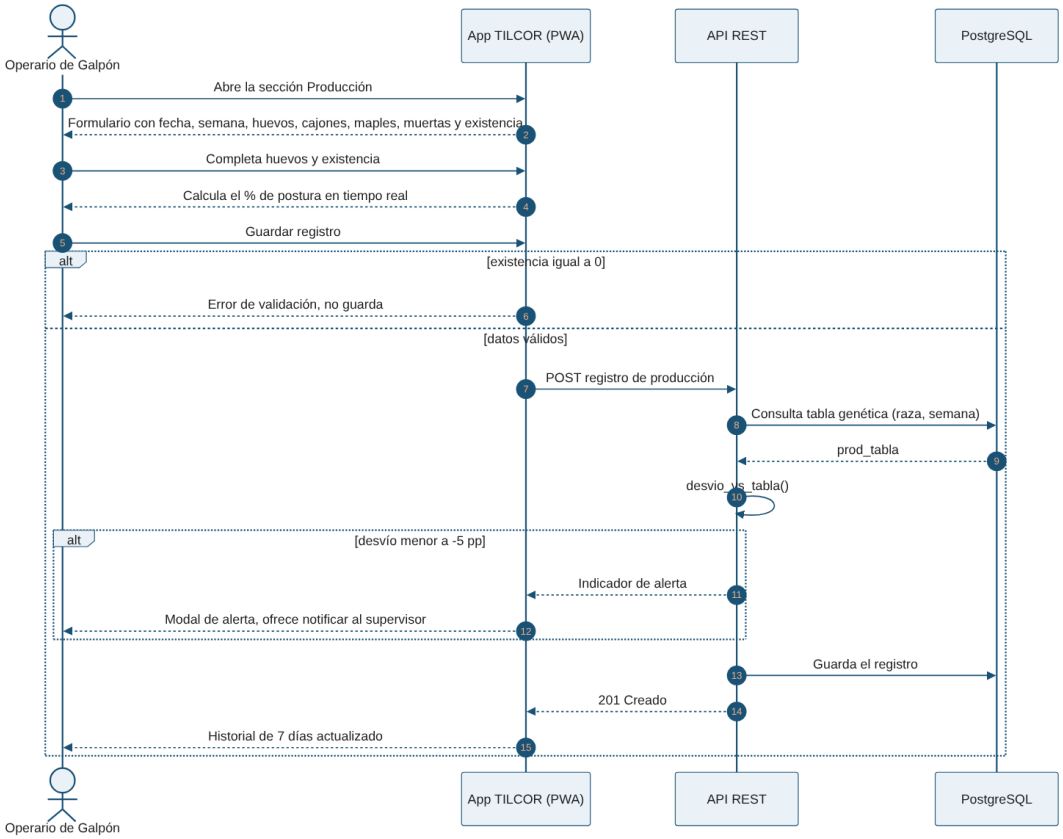


Diagrama de secuencia — CU01

6.3.1.2 CU02 — Registrar mortalidad diaria de aves

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-PROD-05	Control mortalidad	Calcular mortalidad semanal y alertar: Warning >0,5%, Crítico >1,0%.	Crítica
RF08	(Relevamiento)	Registro de mortalidad diaria y calculo de viabilidad acumulada del lote.	—
RF10	(Relevamiento)	Alerta Warning si mortalidad semanal > 0,5% y alerta Critico si > 1,0%.	—

Prototipos de interfaz



Campo de aves muertas

Ficha de caso de uso

FICHA DE CASO DE USO — CU02	
Nombre del CU	Registrar mortalidad diaria de aves
Actores	Operario de Galpón (actor primario)
Tipo	Funcional
Precondición	El operario retiró las aves muertas del día del galpón. La app TILCOR está activa en su celular.
Flujo principal	1. El operario abre la app y navega a la sección 'Producción'. 2. El sistema presenta el formulario de carga diaria que incluye el campo 'Aves muertas'. 3. El operario ingresa la cantidad de bajas del día. 4. El sistema actualiza automáticamente la existencia del lote: existencia = existencia anterior - muertas del día. 5. El sistema calcula la mortalidad semanal acumulada en tiempo real. 6. Si mortalidad semanal > 0,5%: el sistema genera alerta Warning al supervisor (CU06). 7. Si mortalidad semanal > 1,0%: el sistema genera alerta Crítico al supervisor (CU07). 8. El operario guarda el registro junto al resto de los datos de producción del día.

Postcondición	Bajas registradas. Existencia del lote actualizada. Mortalidad semanal recalculada. Supervisor alertado si superó umbral.
Flujo alternativo / Excepción	Si el operario ingresa un valor mayor a la existencia actual: el sistema muestra error de validación.
Prioridad	Crítica

Vista estática (diagrama de clases)



Diagrama de clases — CU02

Vista dinámica (diagrama de secuencia)

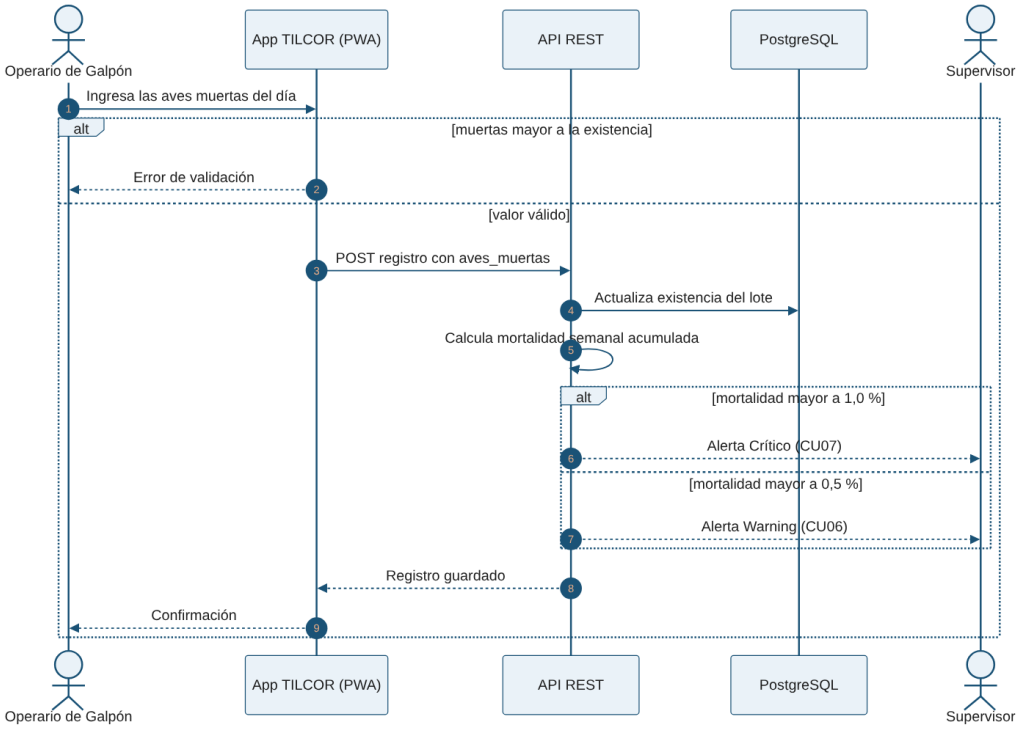


Diagrama de secuencia — CU02

6.3.1.3 CU04 — Ver historial de producción por fecha

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-PROD-08	Historial 7 días	Mostrar últimos 7 registros con fecha, indicadores y desvío en pp.	Alta

Prototipos de interfaz



Historial de 7 días

Ficha de caso de uso

FICHA DE CASO DE USO — CU04	
Nombre del CU	Ver historial de producción por fecha
Actores	Operario de Galpón · Supervisor Productivo (actores primarios)
Tipo	Funcional
Precondición	El usuario tiene sesión activa en la app. Existen registros de producción previos en el sistema.
Flujo principal	1. El usuario navega a la sección 'Producción'. 2. El sistema muestra automáticamente el historial de los últimos 7 registros cargados. 3. Para cada registro el sistema muestra: fecha, semana de edad, huevos totales, cajones, muertas, existencia, % postura y desvío en pp vs tabla. 4. El desvío se muestra en color: verde si es positivo, naranja si es negativo menor a -5pp, rojo si supera -5pp. 5. El usuario puede scrollear para ver registros más antiguos. 6. El supervisor puede ver la gráfica de postura real vs tabla genética actualizada con los datos históricos.
Postcondición	El usuario visualizó el historial de producción con indicadores de desvío respecto a la tabla genética.

Flujo alternativo / Excepción	Si no hay registros cargados: el sistema muestra el mensaje 'Sin registros disponibles. Cargá el primer registro del día'.
Prioridad	Alta

Vista estática (diagrama de clases)

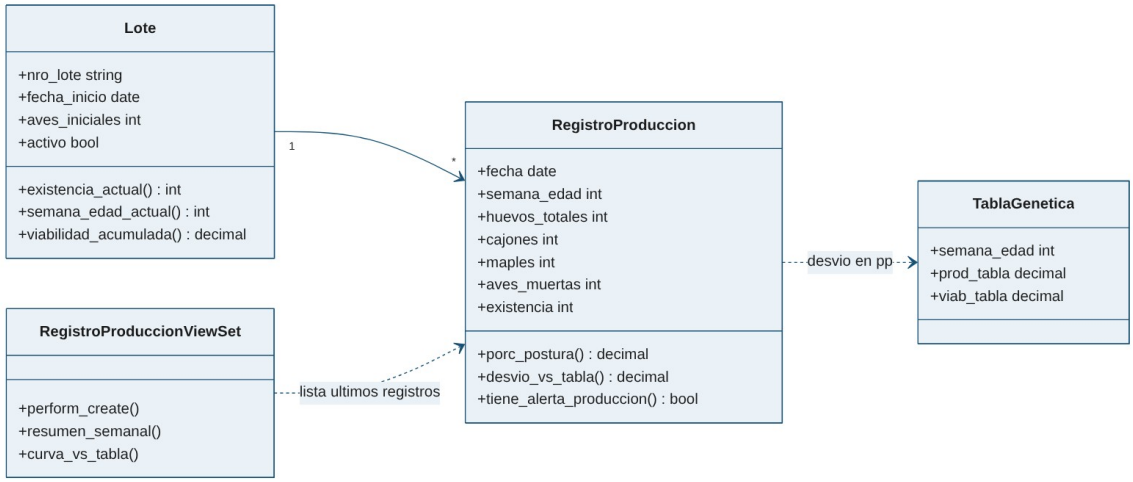


Diagrama de clases — CU04

Vista dinámica (diagrama de secuencia)

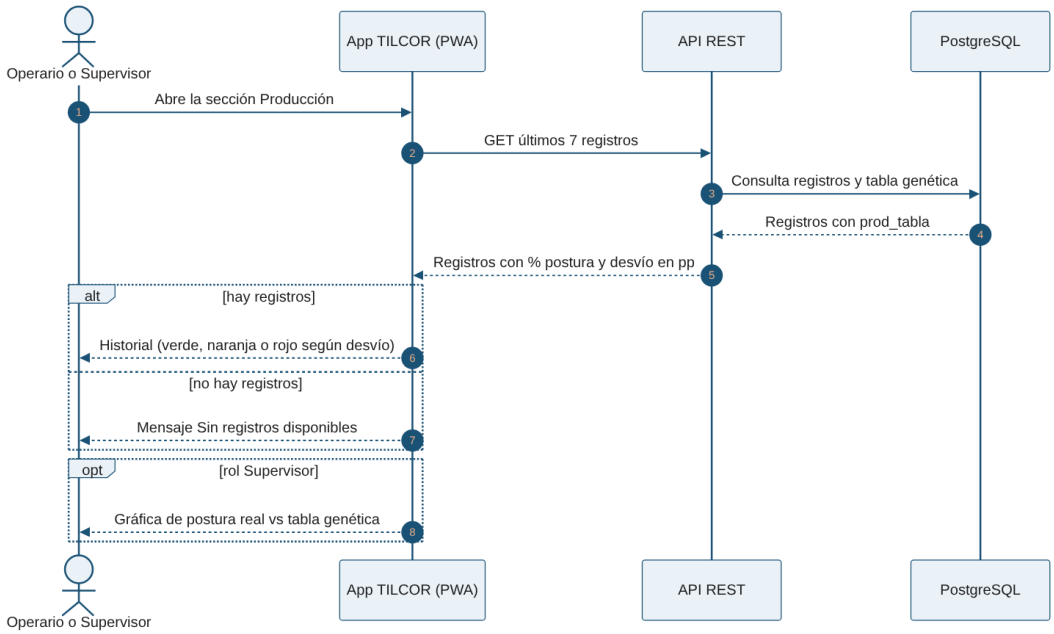


Diagrama de secuencia — CU04

6.3.1.4 CU09 — Ver curva de postura real vs tabla genética

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-PROD-03	Comparación tabla	Comparar % de postura real vs tabla genética Lohmann para la semana de edad.	Alta
RF09	(Relevamiento)	Comparacion automatica vs tabla genetica del fabricante con alerta si desvio > 5pp.	—

Prototipos de interfaz



Curva real vs tabla

Ficha de caso de uso

FICHA DE CASO DE USO — CU09	
Nombre del CU	Ver curva de postura real vs tabla genética Lohmann
Actores	Supervisor Productivo · Propietario (actores primarios)
Tipo	Funcional
Precondición	El usuario tiene sesión activa. Existen registros de producción cargados en el sistema para el lote activo.
Flujo principal	1. El usuario navega al dashboard principal o a la sección 'Producción'. 2. El sistema recupera los registros de producción históricos del lote activo. 3. El sistema recupera los valores de la tabla genética del fabricante Lohmann para la raza correspondiente. 4. El sistema renderiza la gráfica de líneas mostrando: % de postura real (línea verde sólida) vs % de postura tabla (línea roja punteada). 5. El eje X muestra las semanas de edad del lote (semanas 83 a 89 en el período relevado). 6. El eje Y muestra el % de postura de 0 a 100%. 7. El sistema muestra el badge de desvío acumulado expresado en puntos porcentuales (ej: +21.2 AA).

	8. El usuario puede ver el desvío semana a semana comparando ambas líneas.
Postcondición	El usuario visualizó la curva comparativa de producción real vs tabla genética con el desvío acumulado del lote.
Flujo alternativo / Excepción	Si no hay registros suficientes (menos de 3 días): el sistema muestra la gráfica con los datos disponibles y un mensaje indicando que se necesitan más registros para una comparación significativa.
Prioridad	Alta

Vista estática (diagrama de clases)

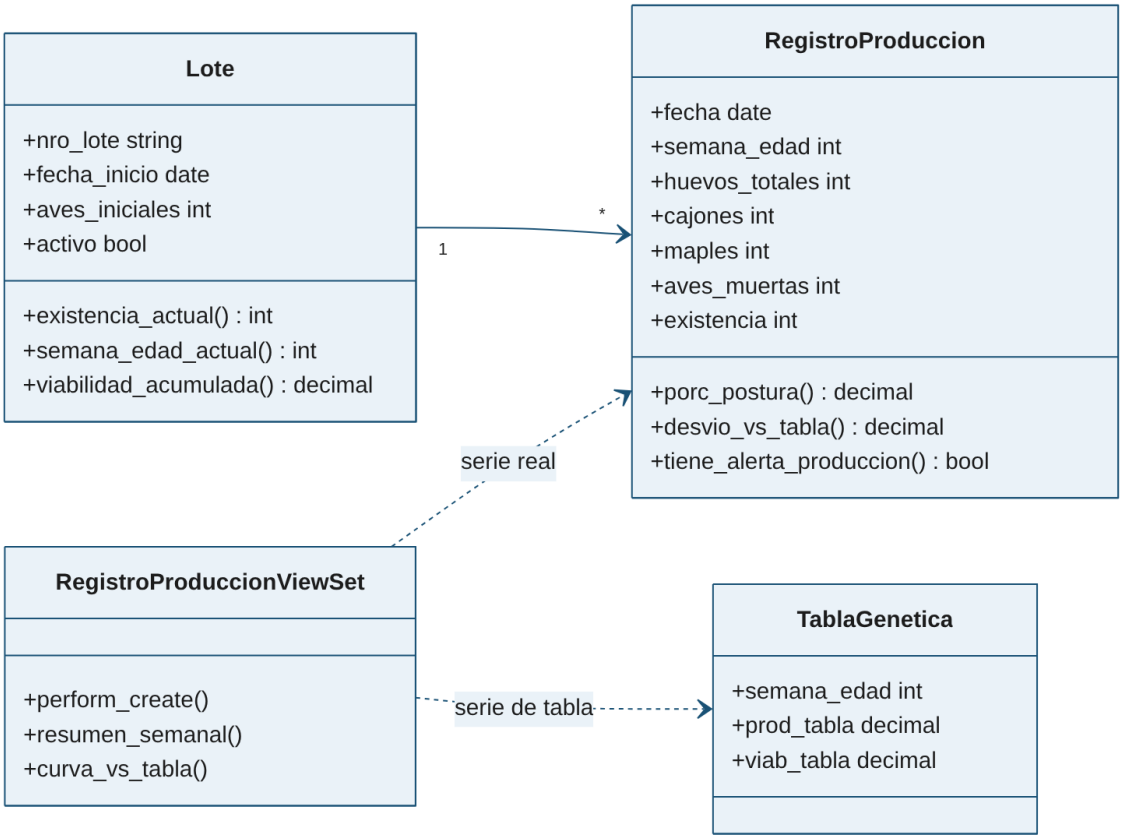


Diagrama de clases — CU09

Vista dinámica (diagrama de secuencia)

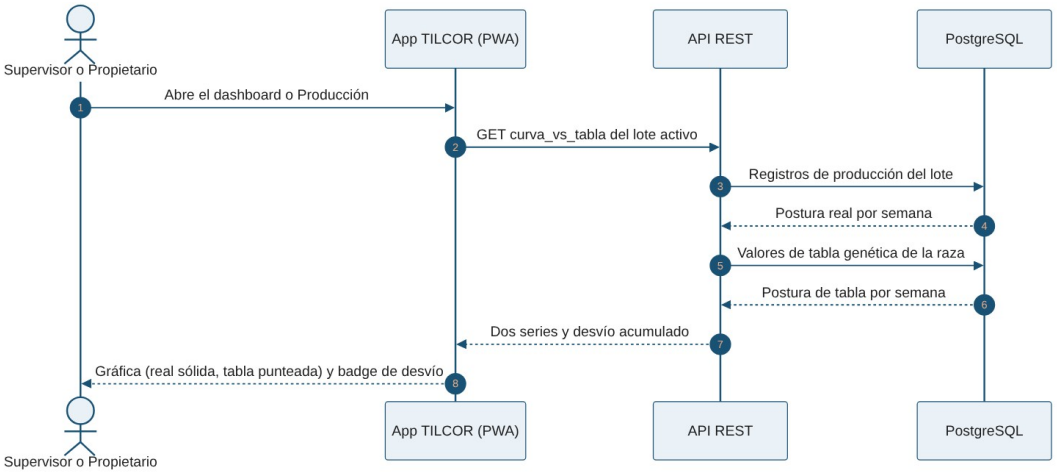


Diagrama de secuencia — CU09

6.3.2 Módulo 2 — Monitoreo IoT y alertas

6.3.2.1 CU03 — Consultar estado actual de sensores IoT

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-IOT-03	Dashboard tiempo real	Actualizar dashboard en < 2 segundos via WebSocket (Django Channels).	Alta
RF-IOT-04	Semáforo visual	Mostrar verde/amarillo/rojo por sensor según su estado respecto a los umbrales.	Alta
RF14	(Relevamiento)	Dashboard con semaforo visual en tiempo real (verde/amarillo/rojo).	—
RNF-02	Performance	Dashboard IoT actualizado en < 2s. Criterio: Tiempo ≤ 2s en red 4G	—
RNF-04	Offline	Funcionar sin conexión con datos de última sesión. Criterio: Service Worker con caché activa	—

Prototipos de interfaz



Panel de sensores

Ficha de caso de uso

FICHA DE CASO DE USO — CU03	
Nombre del CU	Consultar estado actual de sensores IoT
Actores	Operario de Galpón · Supervisor Productivo (actores primarios)
Tipo	Funcional

Precondición	El usuario tiene la app TILCOR abierta y con sesión activa. Al menos un sensor está activo enviando datos al sistema.
Flujo principal	1. El usuario navega a la sección 'IoT' desde la barra de navegación inferior. 2. El sistema consulta los últimos valores disponibles de cada sensor en TimescaleDB. 3. El sistema renderiza el panel de 6 sensores con semáforo visual: verde (normal), amarillo (advertencia), rojo (crítico). 4. El sistema muestra para cada sensor: nombre, valor actual, unidad de medida y estado. 5. El sistema establece una conexión WebSocket y actualiza los valores automáticamente cada vez que llega un nuevo dato. 6. El usuario puede ver el timestamp de la última actualización recibida. 7. El usuario puede navegar al historial de un sensor tocando sobre su card.
Postcondición	El usuario visualiza el estado ambiental actual del galpón con datos actualizados en tiempo real.
Flujo alternativo / Excepción	Sin conexión a internet: el Service Worker sirve la última versión cacheada con los últimos datos disponibles.
Prioridad	Alta

Vista estática (diagrama de clases)

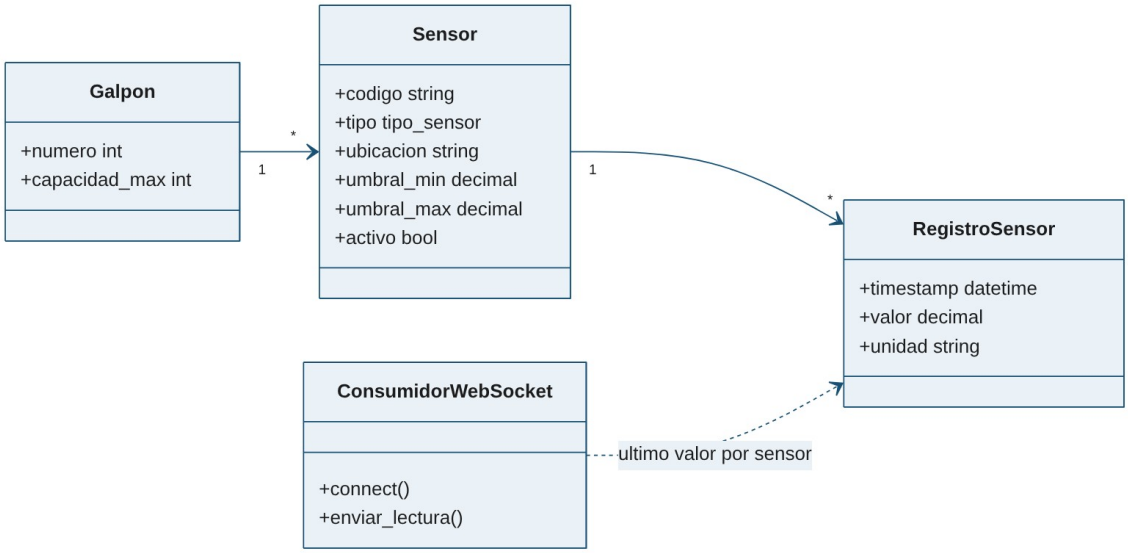


Diagrama de clases — CU03

Vista dinámica (diagrama de secuencia)

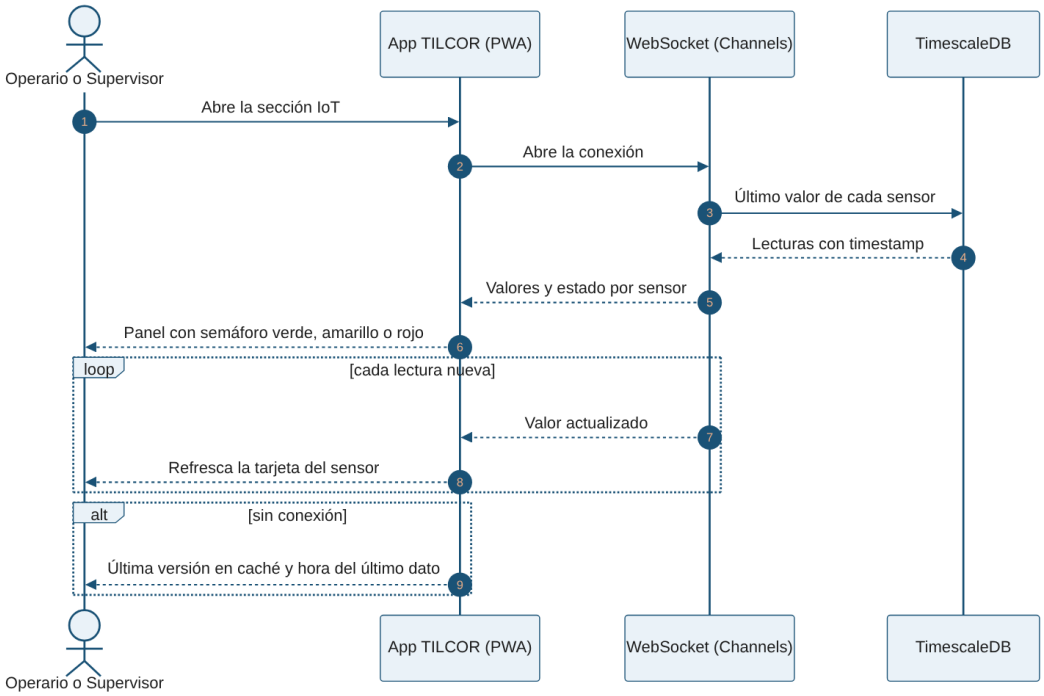


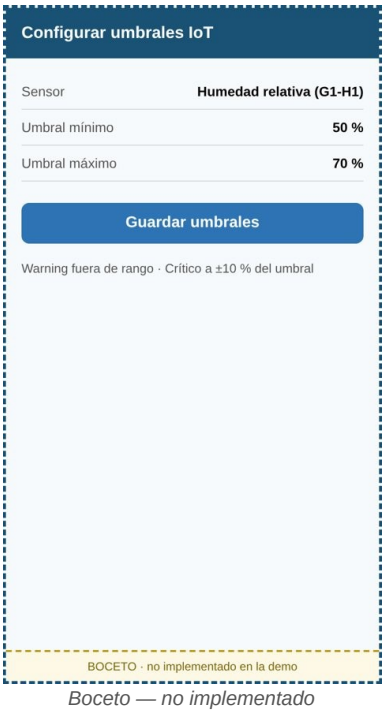
Diagrama de secuencia — CU03

6.3.2.2 CU05 — Configurar umbrales de alerta por sensor

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-IOT-07	Config. umbrales	Permitir configurar min/max de alerta por sensor desde el panel de administración.	Alta

Prototipos de interfaz



Boceto — no implementado

Ficha de caso de uso

FICHA DE CASO DE USO — CU05	
Nombre del CU	Configurar umbrales de alerta por sensor
Actores	Supervisor Productivo (actor primario)
Tipo	Funcional
Precondición	El supervisor tiene sesión activa con rol Admin. El módulo IoT está activo con sensores funcionando.
Flujo principal	1. El supervisor accede al panel de configuración de alertas IoT. 2. El sistema presenta la lista de sensores activos con sus umbrales actuales configurados. 3. El supervisor selecciona el sensor que desea configurar. 4. El sistema muestra los campos: valor mínimo de alerta, valor máximo de alerta y nivel de clasificación. 5. El supervisor modifica los valores según el estándar avícola recomendado. 6. El sistema valida que el mínimo sea menor al máximo. 7. El supervisor confirma los cambios. 8. El sistema guarda la configuración y la aplica inmediatamente al motor de alertas para ese sensor.
Postcondición	Los umbrales actualizados quedan guardados y el motor de alertas los aplica desde ese momento.
Flujo alternativo / Excepción	Si el supervisor ingresa mínimo mayor al máximo: el sistema muestra error de validación y no guarda los cambios.
Prioridad	Alta

Vista estática (diagrama de clases)

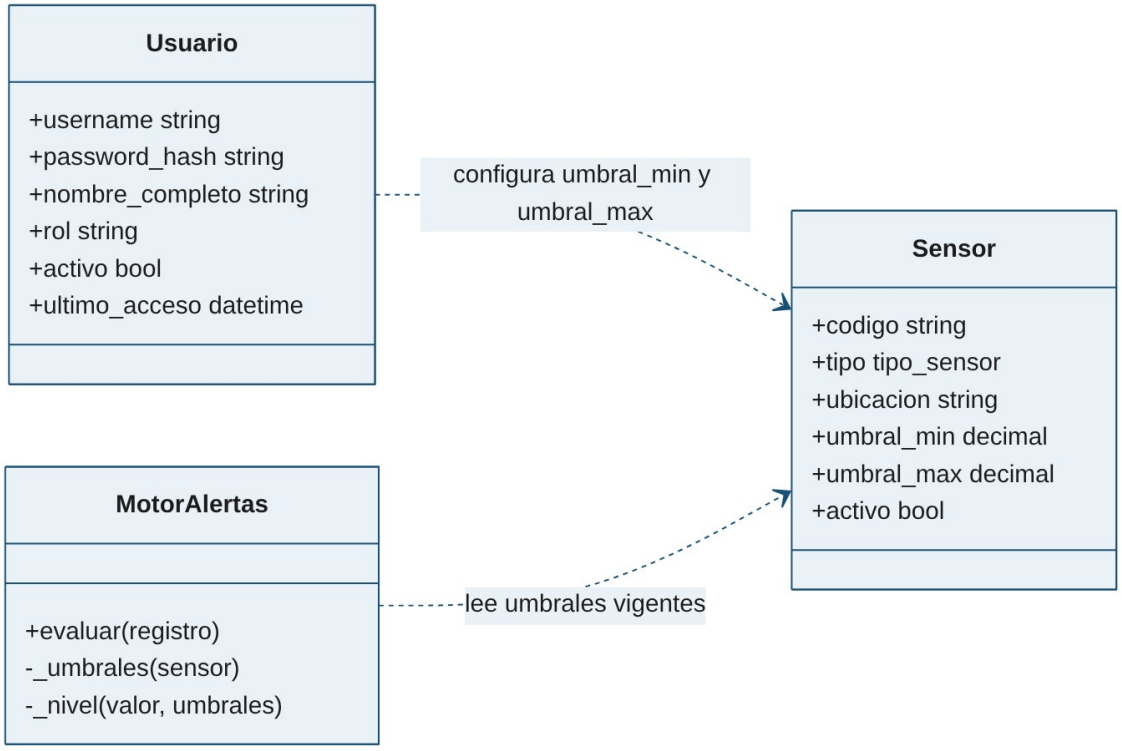


Diagrama de clases — CU05

Vista dinámica (diagrama de secuencia)

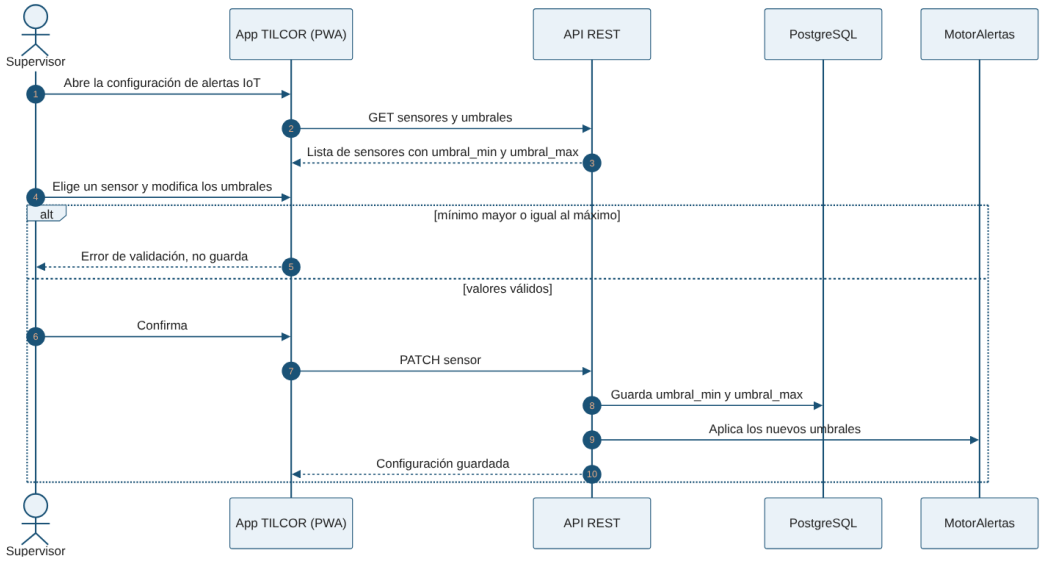


Diagrama de secuencia — CU05

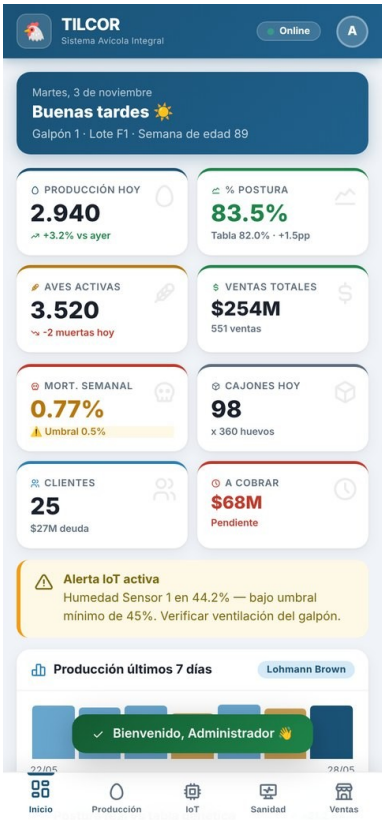
6.3.2.3 CU06 — Recibir alerta Warning

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-IOT-05	Motor de alertas	Evaluar cada dato contra umbrales y clasificar como Warning o Crítico automáticamente.	Crítica
RF-IOT-06	Notificación < 2 min	Enviar notificación push al responsable en máximo 2 minutos desde el evento.	Crítica

ID	Requerimiento	Descripción	Prioridad
RF-IOT-09	Log de alertas	Registrar todas las alertas con timestamp, sensor, valor, umbral, nivel y estado.	Alta
RF12	(Relevamiento)	Alertas automaticas push y WhatsApp en menos de 2 minutos ante desviaciones.	—
RNF-10	Alertas	Notificación en < 2 min desde el evento. Criterio: Tiempo ≤ 120 segundos en pruebas	—

Prototipos de interfaz



Banner de alerta



Estado de sensores

Ficha de caso de uso

FICHA DE CASO DE USO — CU06	
Nombre del CU	Recibir alerta Warning (mortalidad > 0,5% semanal o desvío ambiental)
Actores	Sistema TILCOR — Motor de Alertas (actor automático) · Supervisor Productivo (receptor)
Tipo	Funcional
Precondición	El motor de alertas detectó un valor que supera el umbral mínimo configurado, sin alcanzar el nivel crítico.
Flujo principal	1. El motor de alertas evalúa el dato recibido contra el umbral configurado para ese sensor o indicador. 2. Clasifica el nivel como Warning: valor fuera del rango normal pero sin superar el 110% del umbral.

	3. Registra el evento en el log de alertas con: timestamp, origen, valor registrado, umbral, nivel Warning. 4. Genera el mensaje de notificación con descripción del evento y acción recomendada. 5. Ejecuta CU22: envía notificación push al dispositivo del supervisor en < 2 minutos. 6. El dashboard actualiza el semáforo del sensor/indicador afectado a color amarillo. 7. El supervisor recibe la alerta, evalúa la situación y puede marcarla como resuelta.
Postcondición	Alerta Warning registrada en el log. Supervisor notificado. Dashboard en estado amarillo para el sensor afectado.
Flujo alternativo / Excepción	Si el dispositivo del supervisor no tiene conexión: la notificación se encola y se entrega al recuperar conectividad.
Prioridad	Crítica · << include >> CU22

Vista estática (diagrama de clases)

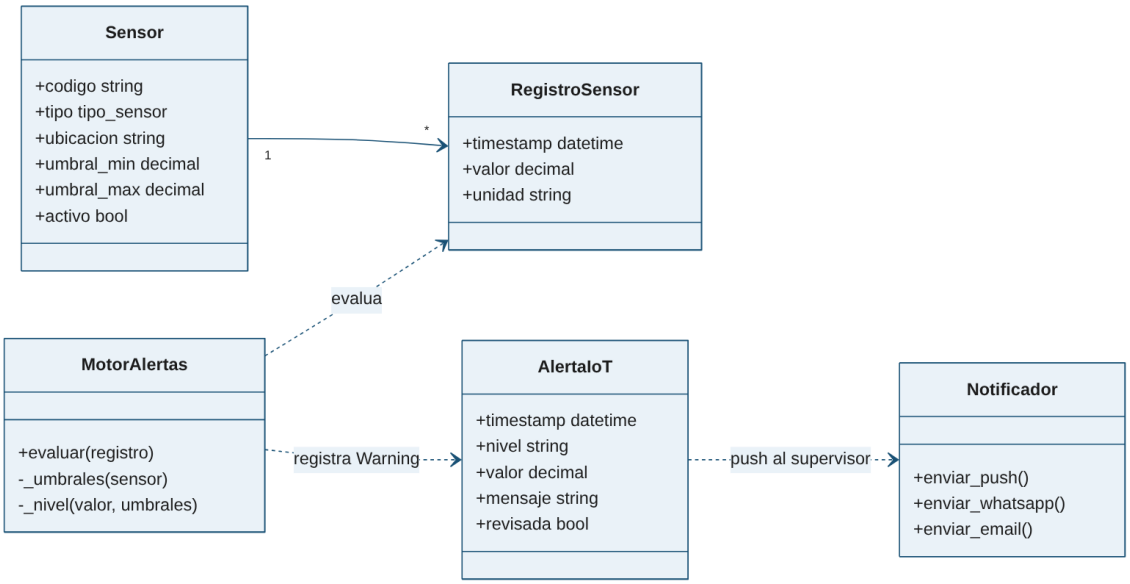


Diagrama de clases — CU06

Vista dinámica (diagrama de secuencia)

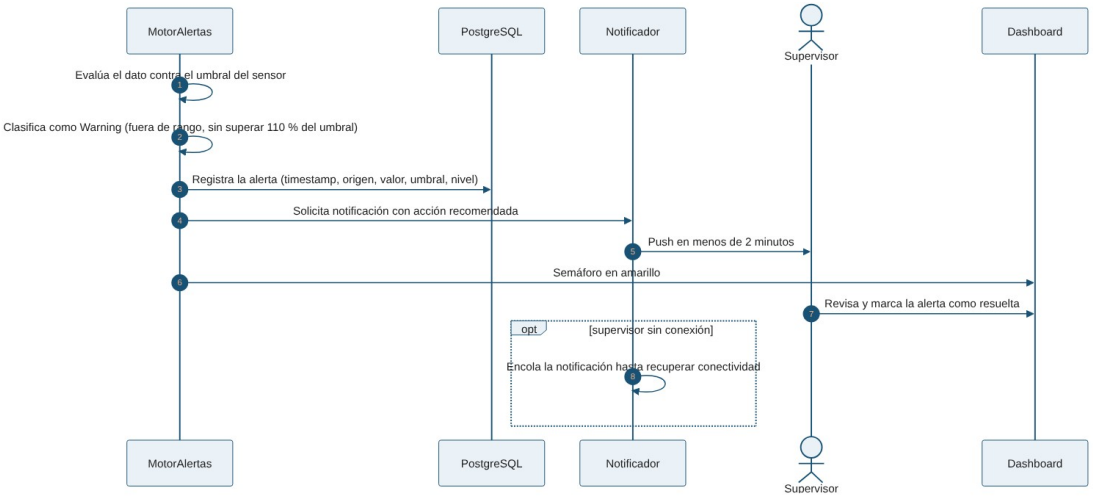


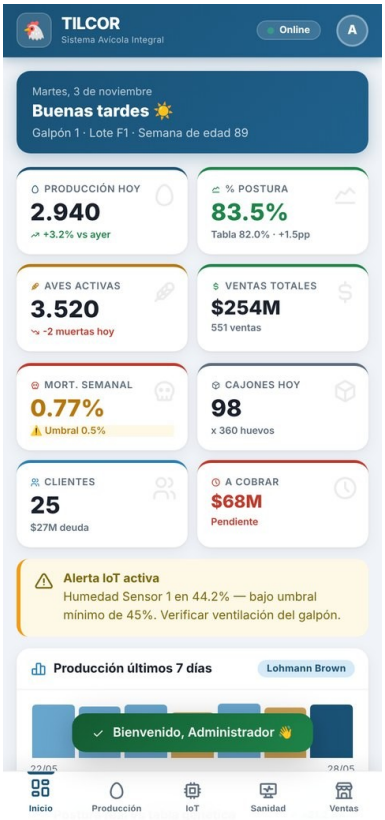
Diagrama de secuencia — CU06

6.3.2.4 CU07 — Recibir alerta Crítico

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-IOT-05	Motor de alertas	Evaluar cada dato contra umbrales y clasificar como Warning o Crítico automáticamente.	Crítica
RF-IOT-06	Notificación < 2 min	Enviar notificación push al responsable en máximo 2 minutos desde el evento.	Crítica
RF-IOT-09	Log de alertas	Registrar todas las alertas con timestamp, sensor, valor, umbral, nivel y estado.	Alta
RF12	(Relevamiento)	Alertas automaticas push y WhatsApp en menos de 2 minutos ante desviaciones.	—
RNF-10	Alertas	Notificación en < 2 min desde el evento. Criterio: Tiempo ≤ 120 segundos en pruebas	—

Prototipos de interfaz



Banner de alerta



Estado de sensores

Ficha de caso de uso

FICHA DE CASO DE USO — CU07	
Nombre del CU	Recibir alerta Crítico (mortalidad > 1,0% semanal o desvío crítico ambiental)
Actores	Sistema TILCOR — Motor de Alertas (actor automático) · Supervisor Productivo (receptor)
Tipo	Funcional
Precondición	El motor de alertas detectó un valor que supera el 110% del umbral configurado o la mortalidad semanal superó el 1,0%.
Flujo principal	1. El motor de alertas evalúa el dato y detecta nivel Crítico. 2. Registra el evento en el log con nivel Crítico: timestamp, origen, valor, umbral, diferencia porcentual. 3. Genera el mensaje de notificación con descripción del evento, nivel Crítico y acción urgente recomendada. 4. Ejecuta CU22: envía notificación push + WhatsApp al supervisor en < 2 minutos. 5. El dashboard actualiza el semáforo del sensor/indicador afectado a color rojo con animación pulsante. 6. Si es mortalidad crítica: el sistema envía automáticamente notificación al veterinario externo. 7. El supervisor recibe la alerta, toma acción correctiva urgente

	y marca como resuelta.
Postcondición	Alerta Crítico registrada. Supervisor y (si aplica) veterinario notificados. Dashboard en estado rojo pulsante.
Flujo alternativo / Excepción	Si no hay conectividad: notificaciones encoladas con máxima prioridad para entrega al recuperar conexión.
Prioridad	Crítica · << include >> CU22 · << extend >> CU06

Vista estática (diagrama de clases)

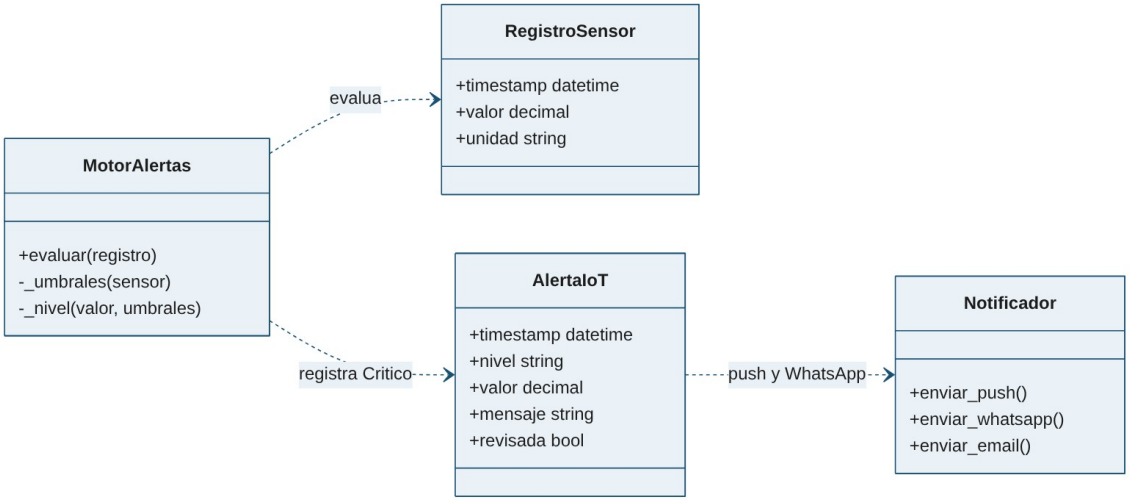


Diagrama de clases — CU07

Vista dinámica (diagrama de secuencia)

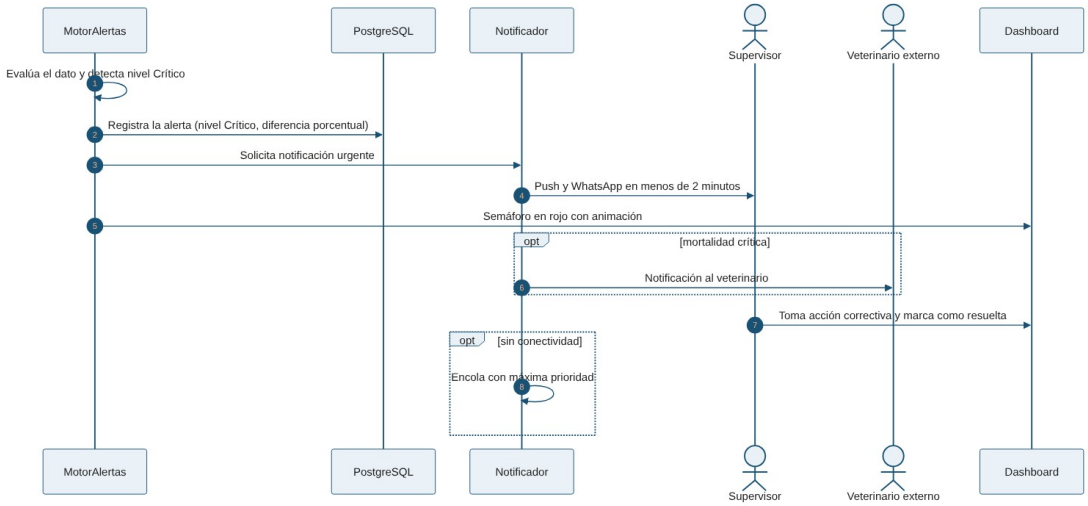


Diagrama de secuencia — CU07

6.3.2.5 CU08 — Consultar dashboard IoT en tiempo real

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-IOT-03	Dashboard tiempo real	Actualizar dashboard en < 2 segundos via WebSocket (Django Channels).	Alta

ID	Requerimiento	Descripción	Prioridad
RF-IOT-04	Semáforo visual	Mostrar verde/amarillo/rojo por sensor según su estado respecto a los umbrales.	Alta
RF-IOT-08	Histórico	Consultar historial de cualquier sensor en cualquier rango de fechas con gráfica.	Media
RF-IOT-09	Log de alertas	Registrar todas las alertas con timestamp, sensor, valor, umbral, nivel y estado.	Alta
RF13	(Relevamiento)	Historico de datos sensoriales con graficas de serie temporal.	—
RF14	(Relevamiento)	Dashboard con semaforo visual en tiempo real (verde/amarillo/rojo).	—

Prototipos de interfaz



Dashboard IoT

Ficha de caso de uso

FICHA DE CASO DE USO — CU08	
Nombre del CU	Consultar dashboard IoT en tiempo real
Actores	Supervisor Productivo · Operario de Galpón (actores primarios)
Tipo	Funcional
Precondición	El usuario tiene sesión activa. Los sensores están transmitiendo datos al sistema via MQTT.
Flujo principal	1. El usuario navega a la sección 'IoT' de la app. 2. El sistema establece conexión WebSocket con el servidor Django Channels.

	3. El sistema carga los últimos valores de los 6 sensores desde TimescaleDB. 4. El sistema muestra el panel de sensores con semáforo visual (verde/amarillo/rojo) por sensor. 5. El sistema muestra las gráficas de temperatura y humedad de las últimas 6 horas. 6. El sistema muestra el log de alertas del día con hora, sensor, nivel y descripción. 7. Los valores se actualizan automáticamente cada vez que llega un nuevo dato por WebSocket. 8. El usuario puede ver el historial de cualquier variable seleccionando el rango de fechas.
Postcondición	El usuario visualiza el estado ambiental completo del galpón en tiempo real con historial de alertas del día.
Flujo alternativo / Excepción	Sin conexión: el Service Worker sirve la última versión cacheada. Se indica el timestamp del último dato disponible.
Prioridad	Alta

Vista estática (diagrama de clases)

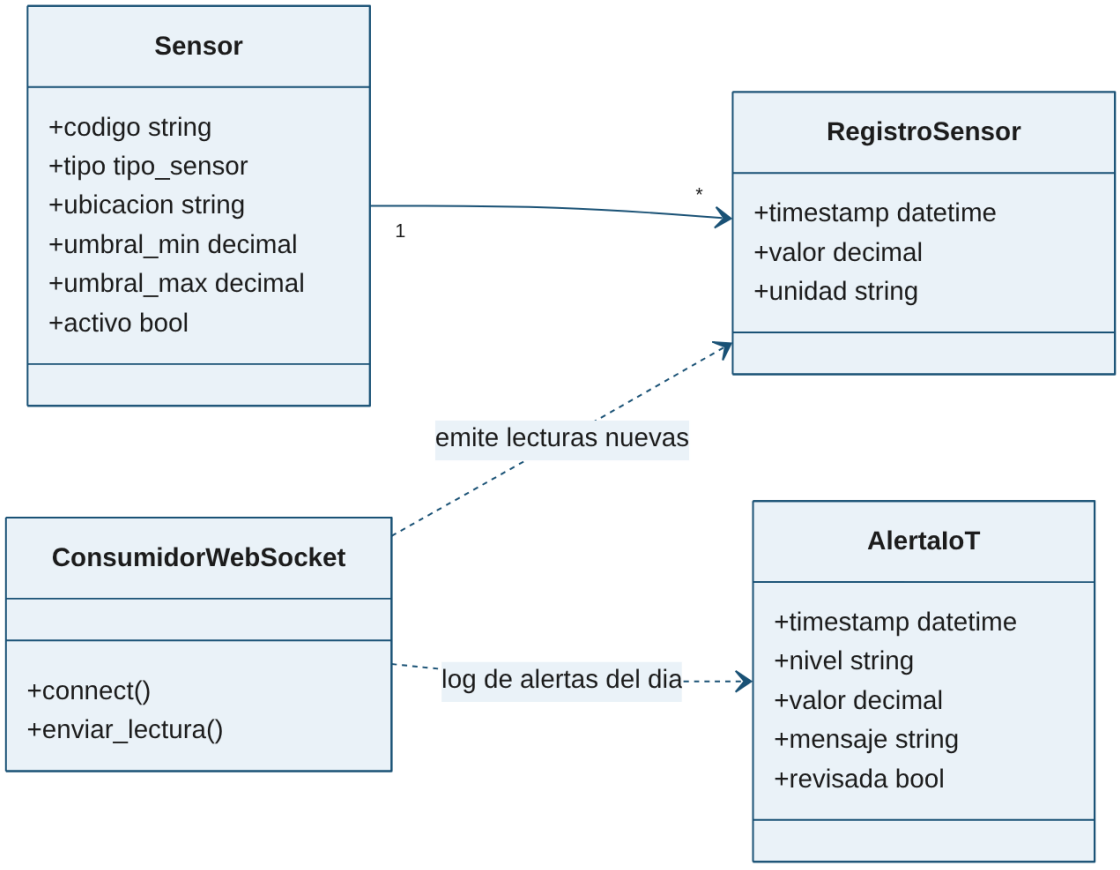


Diagrama de clases — CU08

Vista dinámica (diagrama de secuencia)

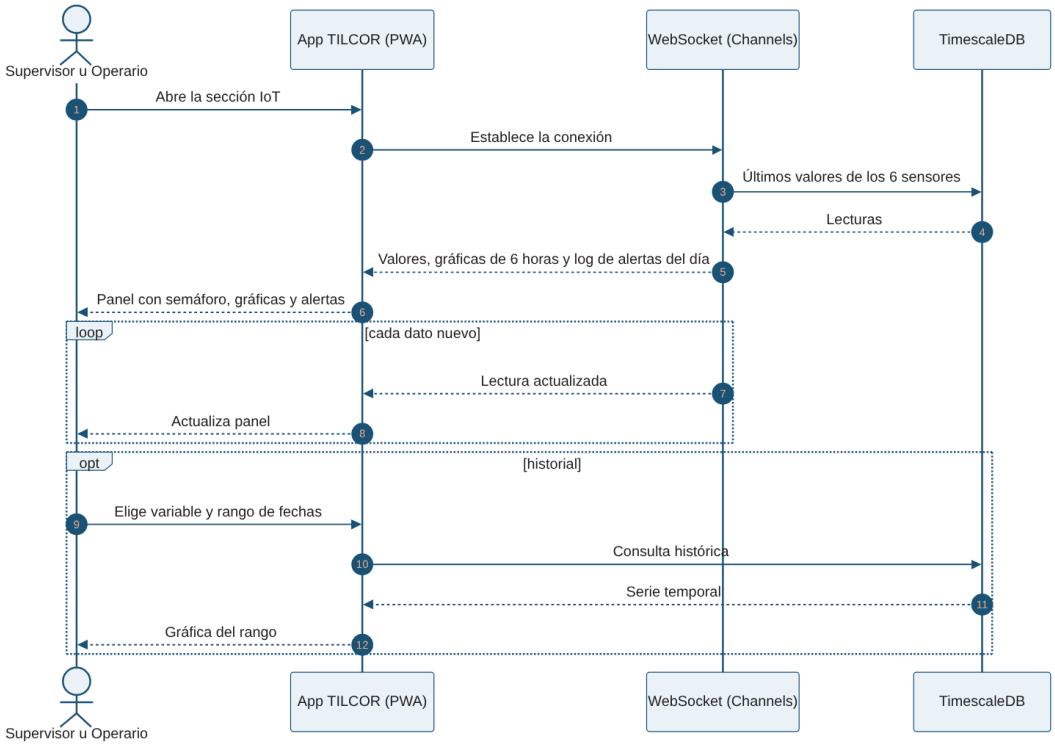


Diagrama de secuencia — CU08

6.3.2.6 CU19-22 — Flujo automático IoT (CU19 a CU22)

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-IOT-01	Recepción MQTT	Recibir datos de sensores via MQTT con QoS 1 a través de Eclipse Mosquitto.	Crítica
RF-IOT-02	Persistencia TimescaleDB	Almacenar cada dato en hypertable particionada por timestamp con compresión a 7 días.	Crítica
RF-IOT-05	Motor de alertas	Evaluar cada dato contra umbrales y clasificar como Warning o Crítico automáticamente.	Crítica
RF-IOT-06	Notificación < 2 min	Enviar notificación push al responsable en máximo 2 minutos desde el evento.	Crítica
RF-IOT-09	Log de alertas	Registrar todas las alertas con timestamp, sensor, valor, umbral, nivel y estado.	Alta
RF11	(Relevamiento)	Integracion con sensores existentes via protocolo MQTT.	—
RF12	(Relevamiento)	Alertas automaticas push y WhatsApp en menos de 2 minutos ante desviaciones.	—
RNF-01	Disponibilidad	Módulo IoT disponible 24/7. Criterio: Uptime ≥ 99,5% mensual	—
RNF-10	Alertas	Notificación en < 2 min desde el evento. Criterio: Tiempo ≤ 120 segundos en pruebas	—

Prototipos de interfaz

No aplica: es un flujo automático sin pantalla propia (sensores, servidor y servicios externos).

Ficha de caso de uso

FICHA DE CASO DE USO — CU19-22

Nombre del CU	Publicación, persistencia, evaluación y notificación automática IoT
Actores	Sensores IoT (actor sistema automático) · Broker MQTT · Gateway Django Channels · TimescaleDB
Tipo	Funcional
Precondición	Sensores activos conectados a red WiFi del galpón. Broker Eclipse Mosquitto corriendo. Gateway Django Channels suscripto al topic MQTT.
Flujo principal	1. CU19 — El sensor mide la variable ambiental y publica el dato en JSON via MQTT con QoS1: {sensor_id, valor, unidad, timestamp}. 2. El broker MQTT recibe el mensaje y lo distribuye al gateway suscripto. 3. CU20 — El gateway valida el formato del payload y persiste el dato en la hypertable de TimescaleDB. 4. CU21 — El gateway evalúa el valor recibido contra los umbrales configurados para ese sensor. 5. Si no hay desvío: el gateway transmite el dato al dashboard via WebSocket y finaliza el flujo. 6. Si hay desvío Warning: ejecuta CU06 que a su vez ejecuta CU22. 7. Si hay desvío Crítico: ejecuta CU07 que a su vez ejecuta CU22 con máxima prioridad. 8. CU22 — El sistema genera y envía la notificación push y/o WhatsApp al supervisor en < 2 minutos.
Postcondición	Dato persistido en TimescaleDB. Dashboard actualizado via WebSocket. Si hubo desvío, supervisor notificado en < 2 minutos.
Flujo alternativo / Excepción	Si el sensor no puede publicar al broker MQTT: el dato se pierde para ese ciclo. El gateway registra el timeout en el log de errores con el sensor_id y el timestamp del intento.
Prioridad	Crítica · CU20 << include >> CU19 · CU21 << include >> CU20 · CU22 << include >> CU21

Vista estática (diagrama de clases)

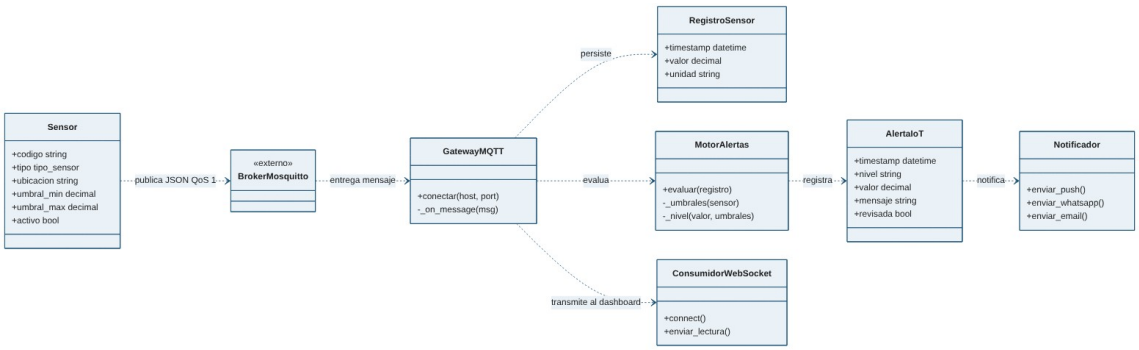


Diagrama de clases — CU19-22

Vista dinámica (diagrama de secuencia)

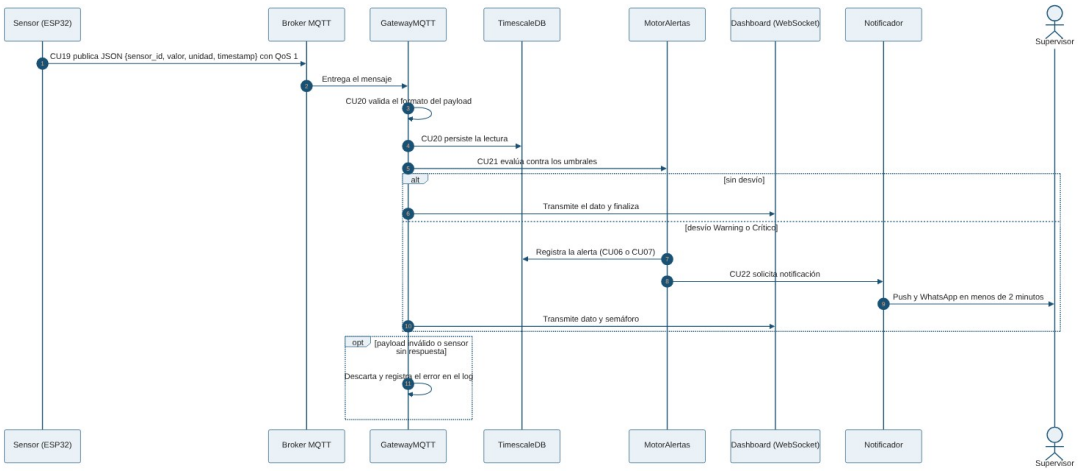


Diagrama de secuencia — CU19-22

6.3.3 Módulo 3 — Sanidad y mortalidad

6.3.3.1 CU14 — Ver viabilidad acumulada de los lotes

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-PROD-06	Viabilidad acumulada	Mantener actualizada la viabilidad del lote vs tabla del fabricante.	Alta
RF08	(Relevamiento)	Registro de mortalidad diaria y calculo de viabilidad acumulada del lote.	—

Prototipos de interfaz



Indicadores



Sanidad

Ficha de caso de uso

FICHA DE CASO DE USO — CU14	
Nombre del CU	Ver viabilidad acumulada de los lotes
Actores	Propietario · Supervisor Productivo (actores secundario y primario)
Tipo	Funcional
Precondición	El usuario tiene sesión activa. Existen registros de mortalidad históricos del lote en el sistema.
Flujo principal	1. El usuario navega al dashboard principal o a la sección 'Sanidad'.

	2. El sistema calcula la viabilidad acumulada de cada lote activo: (aves en existencia / aves al inicio del ciclo) × 100. 3. El sistema recupera el valor de referencia de la tabla genética del fabricante para la raza correspondiente. 4. El sistema muestra las barras de viabilidad con animación para cada lote: Lohmann Brown-Classic y Lohmann LSL-Classic. 5. Cada barra incluye el porcentaje real, el porcentaje de tabla y la diferencia en puntos porcentuales. 6. Si la diferencia supera -5pp: la barra se muestra en color naranja. 7. Si la diferencia supera -15pp: la barra se muestra en color rojo con alerta. 8. El usuario puede navegar a la sección Sanidad para ver el historial de mortalidad semana a semana.
Postcondición	El usuario visualizó la viabilidad real vs tabla de todos los lotes activos con indicadores de desvío.
Flujo alternativo / Excepción	Si el lote recién inició (semana 1): la viabilidad muestra 100% con una nota indicando que el lote está en fase inicial.
Prioridad	Alta

Vista estática (diagrama de clases)

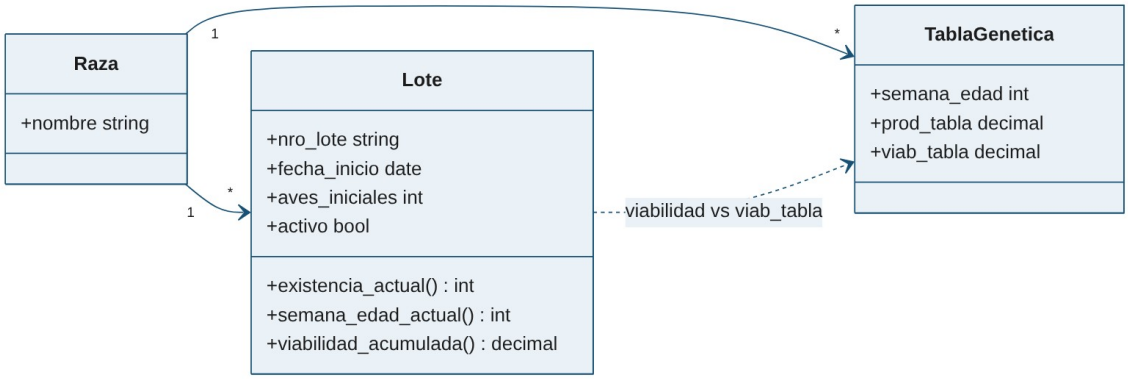


Diagrama de clases — CU14

Vista dinámica (diagrama de secuencia)

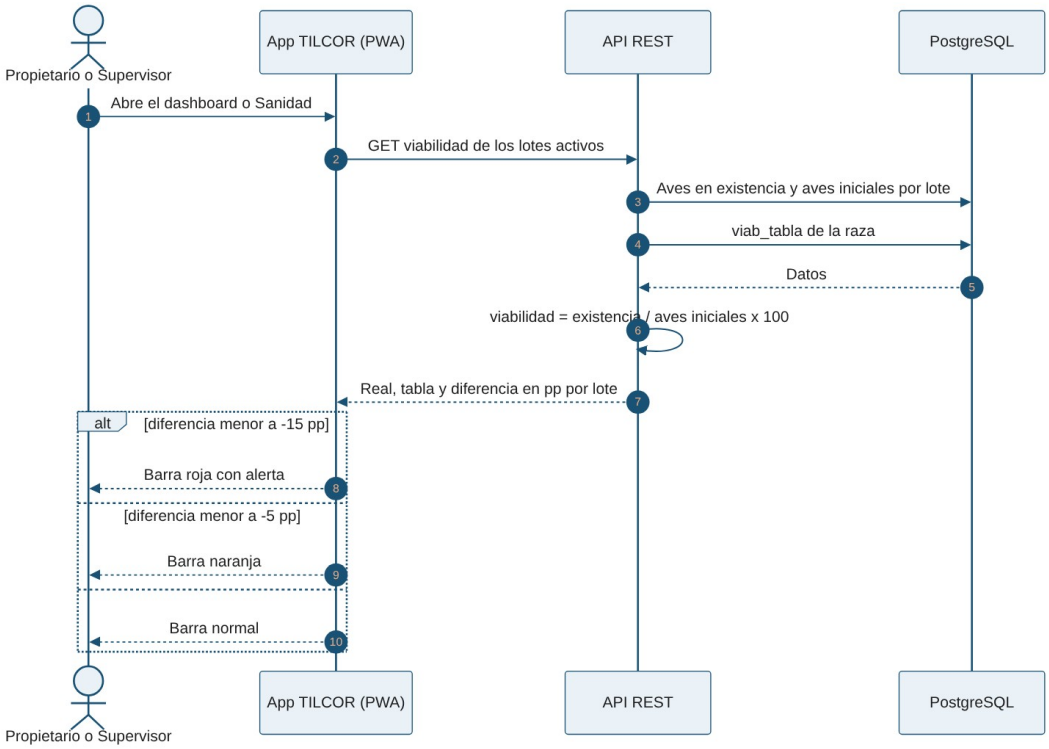


Diagrama de secuencia — CU14

6.3.3.2 CU16 — Recibir informe semanal de mortalidad

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-PROD-07	Informe al veterinario	Enviar automáticamente cada lunes a las 08:00 informe PDF de mortalidad.	Media
RF16	(Relevamiento)	Reporte semanal de produccion real vs tabla genetica.	—

Prototipos de interfaz



Boceto — no implementado

Ficha de caso de uso

FICHA DE CASO DE USO — CU16	
Nombre del CU	Recibir informe semanal de mortalidad y viabilidad
Actores	Veterinario Externo (actor secundario) · Sistema TILCOR — Celery (actor automático)
Tipo	Funcional
Precondición	El sistema tiene configurada la tarea automática semanal. Existen registros de mortalidad de la semana en la base de datos.
Flujo principal	1. Cada lunes a las 08:00, el scheduler Celery dispara la tarea de generación del informe semanal. 2. El sistema consolida los datos de mortalidad de la semana: muertas por día, mortalidad semanal %, existencia final. 3. El sistema calcula la viabilidad acumulada del lote actualizada al cierre de la semana. 4. El sistema genera el informe PDF con: resumen de bajas, curva de mortalidad acumulada, viabilidad vs tabla y alertas de la semana. 5. El sistema envía el informe al correo del veterinario externo configurado. 6. El veterinario recibe el correo con el informe adjunto. 7. El sistema registra el envío exitoso en el log de reportes.
Postcondición	El veterinario externo recibió el informe semanal de mortalidad con todos los indicadores sanitarios del lote.
Flujo alternativo /	Si no hay registros de la semana: el sistema envía un informe

Excepción	indicando 'Sin registros de mortalidad esta semana' y solicita verificación.
Prioridad	Media · Automatizado — sin intervención humana

Vista estática (diagrama de clases)

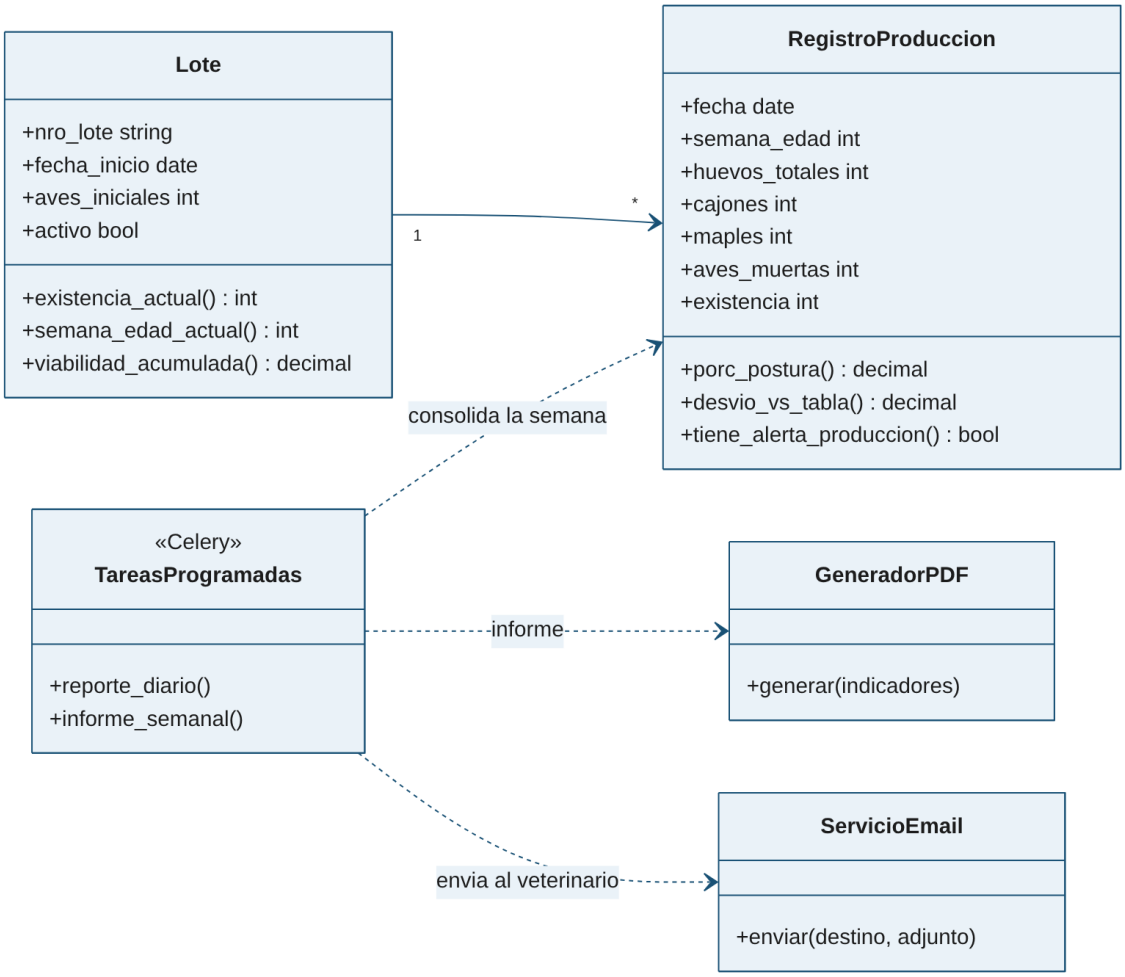


Diagrama de clases — CU16

Vista dinámica (diagrama de secuencia)

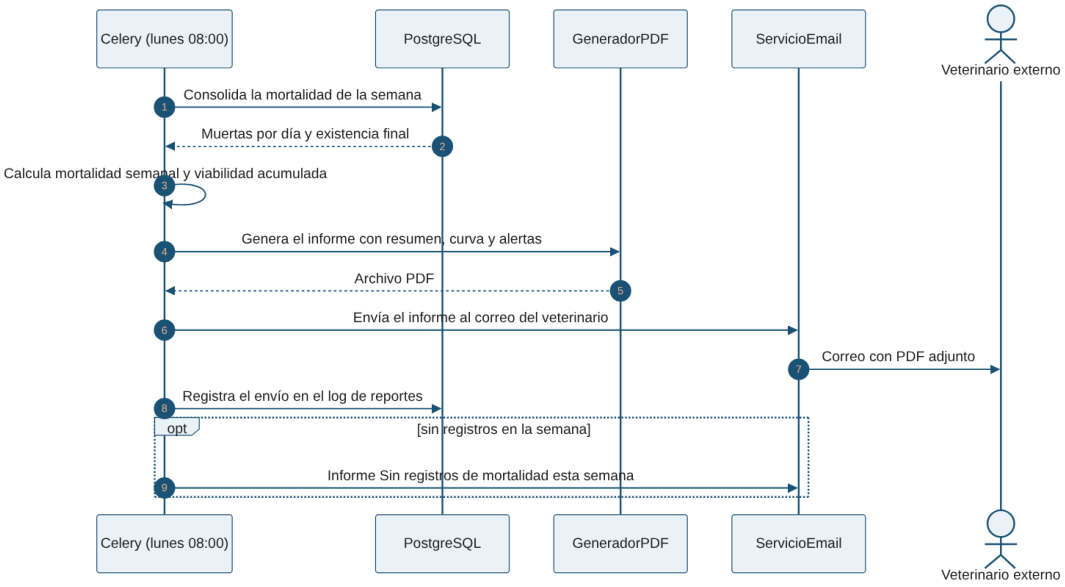


Diagrama de secuencia — CU16

6.3.3.3 CU17 — Consultar historial sanitario del lote

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-PROD-05	Control mortalidad	Calcular mortalidad semanal y alertar: Warning >0,5%, Crítico >1,0%.	Crítica

Prototipos de interfaz



Sanidad



Protocolo de alertas

Ficha de caso de uso

FICHA DE CASO DE USO — CU17	
Nombre del CU	Consultar historial sanitario del lote
Actores	Veterinario Externo · Supervisor Productivo (actores secundario y primario)
Tipo	Funcional
Precondición	El usuario tiene acceso al sistema con su rol correspondiente. Existen registros históricos de mortalidad en la base de datos.
Flujo principal	1. El usuario navega a la sección 'Sanidad' de la app. 2. El sistema muestra los KPIs de sanidad: mortalidad semanal, muertas totales del día, viabilidad Brown-Classic y viabilidad LSL-Classic. 3. El sistema muestra la alerta correspondiente si la mortalidad supera el umbral configurado. 4. El sistema renderiza la gráfica de mortalidad semanal histórica (semanas 76 a 89 del ciclo). 5. La gráfica muestra el porcentaje de mortalidad por semana con línea de tendencia. 6. El usuario puede ver el protocolo de alertas vigente (Normal, Advertencia, Crítico) con los umbrales. 7. El usuario puede comparar la evolución de la mortalidad con los picos identificados.
Postcondición	El usuario visualizó el historial sanitario completo del lote con gráfica histórica y comparación contra umbrales.
Flujo alternativo / Excepción	Si no hay registros históricos suficientes: el sistema muestra los datos disponibles con una nota indicando la cantidad de semanas relevadas.
Prioridad	Media

Vista estática (diagrama de clases)

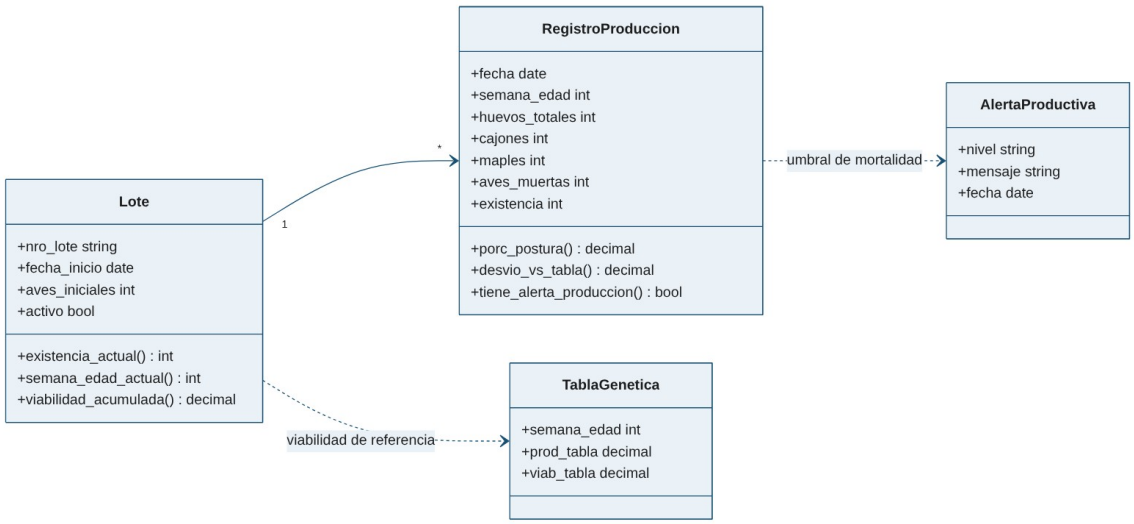


Diagrama de clases — CU17

Vista dinámica (diagrama de secuencia)

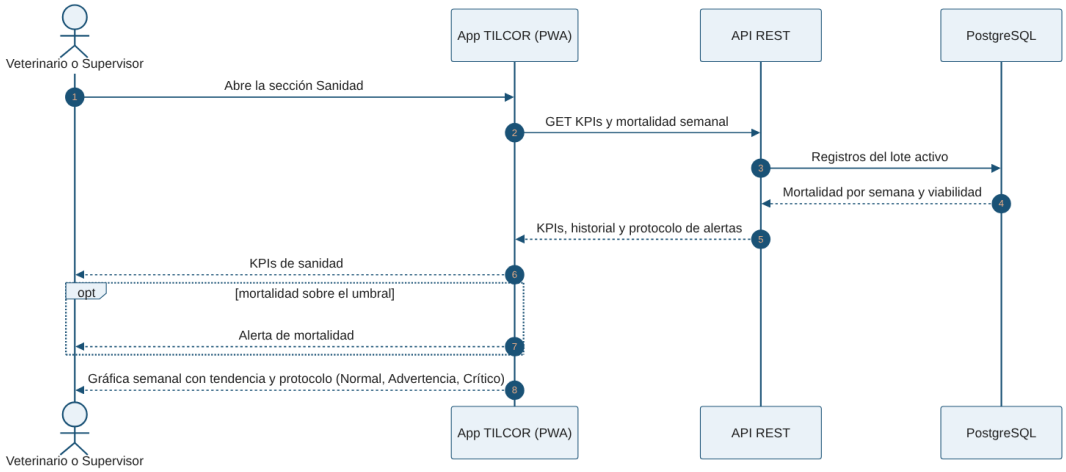


Diagrama de secuencia — CU17

6.3.3.4 CU18 — Ver curva de mortalidad histórica

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-PROD-05	Control mortalidad	Calcular mortalidad semanal y alertar: Warning >0,5%, Crítico >1,0%.	Crítica

Prototipos de interfaz



Curva de mortalidad

Ficha de caso de uso

FICHA DE CASO DE USO — CU18	
Nombre del CU	Ver curva de mortalidad histórica (semanas 76-89)
Actores	Veterinario Externo · Supervisor Productivo (actores secundario y primario)
Tipo	Funcional
Precondición	El usuario tiene acceso al sistema. El lote lleva al menos 5 semanas de registros de mortalidad cargados.
Flujo principal	1. El usuario navega a la sección 'Sanidad'. 2. El sistema recupera los registros de mortalidad semanal históricos del lote activo. 3. El sistema calcula el % de mortalidad semanal para cada semana relevada. 4. El sistema renderiza la gráfica de línea con el % de mortalidad semanal en el eje Y y las semanas de edad en el eje X. 5. La gráfica muestra los picos de mortalidad: semana 82 (0,45%), semana 86 (0,73%), semana 88 (0,77%). 6. El sistema muestra una línea de referencia horizontal en 0,5% (umbral Warning) en color naranja. 7. El usuario puede identificar visualmente los períodos de mortalidad elevada y correlacionarlos con eventos

	ambientales.
Postcondición	El usuario visualizó la curva histórica de mortalidad con los picos identificados y la línea de umbral de referencia.
Flujo alternativo / Excepción	Si hay menos de 3 semanas de datos: el sistema muestra los puntos disponibles con una nota indicando que se necesitan más semanas para identificar tendencias.
Prioridad	Media

Vista estática (diagrama de clases)

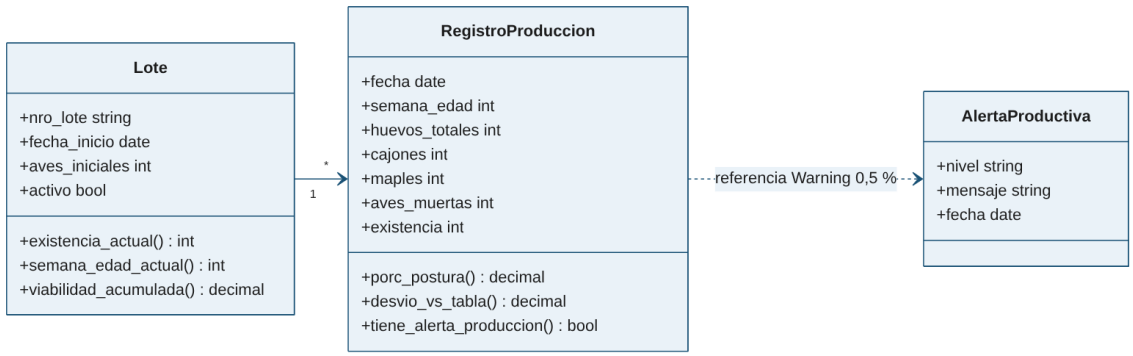


Diagrama de clases — CU18

Vista dinámica (diagrama de secuencia)

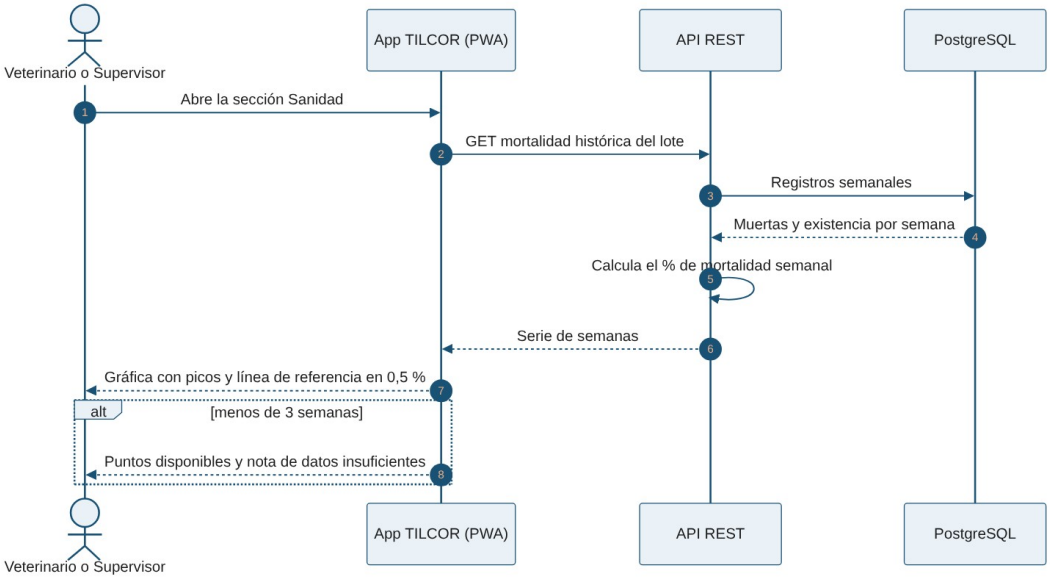


Diagrama de secuencia — CU18

6.3.4 Módulo 4 — Dashboard, reportes y plan de implementación

6.3.4.1 CU10 — Editar progreso del plan de implementación

Requerimientos específicos

Sin requerimiento funcional numerado: surge de la necesidad del equipo de seguimiento del avance del proyecto (ver plan de implementación, sección 3.2.5).

Prototipos de interfaz



Edición del plan

Ficha de caso de uso

FICHA DE CASO DE USO — CU10	
Nombre del CU	Editar progreso del plan de implementación
Actores	Supervisor Productivo (actor primario con rol Admin)
Tipo	Funcional
Precondición	El usuario tiene sesión activa con rol Admin. El dashboard principal está visible.
Flujo principal	1. El usuario localiza la sección 'Plan de implementación' en el dashboard. 2. El usuario toca el botón 'Editar' en la card del plan. 3. El sistema abre un modal con un slider por cada una de las 5 fases del proyecto. 4. Cada slider muestra el progreso actual de la fase (0% a

	100% en pasos de 5%). 5. El usuario mueve los sliders para actualizar el porcentaje de cada fase. 6. El porcentaje se actualiza visualmente en tiempo real mientras se mueve el slider. 7. El usuario toca 'Guardar cambios'. 8. El sistema guarda el progreso en localStorage y actualiza las barras del plan con animación.
Postcondición	El progreso de las fases quedó actualizado y guardado. Las barras reflejan los nuevos valores con animación.
Flujo alternativo / Excepción	Si el usuario cierra el modal sin guardar: los cambios se descartan y se mantiene el progreso anterior.
Prioridad	Media

Vista estática (diagrama de clases)

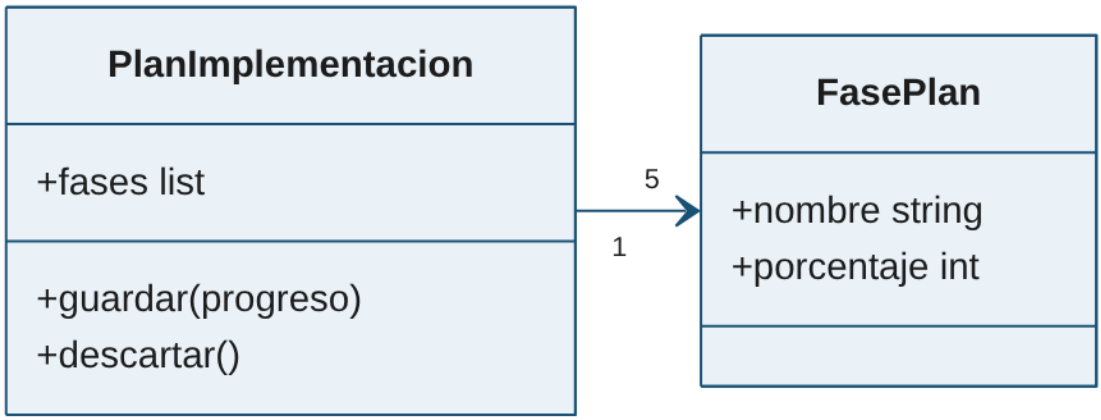


Diagrama de clases — CU10

Vista dinámica (diagrama de secuencia)

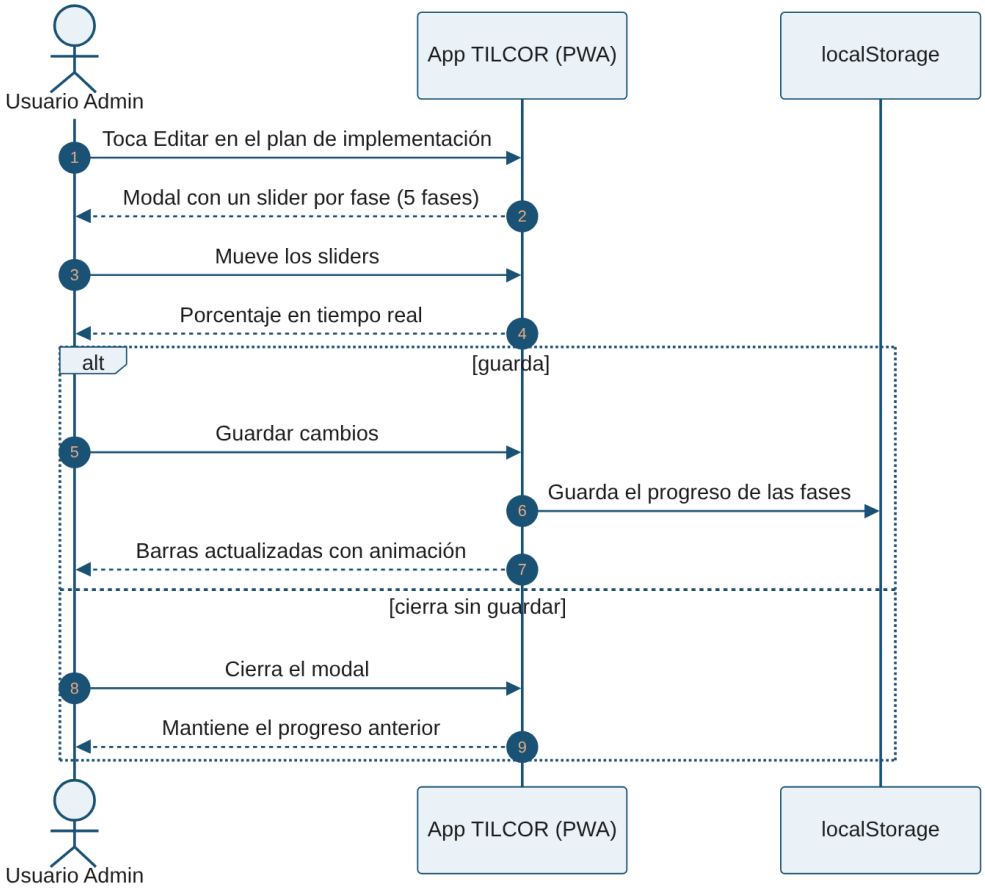


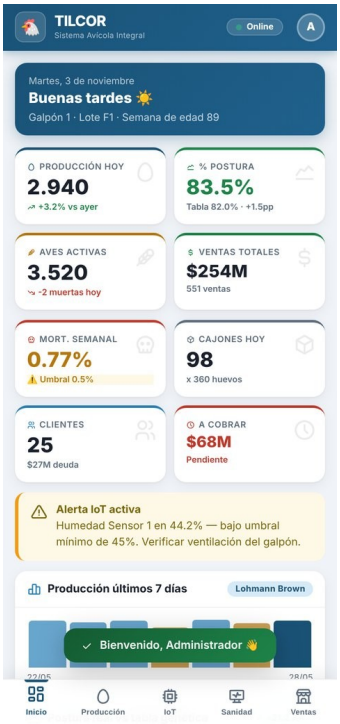
Diagrama de secuencia — CU10

6.3.4.2 CU12 — Ver KPIs generales en dashboard principal

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF-IOT-04	Semáforo visual	Mostrar verde/amarillo/rojo por sensor según su estado respecto a los umbrales.	Alta
RF-PROD-08	Historial 7 días	Mostrar últimos 7 registros con fecha, indicadores y desvío en pp.	Alta
RNF-03	Usabilidad	Interfaz operable con una sola mano en celular. Criterio: Todos los elementos accesibles con pulgar	—

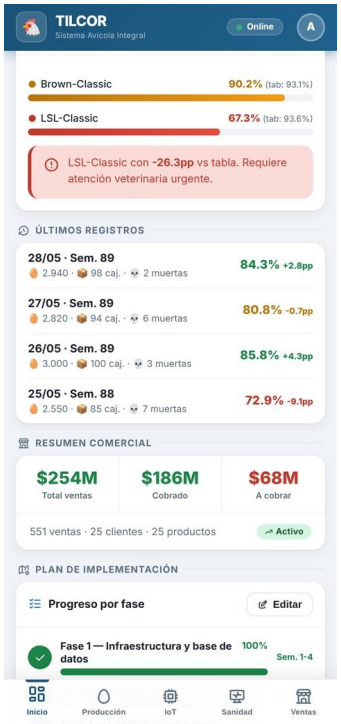
Prototipos de interfaz



Inicio



Indicadores



Resumen

Ficha de caso de uso

FICHA DE CASO DE USO — CU12	
Nombre del CU	Ver KPIs generales en dashboard principal
Actores	Propietario · Supervisor Productivo · Operario (todos los actores humanos)
Tipo	Funcional
Precondición	El usuario tiene sesión activa. El dashboard principal está cargado.
Flujo principal	1. El usuario abre la app TILCOR — el dashboard es la pantalla de inicio por defecto. 2. El sistema muestra el saludo dinámico con la hora y fecha actual. 3. El sistema renderiza los 8 KPIs principales en dos filas de 4 tarjetas: producción del día, % postura, aves activas, ventas totales, mortalidad semanal, cajones del día, clientes y monto a cobrar. 4. El sistema muestra el gráfico de barras de producción de los últimos 7 días. 5. El sistema muestra la curva real vs tabla genética (CU09). 6. El sistema muestra las barras de viabilidad de los dos lotes activos con animación de entrada. 7. Si hay alertas activas: el sistema muestra el banner de alerta correspondiente al tope del contenido. 8. El sistema muestra el resumen comercial con los datos de Don Gestión y el plan de implementación.

Postcondición	El usuario visualizó todos los KPIs actualizados del sistema en una sola pantalla.
Flujo alternativo / Excepción	Sin datos de producción del día: los KPIs muestran los datos del último registro disponible con indicación de la fecha.
Prioridad	Alta

Vista estática (diagrama de clases)

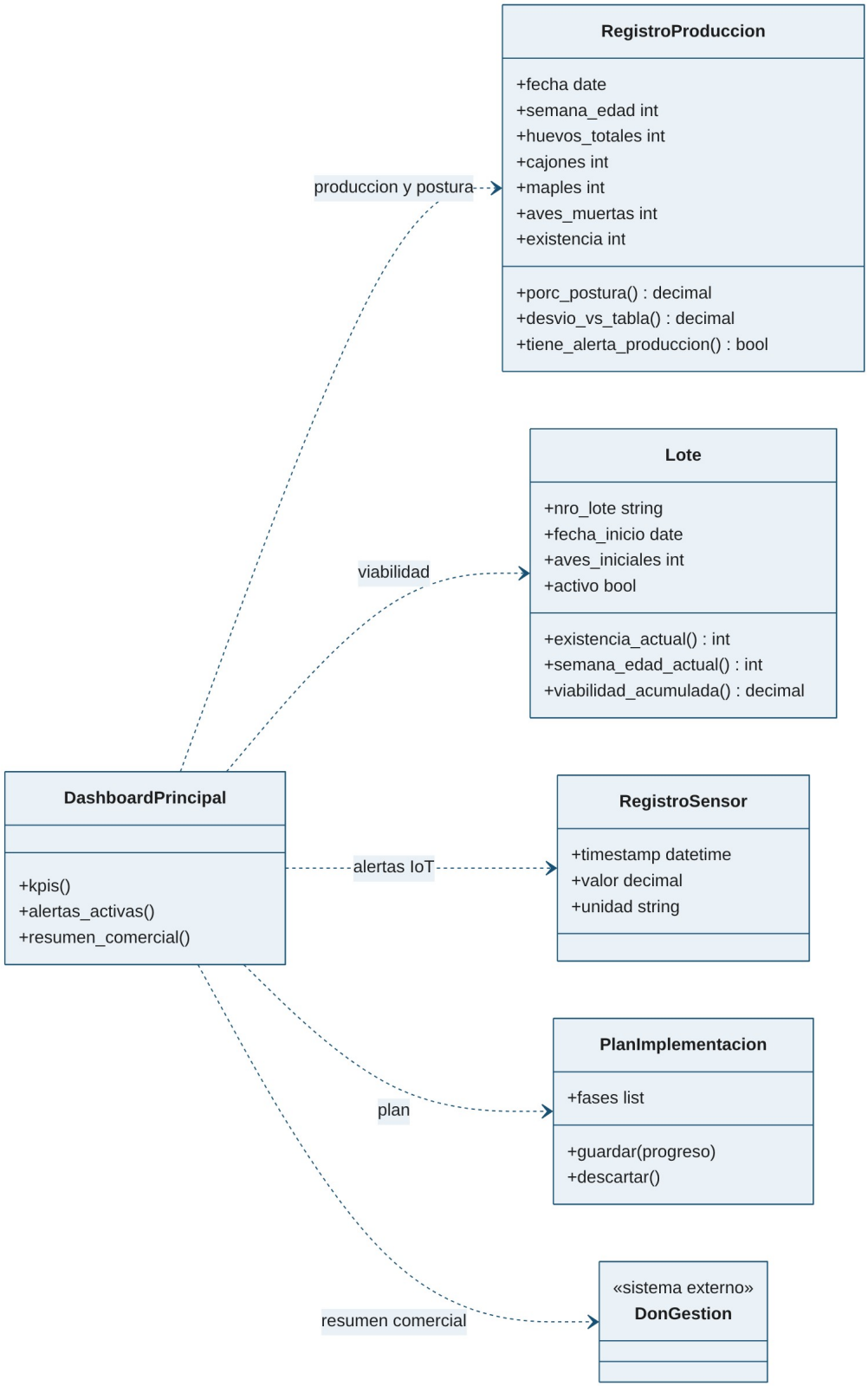


Diagrama de clases — CU12

Vista dinámica (diagrama de secuencia)

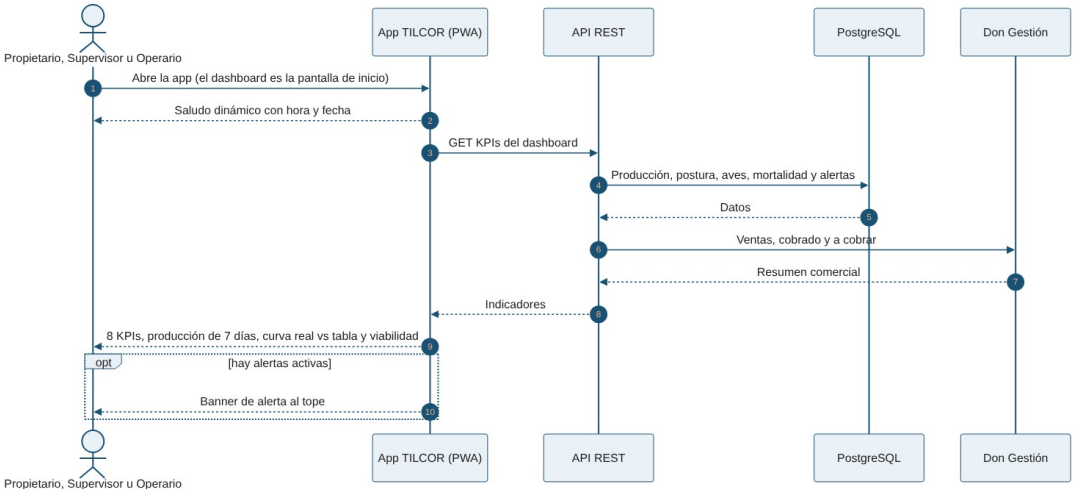


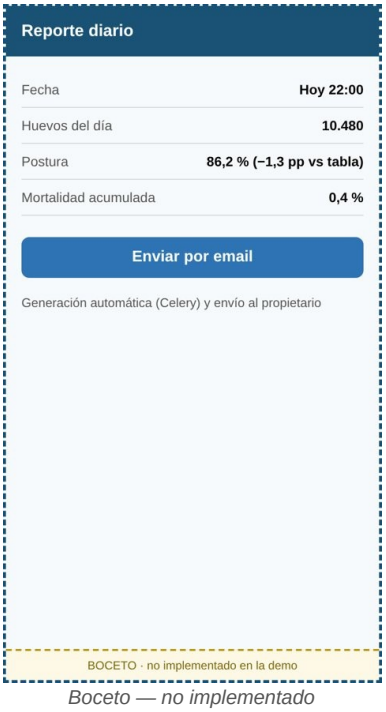
Diagrama de secuencia — CU12

6.3.4.3 CU13 — Recibir reporte diario automatizado

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF15	(Relevamiento)	Reporte diario automatico enviado al propietario al cierre del dia.	—
RF17	(Relevamiento)	Exportacion a Excel y PDF de todos los modulos.	—

Prototipos de interfaz



Boceto — no implementado

Ficha de caso de uso

FICHA DE CASO DE USO — CU13	
Nombre del CU	Recibir reporte diario automatizado de producción

Actores	Propietario (actor secundario) · Sistema TILCOR — Celery (actor automático)
Tipo	Funcional
Precondición	El sistema tiene configurada la tarea automática de reporte diario. Existen registros del día en la base de datos.
Flujo principal	1. Al cierre del día (22:00 hs), el scheduler de Celery dispara la tarea de generación de reporte. 2. El sistema consulta todos los registros de producción del día: huevos, cajones, muertas, existencia, % postura. 3. El sistema calcula los indicadores del día: desvío vs tabla, mortalidad acumulada, tendencia semanal. 4. El sistema genera el reporte en formato PDF con los indicadores del día, gráficas y alertas registradas. 5. El sistema envía el reporte PDF al correo electrónico del propietario configurado. 6. El propietario recibe el correo con el reporte adjunto. 7. El sistema registra el envío exitoso en el log de reportes.
Postcondición	El propietario recibió el reporte diario por email con todos los indicadores productivos del día.
Flujo alternativo / Excepción	Si no hay registros del día: el sistema envía un reporte indicando que no se registró producción y solicita verificación al operario.
Prioridad	Alta · Automatizado — sin intervención humana

Vista estática (diagrama de clases)

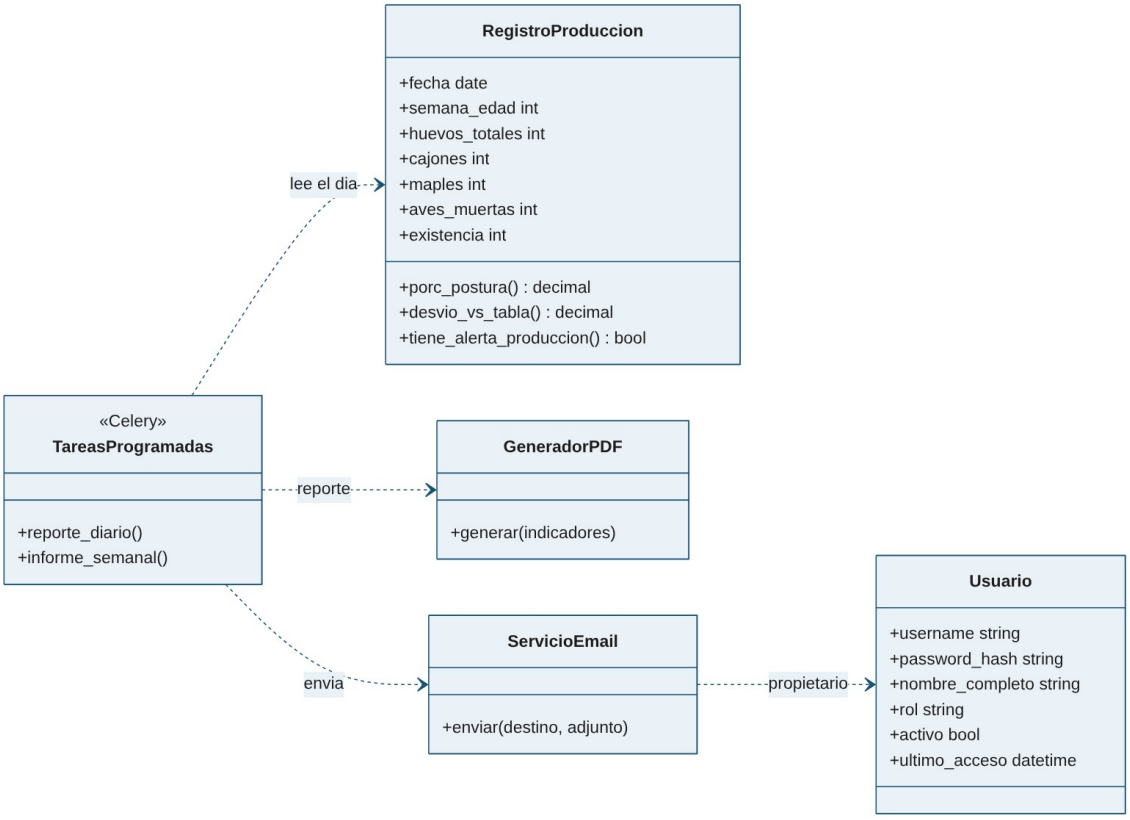


Diagrama de clases — CU13

Vista dinámica (diagrama de secuencia)

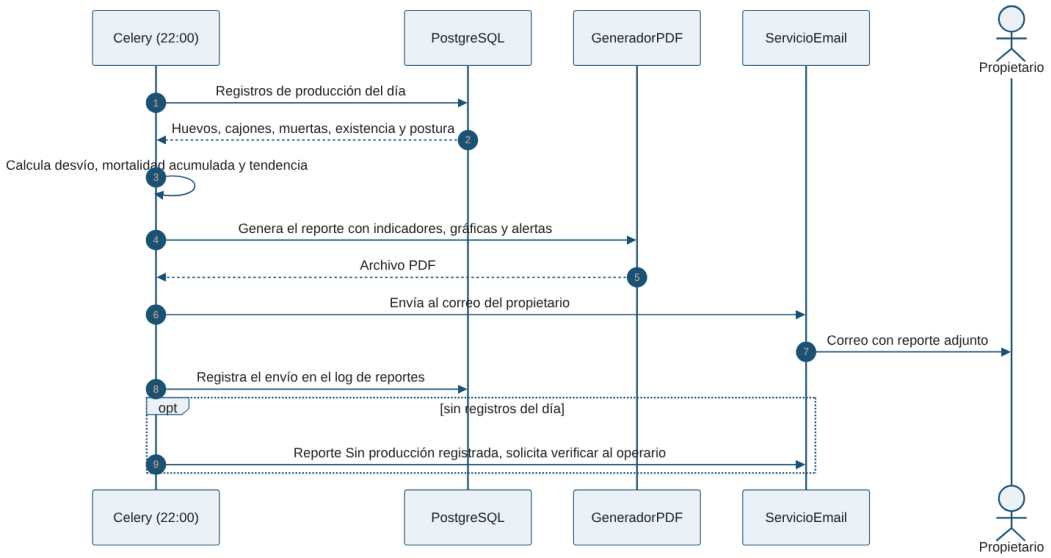


Diagrama de secuencia — CU13

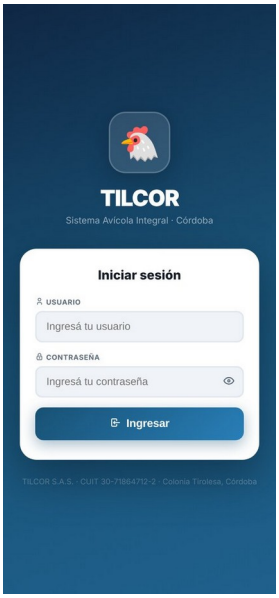
6.3.5 Módulo 5 — Usuarios y roles

6.3.5.1 CU15 — Iniciar sesión con rol diferenciado

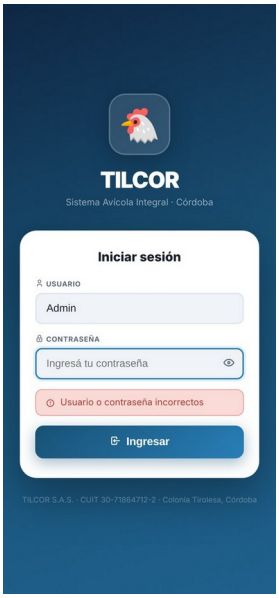
Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RNF-05	Seguridad	Autenticación con roles y transmisión cifrada. Criterio: JWT + HTTPS obligatorio	—
RNF-09	Trazabilidad	Audit log de toda operación de escritura. Criterio: Log con usuario, timestamp, datos anteriores	—

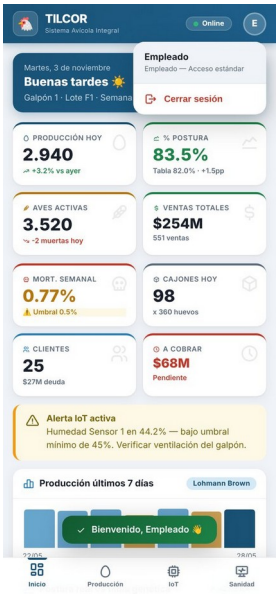
Prototipos de interfaz



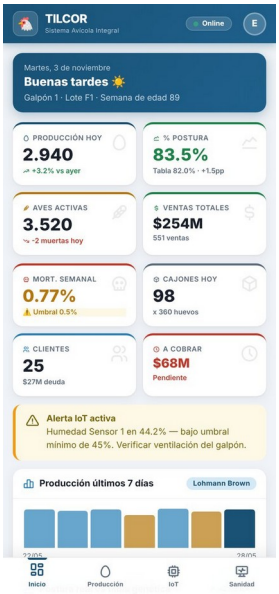
Login



Error de login



Perfil empleado



Empleado sin Ventas

Ficha de caso de uso

FICHA DE CASO DE USO — CU15	
Nombre del CU	Iniciar sesión con rol diferenciado (Admin / Empleado)
Actores	Todos los actores humanos: Operario, Supervisor, Propietario
Tipo	Funcional
Precondición	El usuario tiene la app TILCOR abierta en la pantalla de login. No tiene sesión activa.
Flujo principal	1. El usuario visualiza la pantalla de login con logo TILCOR, campo de usuario y contraseña. 2. El usuario ingresa su nombre de usuario y contraseña. 3. El usuario puede tocar el ícono de ojo para mostrar/ocultar la contraseña. 4. El usuario toca el botón 'Ingresar' o presiona Enter. 5. El sistema verifica las credenciales contra la tabla de usuarios registrados.

	6. Si son correctas: el sistema establece la sesión con el rol correspondiente (Admin o Empleado). 7. El rol Admin habilita todos los módulos incluyendo Ventas/Don Gestión. 8. El rol Empleado oculta el tab de Ventas y su sección del sistema. 9. El sistema muestra el dashboard con los módulos habilitados y un toast de bienvenida con el nombre del usuario.
Postcondición	El usuario tiene sesión activa con su rol asignado. Los módulos disponibles corresponden a sus permisos.
Flujo alternativo / Excepción	Credenciales incorrectas: el sistema muestra mensaje de error en rojo con animación shake y limpia el campo de contraseña.
Prioridad	Crítica · Requisito base para todos los demás casos de uso

Vista estática (diagrama de clases)

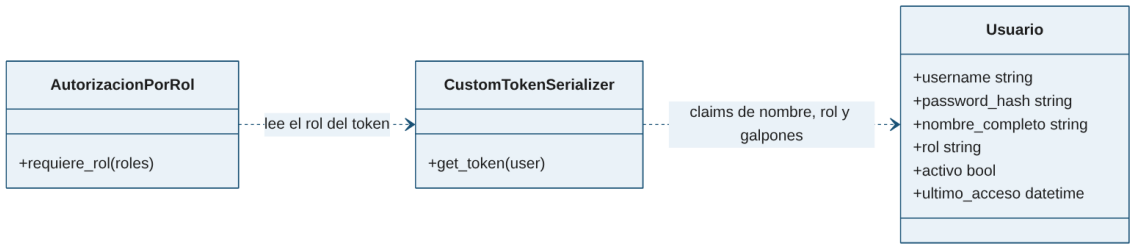


Diagrama de clases — CU15

Vista dinámica (diagrama de secuencia)

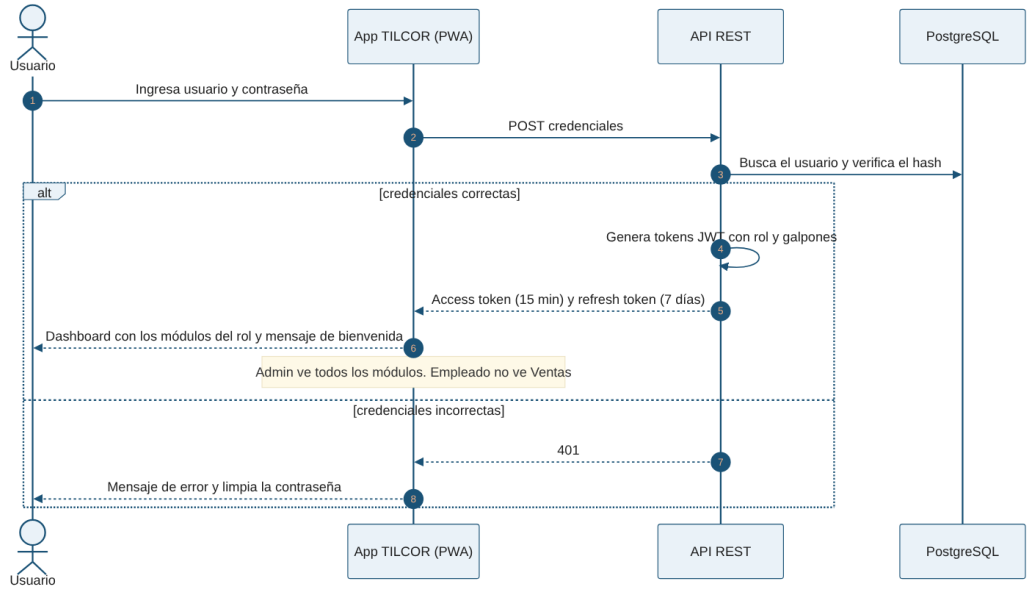


Diagrama de secuencia — CU15

6.3.6 Módulo 6 — Gestión comercial (Don Gestión)

6.3.6.1 CU11 — Aprobar pedido que supera límite de crédito

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF01	(Relevamiento)	Alta, baja logica y modificacion de clientes (razon social, CUIT, condicion IVA, categoria, limite de credito).	—
RF05	(Relevamiento)	Cuenta corriente con saldo en tiempo real y alerta de limite de credito.	—

Prototipos de interfaz



Boceto — no implementado

Ficha de caso de uso

FICHA DE CASO DE USO — CU11	
Nombre del CU	Aprobar pedido que supera límite de crédito del cliente
Actores	Supervisor Productivo (actor primario)
Tipo	Funcional
Precondición	El vendedor creó un pedido para un cliente cuyo monto total supera el límite de crédito autorizado. El sistema genera una alerta de aprobación.
Flujo principal	1. El sistema detecta que el monto del pedido nuevo supera el límite de crédito del cliente. 2. El sistema genera una notificación de aprobación pendiente al supervisor. 3. El supervisor recibe la notificación con los detalles: cliente,

	monto del pedido, saldo de cuenta corriente y límite de crédito. 4. El supervisor evalúa la situación del cliente y su historial de pagos en Don Gestión. 5. El supervisor decide aprobar o rechazar el pedido. 6. Si aprueba: el sistema confirma el pedido y continúa el flujo de venta. 7. Si rechaza: el sistema notifica al vendedor con el motivo del rechazo.
Postcondición	El pedido quedó aprobado o rechazado según la decisión del supervisor. El vendedor fue notificado del resultado.
Flujo alternativo / Excepción	Si el supervisor no responde en 30 minutos: el sistema envía recordatorio. Si no responde en 2 horas: escala al propietario.
Prioridad	Alta · << extend >> CU01 comercial

Vista estática (diagrama de clases)

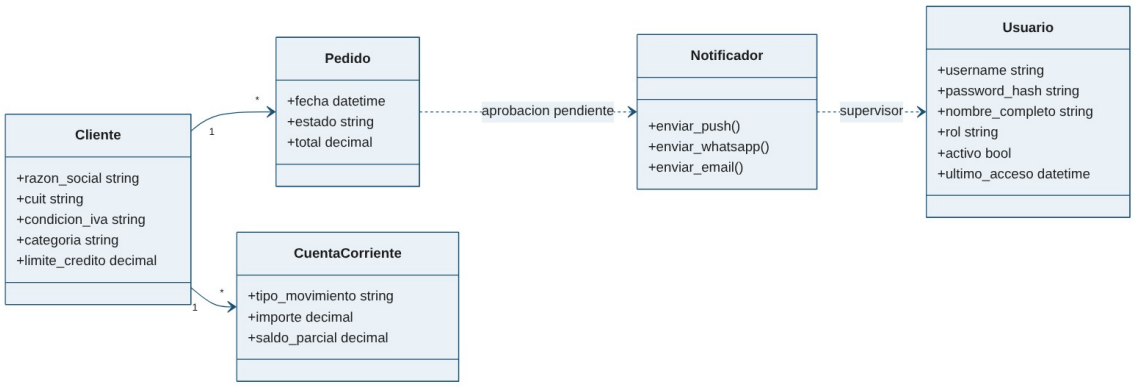


Diagrama de clases — CU11

Vista dinámica (diagrama de secuencia)

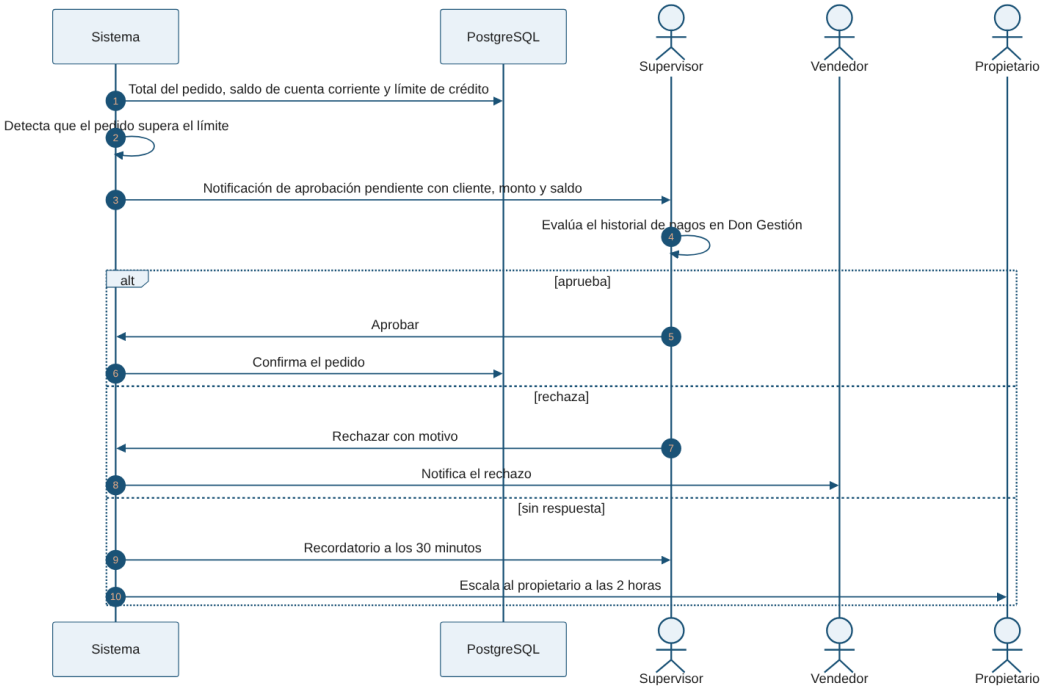


Diagrama de secuencia — CU11

6.3.6.2 CU23 — Actualizar datos comerciales (Don Gestión)

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF02	(Relevamiento)	Historial completo de pedidos y compras por cliente.	—
RF05	(Relevamiento)	Cuenta corriente con saldo en tiempo real y alerta de limite de credito.	—

Prototipos de interfaz



Boceto — no implementado

Ficha de caso de uso

FICHA DE CASO DE USO — CU23	
Nombre del CU	Actualizar datos comerciales en tiempo real desde Don Gestión
Actores	Don Gestión (actor sistema externo) · Supervisor Productivo · Vendedor
Tipo	Funcional
Precondición	El usuario tiene sesión activa con rol Admin. El iframe de Don Gestión está disponible en el módulo Ventas.
Flujo principal	1. El usuario navega a la sección 'Ventas' de la app TILCOR (solo disponible para rol Admin). 2. El sistema carga el iframe de Don Gestión apuntando a la URL de la empresa (avicola-tilcor.dongestion.com). 3. El sistema muestra el loader mientras Don Gestión carga sus datos. 4. Don Gestión carga el módulo seleccionado (Ventas, Clientes, Cuentas, Productos o Informes) con los datos reales de TILCOR. 5. El usuario interactúa directamente con Don Gestión dentro del iframe de la app TILCOR. 6. La barra de navegación interna permite cambiar entre módulos de Don Gestión sin salir de la app. 7. El usuario puede abrir Don Gestión en pantalla completa tocando el ícono de nueva pestaña.
Postcondición	El usuario accedió a los datos comerciales reales de TILCOR a través de Don Gestión integrado en la app.
Flujo alternativo / Excepción	Si Don Gestión bloquea el iframe por X-Frame-Options: el sistema muestra el error y ofrece abrir en nueva pestaña.
Prioridad	Alta · Integración via iframe — datos reales de Don Gestión

Vista estática (diagrama de clases)

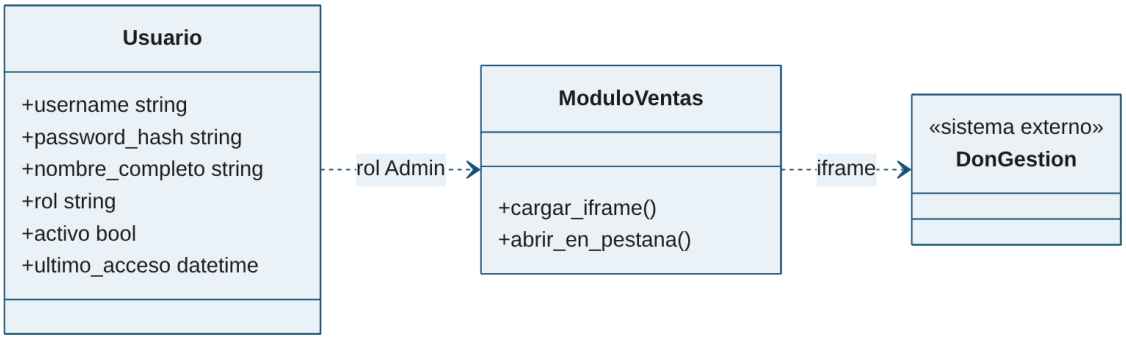


Diagrama de clases — CU23

Vista dinámica (diagrama de secuencia)

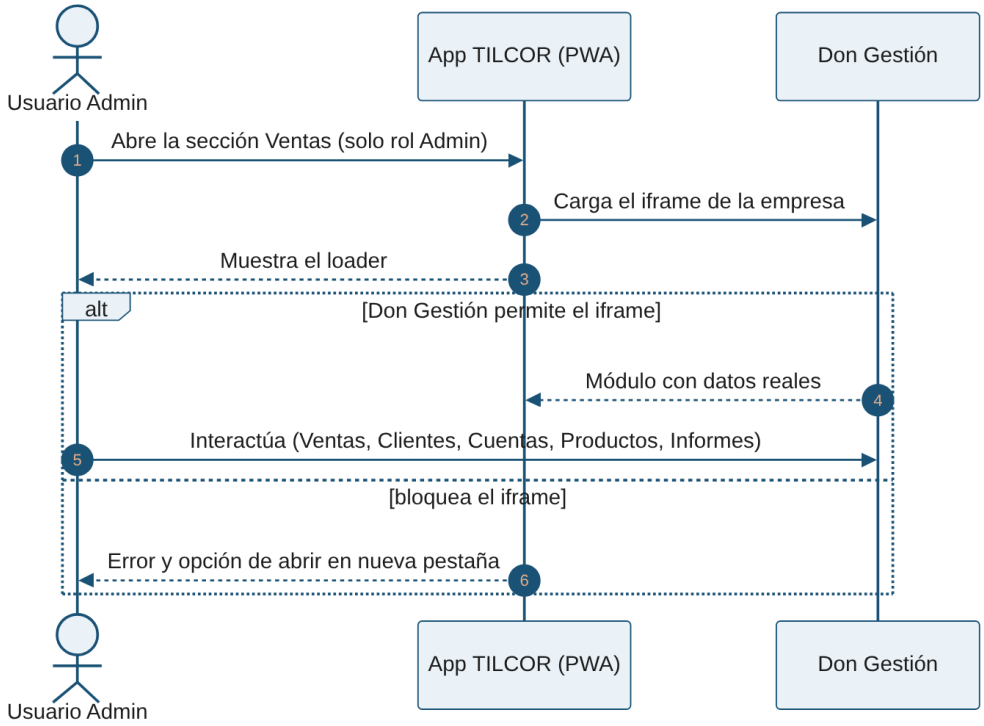


Diagrama de secuencia — CU23

6.3.6.3 CU24 — Emitir factura electrónica ARCA/AFIP

Requerimientos específicos

ID	Requerimiento	Descripción	Prioridad
RF04	(Relevamiento)	Emision de remitos PDF y facturas electronicas integradas con AFIP-WSFE.	—

Prototipos de interfaz

No aplica: la emisión se realiza en Don Gestión y ARCA/AFIP, sin pantalla en esta aplicación.

Ficha de caso de uso

FICHA DE CASO DE USO — CU24	
Nombre del CU	Emitir factura electrónica via ARCA/AFIP (extiende CU23)
Actores	Don Gestión (actor sistema externo) · Sistema AFIP/ARCA · Vendedor
Tipo	Funcional
Precondición	El vendedor confirmó la entrega de un pedido en Don Gestión. El cliente tiene configurada su condición de IVA en el sistema.
Flujo principal	1. El vendedor confirma la entrega del pedido en Don Gestión. 2. Don Gestión determina el tipo de comprobante según la condición de IVA del cliente: Factura A (Resp. Inscripto), B (Cons. Final) o C (Exento/Monotributista). 3. Don Gestión llama al servicio WSFE de ARCA/AFIP via protocolo SOAP con los datos del comprobante.

	4. ARCA/AFIP valida los datos y devuelve el CAE (Código de Autorización Electrónico) con su fecha de vencimiento. 5. Don Gestión almacena el CAE junto al comprobante y actualiza la cuenta corriente del cliente. 6. Don Gestión genera el PDF de la factura con el CAE impreso. 7. El sistema envía automáticamente el PDF de la factura por email al cliente. 8. El movimiento queda registrado en el flujo de caja de Don Gestión.
Postcondición	Factura electrónica emitida con CAE válido. Cuenta corriente del cliente actualizada. PDF enviado al cliente por email.
Flujo alternativo / Excepción	Si ARCA/AFIP no responde o devuelve error: Don Gestión guarda el comprobante como pendiente de autorización y reintenta en 5 minutos.
Prioridad	Alta · << extend >> CU23 — Se ejecuta condicionalmente al confirmar entrega

Vista estática (diagrama de clases)

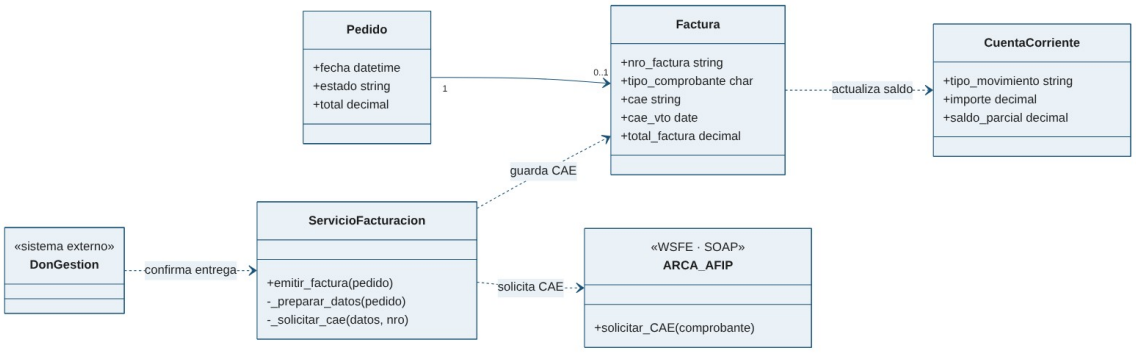


Diagrama de clases — CU24

Vista dinámica (diagrama de secuencia)

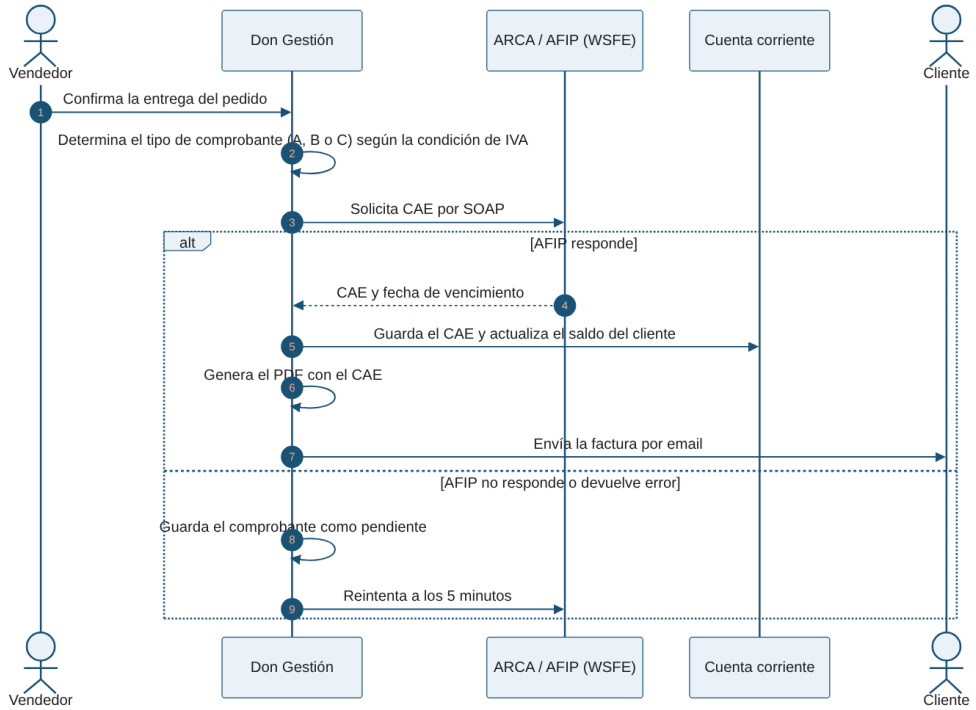


Diagrama de secuencia — CU24

7 Implementación del software (desarrollo)

Este capítulo documenta cómo se llevó cada caso de uso a código. Se distinguen tres estados, para que se vea con claridad qué está construido y qué está diseñado:

- **Implementado y probado:** código real en el repositorio con pruebas automáticas (módulo IoT y auditoría SQL).
- **Implementado en la demo:** funciona en la aplicación publicada (PWA), con datos de ejemplo guardados en el navegador, sin backend.
- **Diseñado:** especificado con su ficha, diagramas y código de referencia, pendiente de construir en las fases 3 a 5 del plan.

Módulo	Caso de uso	Estado
1. Producción avícola	CU01 — Registrar producción diaria de huevos	Implementado en la demo
1. Producción avícola	CU02 — Registrar mortalidad diaria de aves	Implementado en la demo
1. Producción avícola	CU04 — Ver historial de producción por fecha	Implementado en la demo
1. Producción avícola	CU09 — Ver curva de postura real vs tabla genética	Implementado en la demo (datos de ejemplo)
2. Monitoreo IoT y alertas	CU03 — Consultar estado actual de sensores IoT	Demo con datos simulados
2. Monitoreo IoT y alertas	CU05 — Configurar umbrales de alerta por sensor	Umbrales implementados; pantalla diseñada
2. Monitoreo IoT y alertas	CU06 — Recibir alerta Warning	Evaluación implementada y probada; notificación diseñada
2. Monitoreo IoT y alertas	CU07 — Recibir alerta Crítico	Evaluación implementada y probada; notificación diseñada
2. Monitoreo IoT y alertas	CU08 — Consultar dashboard IoT en tiempo real	Demo con datos simulados
2. Monitoreo IoT y alertas	CU19-22 — Flujo automático IoT (CU19 a CU22)	Implementado y probado con simulador
3. Sanidad y mortalidad	CU14 — Ver viabilidad acumulada de los lotes	Implementado en la demo (datos de ejemplo)
3. Sanidad y mortalidad	CU16 — Recibir informe semanal de mortalidad	Diseñado
3. Sanidad y mortalidad	CU17 — Consultar historial sanitario del lote	Implementado en la demo (datos de ejemplo)
3. Sanidad y mortalidad	CU18 — Ver curva de mortalidad histórica	Implementado en la demo (datos de ejemplo)
4. Dashboard, reportes y plan de implementación	CU10 — Editar progreso del plan de implementación	Implementado en la demo
4. Dashboard, reportes y plan de implementación	CU12 — Ver KPIs generales en dashboard principal	Implementado en la demo (datos de ejemplo)
4. Dashboard, reportes y plan de implementación	CU13 — Recibir reporte diario automatizado	Diseñado
5. Usuarios y roles	CU15 — Iniciar sesión con rol diferenciado	Demo implementada; JWT diseñado; auditoría SQL implementada
6. Gestión comercial (Don Gestión)	CU11 — Aprobar pedido que supera límite de crédito	Diseñado
6. Gestión comercial (Don Gestión)	CU23 — Actualizar datos comerciales (Don Gestión)	Implementado en la demo
6. Gestión comercial (Don Gestión)	CU24 — Emitir factura electrónica ARCA/AFIP	Diseñado

Repositorio: github.com/Mcarnielli/Ingenieria-en-SFT-Tilcor · Demo: tilcorvercel.vercel.app

7.1 Módulo 1 — Producción avícola

7.1.1 CU01 — Registrar producción diaria de huevos

Estado: Implementado en la demo
Demo (frontend/index.html): sección Producción con cálculo de % de postura y alerta de desvío.

frontend/index.html — función calcPost

```
function calcPost(){
  const h=parseInt($('rHuevos').value)||0;
  const e=parseInt($('rExist').value)||1;
  const p=(h/e*100).toFixed(2);
  const el=$('postCalc');
  el.textContent=p+'%';
  if(p>=80){el.style.color='var(--gn)';el.style.borderColor='var(--gn)'}
  else if(p>=75){el.style.color='var(--am)';el.style.borderColor='#F39C12'}
  else{el.style.color='var(--rd)';el.style.borderColor='var(--rd)'}
}
```

frontend/index.html — función guardarReg

```
function guardarReg(){
  const h=parseInt($('rHuevos').value)||0;
  const e=parseInt($('rExist').value)||0;
  const m=parseInt($('rMuertas').value)||0;
  if(!h||!e){toast('Completá huevos y existencia','err');return}
  const pos=e>0?(h/e*100).toFixed(2):0;
  const fecha=new Date().toLocaleDateString('es-AR',{day:'2-digit',month:'2-digit'});
  PROD_DATA.push({fecha,sem:parseInt($('rSem').value)||89,
    h,caj:parseInt($('rCaj').value)||0,m,ex:e,pos:parseFloat(pos),tab:81.5});
  renderProdLog();renderMiniChart();
  if(parseFloat(pos)<77){
    pendingAction=()=>toast('✅ Alerta enviada al supervisor');
    $('modalTitle').textContent='⚠️ Alerta de producción';
    $('modalMsg').textContent=`El % de postura (${pos}%) está más de 5pp bajo la tabla genética.
¿Notificamos al supervisor?`;
    openModal('modalConfirm');
  } else {
    toast('✅ Registro guardado correctamente');
  }
  $('rObs').value='';
}
```

7.1.2 CU02 — Registrar mortalidad diaria de aves

Estado: Implementado en la demo
Demo: campo de aves muertas y verificación de mortalidad al guardar.

frontend/index.html — función checkMort

```
function checkMort(){
  const m=parseInt($('rMuertas').value)||0;
  const alertEl=$('mortAlert');
  if(m>17){
    alertEl.style.display='flex';
    $('mortAlertMsg').textContent=`${m} muertas — mortalidad estimada supera umbral. Revisar lote.`;
  } else alertEl.style.display='none';
}
```

7.1.3 CU04 — Ver historial de producción por fecha

Estado: Implementado en la demo
Demo: historial de los últimos registros con semáforo de desvío.

frontend/index.html — función renderProdLog

```
function renderProdLog(){
  $('prodLog').innerHTML=PROD_DATA.slice().reverse().slice(0,7).map((r,i)=>{
    const dev=(r.pos-r.tab).toFixed(2);
    const dc=dev>=0?'var(--gn)':dev<-5?'var(--rd)':'var(--am)';
    return `<div class="prod-row" style="animation-delay:${i*.06}s">
      <div class="prod-row-hdr">
        <span class="prod-row-date">📅 ${r.fecha} · Sem. ${r.sem}</span>
```

```
        <span class="prod-row-pct" style="color:${dc}">${r.pos.toFixed(2)}% <small
style="color:var(--gy);font-weight:400">(${dev>=0?'+':''}${dev}pp)</small></span>
      </div>
      <div class="prod-row-details">
        <span class="prod-detail">🥚 <span>${r.h.toLocaleString()}</span></span>
        <span class="prod-detail">📦 <span>${r.caj} caj.</span></span>
        <span class="prod-detail">💀 <span>${r.m}</span></span>
        <span class="prod-detail">🐔 <span>${r.ex.toLocaleString()}</span></span>
      </div>
    </div>`;
  }).join('');
}
```

7.1.4 CU09 — Ver curva de postura real vs tabla genética

Estado: Implementado en la demo (datos de ejemplo)
Demo: gráfica de postura real vs tabla genética en el inicio.

7.1.5 Diseño y código de referencia (Tesis)

7.1.5.1 Modelos de Dominio — Código Fuente

> *Los modelos encapsulan la logica de negocio avicola: calculo de % de postura, desvios vs tabla genetica del fabricante y deteccion de alertas productivas.*

```
# models/produccion.py
# ======================================================================
# Modelos del dominio — Modulo de Produccion Avicola
# Framework: Django ORM  |  Python 3.11+
# ======================================================================

from django.db import models
from django.core.validators import MinValueValidator, MaxValueValidator
from decimal import Decimal
import datetime


class Raza(models.Model):
    nombre     = models.CharField(max_length=100, unique=True)
    fabricante = models.CharField(max_length=100, blank=True)
    descripcion = models.TextField(blank=True)

    def __str__(self):
        return self.nombre


class TablaGenetica(models.Model):
    """Curva estandar del fabricante por semana de edad."""
    raza        = models.ForeignKey(Raza, on_delete=models.CASCADE,
                                    related_name='tabla_genetica')
    semana_edad = models.PositiveIntegerField(
                    validators=[MinValueValidator(15), MaxValueValidator(115)])
    prod_tabla  = models.DecimalField(max_digits=5, decimal_places=2)
    viab_tabla  = models.DecimalField(max_digits=5, decimal_places=2)
    peso_huevo_g = models.DecimalField(max_digits=5, decimal_places=1,
                    null=True, blank=True)

    class Meta:
        unique_together = ('raza', 'semana_edad')
```

```
        ordering = ['raza', 'semana_edad']


class Galpon(models.Model):
    numero        = models.PositiveIntegerField(unique=True)
    descripcion   = models.CharField(max_length=200, blank=True)
    capacidad_max = models.PositiveIntegerField()
    activo        = models.BooleanField(default=True)

    def __str__(self):
        return f'Galpon {self.numero}'


class Lote(models.Model):
    galpon         = models.ForeignKey(Galpon, on_delete=models.PROTECT,
                                       related_name='lotes')
    raza           = models.ForeignKey(Raza, on_delete=models.PROTECT)
    nro_lote       = models.CharField(max_length=20)
    fecha_inicio   = models.DateField()
    fecha_fin      = models.DateField(null=True, blank=True)
    aves_iniciales = models.PositiveIntegerField()
    activo         = models.BooleanField(default=True)
    observaciones  = models.TextField(blank=True)

    class Meta:
        unique_together = ('galpon', 'nro_lote')

    def existencia_actual(self) -> int:
        ultimo = self.registros.order_by('-fecha').first()
        return ultimo.existencia if ultimo else self.aves_iniciales

    def semana_edad_actual(self) -> int:
        dias = (datetime.date.today() - self.fecha_inicio).days
        return max(1, (dias // 7) + 1)

    def viabilidad_acumulada(self) -> Decimal:
        return Decimal(
            self.existencia_actual() / self.aves_iniciales * 100
        ).quantize(Decimal('0.01'))


class RegistroProduccion(models.Model):
    """Registro diario de produccion y mortalidad."""
    lote           = models.ForeignKey(Lote, on_delete=models.PROTECT,
                                       related_name='registros')
    fecha          = models.DateField()
    semana_edad    = models.PositiveIntegerField()
    huevos_totales = models.PositiveIntegerField(default=0)
    cajones        = models.PositiveIntegerField(default=0)
    maples         = models.PositiveIntegerField(default=0)
    aves_muertas   = models.PositiveIntegerField(default=0)
    existencia     = models.PositiveIntegerField()
    observaciones  = models.TextField(blank=True)
    usuario_carga  = models.CharField(max_length=100, blank=True)

    class Meta:
```

```
        unique_together = ('lote', 'fecha')
        ordering = ['-fecha']

    @property
    def porc_postura(self) -> Decimal:
        if self.existencia == 0:
            return Decimal('0.00')
        return Decimal(
            self.huevos_totales / self.existencia * 100
        ).quantize(Decimal('0.01'))

    def desvio_vs_tabla(self) -> Decimal:
        try:
            tabla = TablaGenetica.objects.get(
                raza=self.lote.raza,
                semana_edad=self.semana_edad
            )
            return self.porc_postura - tabla.prod_tabla
        except TablaGenetica.DoesNotExist:
            return Decimal('0.00')

    def tiene_alerta_produccion(self) -> bool:
        """True si la produccion cae mas de 5pp bajo la tabla."""
        return self.desvio_vs_tabla() < Decimal('-5.00')
```

Codigo 9.1 — models/produccion.py: modelos Django con logica de negocio

7.1.5.2 API REST — Vistas y Endpoints

```
# views/produccion.py
# ============================================================
# API REST — Endpoints del Modulo de Produccion
# ============================================================

from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from django.db.models import Sum, Avg
from .models import RegistroProduccion, Lote, TablaGenetica
from .serializers import RegistroProduccionSerializer
from .services import ProduccionService


class RegistroProduccionViewSet(viewsets.ModelViewSet):
    """CRUD para registros de produccion diaria.
    Base: /api/v1/produccion/registros/"""

    serializer_class   = RegistroProduccionSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        qs = RegistroProduccion.objects.select_related(
            'lote', 'lote__galpon', 'lote__raza')
        lote_id = self.request.query_params.get('lote')
```

```
        desde   = self.request.query_params.get('desde')
        hasta   = self.request.query_params.get('hasta')
        if lote_id: qs = qs.filter(lote_id=lote_id)
        if desde:   qs = qs.filter(fecha__gte=desde)
        if hasta:   qs = qs.filter(fecha__lte=hasta)
        return qs.order_by('-fecha')

    def perform_create(self, serializer):
        registro = serializer.save(
            usuario_carga=self.request.user.username)
        # Verificacion de alertas (tarea asincrona via Celery)
        ProduccionService.verificar_alertas(registro)

    @action(detail=False, methods=['get'],
            url_path='resumen-semanal')
    def resumen_semanal(self, request):
        """GET /api/v1/produccion/registros/resumen-semanal/?lote=N"""
        lote_id = request.query_params.get('lote')
        if not lote_id:
            return Response({'error': 'Parametro lote requerido'},
                            status=status.HTTP_400_BAD_REQUEST)
        data = RegistroProduccion.objects.filter(
            lote_id=lote_id
        ).values('semana_edad').annotate(
            huevos_semana  = Sum('huevos_totales'),
            cajones_semana = Sum('cajones'),
            muertas_semana = Sum('aves_muertas'),
            prom_postura   = Avg('porc_postura'),
        ).order_by('semana_edad')
        return Response(list(data))

    @action(detail=False, methods=['get'],
            url_path='curva-vs-tabla')
    def curva_vs_tabla(self, request):
        """Datos para graficar produccion real vs curva del fabricante.
        GET /api/v1/produccion/registros/curva-vs-tabla/?lote=N"""
        lote_id = request.query_params.get('lote')
        lote = Lote.objects.get(id=lote_id)
        reales = list(RegistroProduccion.objects.filter(
            lote=lote).values('semana_edad').annotate(
                prod_real=Avg('porc_postura')
            ).order_by('semana_edad'))
        tabla = list(TablaGenetica.objects.filter(
            raza=lote.raza
        ).values('semana_edad','prod_tabla','viab_tabla')
         .order_by('semana_edad'))
        return Response({'reales':reales, 'tabla':tabla,
                         'raza':lote.raza.nombre})
```

Codigo 9.2 — views/produccion.py: endpoints REST con resumen semanal y curva vs tabla

7.1.5.3 Tabla de Alertas Productivas

Condicion	Nivel	Notificacion	Accion
% Postura < Tabla - 5pp	Advertencia	Push + email supervisor	Revisar alimentacion y sanidad

Condicion	Nivel	Notificacion	Accion
% Postura < Tabla - 10pp	Critico	Push + WhatsApp	Inspeccion inmediata del lote
Mortalidad semanal > 0.5%	Advertencia	Push supervisor	Revisar causa de muerte
Mortalidad semanal > 1.0%	Critico	Push + WhatsApp	Llamar al veterinario
Viabilidad acum. < tabla - 5pp	Informativo	Email semanal	Analisis de tendencia

7.1.5.4 Dashboard de Producción — Frontend React

```
// components/ProduccionDashboard.jsx
// ============================================================
// Dashboard de produccion — React + Recharts
// ============================================================

import { useState, useEffect } from 'react';
import {
  LineChart, Line, XAxis, YAxis, CartesianGrid,
  Tooltip, Legend, ResponsiveContainer
} from 'recharts';
import { fetchCurvaVsTabla, fetchResumenSemanal } from '../api/produccion';


export default function ProduccionDashboard({ loteId }) {
  const [curvaData,   setCurvaData]   = useState([]);
  const [resumen,     setResumen]     = useState([]);
  const [raza,        setRaza]        = useState('');
  const [loading,     setLoading]     = useState(true);

  useEffect(() => {
    if (!loteId) return;
    Promise.all([fetchCurvaVsTabla(loteId), fetchResumenSemanal(loteId)])
      .then(([curva, res]) => {
        const merged = curva.tabla.map(row => {
          const real = curva.reales.find(r => r.semana_edad ===
row.semana_edad);
          return {
            semana: row.semana_edad,
            tabla:  parseFloat(row.prod_tabla),
            viab:   parseFloat(row.viab_tabla),
            real:   real ? parseFloat(real.prod_real) : null,
          };
        });
        setCurvaData(merged);
        setResumen(res);
        setRaza(curva.raza);
      })
      .finally(() => setLoading(false));
  }, [loteId]);

  if (loading) return <div>Cargando...</div>;

  return (
```

```
    <div className='dashboard'>
      <h2>Curva de Produccion — {raza}</h2>

      <ResponsiveContainer width='100%' height={350}>
        <LineChart data={curvaData}
                   margin={{top:10,right:30,left:0,bottom:0}}>
          <CartesianGrid strokeDasharray='3 3' />
          <XAxis dataKey='semana'
                 label={{value:'Semana de Edad',position:'insideBottom'}} />
          <YAxis domain={[0,100]}
                 label={{value:'%',angle:-90,position:'insideLeft'}} />
          <Tooltip formatter={v => `${v?.toFixed(2)}%`} />
          <Legend />
          <Line type='monotone' dataKey='tabla'
                stroke='#e74c3c' strokeDasharray='5 5'
                name='Tabla Fabricante' dot={false} />
          <Line type='monotone' dataKey='real'
                stroke='#27ae60' strokeWidth={2}
                name='Produccion Real' connectNulls />
          <Line type='monotone' dataKey='viab'
                stroke='#2980b9' strokeDasharray='3 3'
                name='Viabilidad Tabla' dot={false} />
        </LineChart>
      </ResponsiveContainer>

      <table className='tabla-resumen'>
        <thead>
          <tr><th>Sem.</th><th>Huevos</th><th>Cajones</th>
              <th>Muertas</th><th>% Postura</th><th>Estado</th></tr>
        </thead>
        <tbody>
          {resumen.map(r => (
            <tr key={r.semana_edad}
                className={r.prom_postura < 75 ? 'alerta' : ''}>
              <td>{r.semana_edad}</td>
              <td>{r.huevos_semana?.toLocaleString()}</td>
              <td>{r.cajones_semana}</td>
              <td>{r.muertas_semana}</td>
              <td>{r.prom_postura?.toFixed(2)}%</td>
              <td>{r.prom_postura >= 80 ? 'OK' :
                   r.prom_postura >= 75 ? 'Atencion' : 'Alerta'}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

Codigo 9.3 — ProduccionDashboard.jsx: curva real vs tabla genetica con Recharts

7.2 Módulo 2 — Monitoreo IoT y alertas

7.2.1 CU03 — Consultar estado actual de sensores IoT

Estado: Demo con datos simulados
Demo: panel IoT con estado por sensor. Los valores son simulados en el navegador.

frontend/index.html — función renderSensors

```
function renderSensors(){
  const lbl={ok:'Normal',warn:'Atención',crit:'CRÍTICO'};
  const bc={ok:'bgn',warn:'bam',crit:'brd'};
  const ic={ok:'ti-check',warn:'ti-alert-triangle',crit:'ti-alert-octagon'};
  $('sGrid').innerHTML=SENSORS.map(s=>{
    const n=sNivel(s);
    return `<div class="s-card ${n}">
      <div class="s-dot"></div>
      <div class="s-name">${s.name}</div>
      <div class="s-val">${s.val.toFixed(s.id==='co2'?0:1)}<span
class="s-unit">${s.unit}</span></div>
      <div style="margin-top:8px"><span class="bdg ${bc[n]}"><i class="ti ${ic[n]}" style="font-
size:10px"></i>${lbl[n]}</span></div>
      <i class="ti ${s.ico} s-icon"></i>
    </div>`;
  }).join('');
  tick();
}
```

frontend/index.html — función sNivel

```
function sNivel(s){
  if(s.val<s.min*.9||s.val>s.max*1.1)return'crit';
  if(s.val<s.min||s.val>s.max)return'warn';
  return'ok';
}
```

7.2.2 CU05 — Configurar umbrales de alerta por sensor

Estado: Umbrales implementados; pantalla diseñada
Los umbrales vigentes están definidos en backend/iot/umbrales.py (valores de la sección 7.2.7.4). La pantalla de configuración no está construida.

7.2.3 CU06 — Recibir alerta Warning

Estado: Evaluación implementada y probada; notificación diseñada
backend/iot/umbrales.py (evaluar) y backend/iot/ingest.py: clasifican cada lectura como normal, Warning o Crítico y guardan la alerta. El envío de push/WhatsApp está diseñado.

backend/iot/umbrales.py

```
"""Umbrales de alerta por variable (Tesis, cap. 10.4)."""

# tipo: (min_ok, max_ok, crit_bajo, crit_alto)
UMBRALES = {
    'temperatura':  (15, 30, 13.5, 33),
    'humedad':      (45, 75, 40, 82),
    'calidad_aire': (None, 3000, None, 3300),
    'temp_agua':    (10, 30, 9, 33),
    'flujo_agua':   (5, 200, None, None),   # el cero sostenido >2 h se evalúa aparte
}

RANGOS_FISICOS = {   # fuera de estos valores la lectura se descarta como falla del sensor
    'temperatura': (-20, 80), 'humedad': (0, 100), 'calidad_aire': (0, 10000),
    'temp_agua': (-5, 80), 'flujo_agua': (0, 5000),
}


def evaluar(tipo: str, valor: float) -> str:
    """Devuelve 'ok', 'warning' o 'critico'."""
    mn, mx, cb, ca = UMBRALES[tipo]
    if ca is not None and valor > ca:
        return 'critico'
    if cb is not None and valor < cb:
```

```
        return 'critico'
    if mx is not None and valor > mx:
        return 'warning'
    if mn is not None and valor < mn:
        return 'warning'
    return 'ok'
```

7.2.4 CU07 — Recibir alerta Crítico

Estado: Evaluación implementada y probada; notificación diseñada

Misma lógica que CU06 para el nivel Crítico (backend/iot).

7.2.5 CU08 — Consultar dashboard IoT en tiempo real

Estado: Demo con datos simulados

Demo: dashboard IoT con semáforo y gráficas (datos simulados). El canal WebSocket de tiempo real está diseñado.

frontend/index.html — función simSensors

```
function simSensors(){
  SENSORS.forEach(s=>{s.val=Math.max(s.min*.75,Math.min(s.max*1.1,s.val+(Math.random()-.5)*.6));});
  renderSensors();
}
```

7.2.6 CU19-22 — Flujo automático IoT (CU19 a CU22)

Estado: Implementado y probado con simulador

backend/iot: ingest.py (suscripción MQTT, validación, persistencia, alertas), schema.sql y simulador.py; 29 pruebas automáticas aprobadas (cap. 8). Con sensores reales: pendiente.

backend/iot/schema.sql

```
-- Esquema IoT — TILCOR S.A.S. (PostgreSQL 14+; compatible con TimescaleDB)
CREATE TABLE IF NOT EXISTS sensores (
  id          INTEGER PRIMARY KEY,
  galpon      INTEGER NOT NULL,
  tipo        TEXT    NOT NULL,           -- temperatura | humedad | calidad_aire | flujo_agua |
temp_agua
  numero      INTEGER NOT NULL,
  unidad      TEXT    NOT NULL,
  UNIQUE (galpon, tipo, numero)
);

CREATE TABLE IF NOT EXISTS lecturas (
  ts          TIMESTAMPTZ      NOT NULL DEFAULT now(),
  sensor_id   INTEGER          NOT NULL REFERENCES sensores(id),
  valor       DOUBLE PRECISION NOT NULL,
  estado      TEXT             NOT NULL CHECK (estado IN ('ok','warning','critico'))
);
CREATE INDEX IF NOT EXISTS ix_lecturas_sensor_ts ON lecturas (sensor_id, ts DESC);
-- Con TimescaleDB instalado (producción):
-- SELECT create_hypertable('lecturas', 'ts', if_not_exists => TRUE);

CREATE TABLE IF NOT EXISTS alertas_iot (
  id          BIGSERIAL PRIMARY KEY,
  ts          TIMESTAMPTZ NOT NULL DEFAULT now(),
  sensor_id   INTEGER     NOT NULL REFERENCES sensores(id),
  severidad   TEXT        NOT NULL CHECK (severidad IN ('warning','critico')),
  valor       DOUBLE PRECISION NOT NULL,
  mensaje     TEXT        NOT NULL,
  revisada    BOOLEAN     NOT NULL DEFAULT FALSE
);

INSERT INTO sensores (id, galpon, tipo, numero, unidad) VALUES
  (1,1,'temperatura',1,'C'), (2,1,'temperatura',2,'C'),
  (3,1,'humedad',1,'%'),     (4,1,'humedad',2,'%'),
  (5,1,'calidad_aire',1,'ppm'), (6,1,'flujo_agua',1,'L'),
  (7,1,'temp_agua',1,'C')
ON CONFLICT DO NOTHING;
```

backend/iot/ingest.py

```
"""Ingesta MQTT -> PostgreSQL con motor de alertas.

Topic esperado:  gallinero/<galpon>/sensores/<tipo>/<n>
```

```
Payload JSON:    {"sensor_id": 1, "valor": 25.6, "unidad": "C"}

Variables de entorno: MQTT_HOST, MQTT_PORT, MQTT_USER, MQTT_PASS, MQTT_TLS=1, DATABASE_URL
"""
import json, logging, os, re
import paho.mqtt.client as mqtt
import psycopg
from umbrales import evaluar, RANGOS_FISICOS

log = logging.getLogger('tilcor.iot')
TOPIC_RE = re.compile(r'^gallinero/(\d+)/sensores/([a-z_]+)/(\d+)$')
TOPIC_SUB = 'gallinero/+/sensores/+/+'


class Ingesta:
    def __init__(self, dsn: str):
        self.conn = psycopg.connect(dsn, autocommit=True)
        self.estado_previo = {}          # sensor_id -> último estado (para no repetir alertas)
        self.descartadas = 0

    def procesar(self, topic: str, payload: bytes) -> str | None:
        """Valida, guarda y evalúa una lectura. Devuelve el estado o None si se descartó."""
        m = TOPIC_RE.match(topic)
        if not m:
            return self._descartar(f'topic inválido: {topic}')
        galpon, tipo, numero = int(m[1]), m[2], int(m[3])
        if tipo not in RANGOS_FISICOS:
            return self._descartar(f'tipo desconocido: {tipo}')
        try:
            d = json.loads(payload)
            valor = float(d['valor'])
        except (ValueError, KeyError, TypeError):
            return self._descartar(f'payload inválido en {topic}')
        lo, hi = RANGOS_FISICOS[tipo]
        if not (lo <= valor <= hi) or valor != valor:
            return self._descartar(f'valor fuera de rango físico ({valor}) en {topic}')

        row = self.conn.execute(
            'SELECT id FROM sensores WHERE galpon=%s AND tipo=%s AND numero=%s',
            (galpon, tipo, numero)).fetchone()
        if not row:
            return self._descartar(f'sensor no registrado: {topic}')
        sid = row[0]

        estado = evaluar(tipo, valor)
        self.conn.execute('INSERT INTO lecturas (sensor_id, valor, estado) VALUES (%s,%s,%s)',
                          (sid, valor, estado))
        previo = self.estado_previo.get(sid, 'ok')
        if estado != 'ok' and estado != previo:      # alerta solo al cambiar de estado
            msg = f'{tipo} {numero} (galpón {galpon}): {valor} — nivel {estado}'
            self.conn.execute(
                'INSERT INTO alertas_iot (sensor_id, severidad, valor, mensaje) VALUES (%s,%s,%s,
%s)',
                (sid, estado, valor, msg))
            log.warning('ALERTA %s', msg)
        self.estado_previo[sid] = estado
        return estado

    def _descartar(self, motivo):
        self.descartadas += 1
        log.error('Lectura descartada: %s', motivo)
        return None


def main():
    logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %(message)s')
    ing = Ingesta(os.environ['DATABASE_URL'])
    cli = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    if os.getenv('MQTT_USER'):
        cli.username_pw_set(os.environ['MQTT_USER'], os.getenv('MQTT_PASS', ''))
    if os.getenv('MQTT_TLS') == '1':
        cli.tls_set()
    cli.on_connect = lambda c, u, f, rc, p=None: (log.info('Conectado al broker (rc=%s)', rc),
c.subscribe(TOPIC_SUB, qos=1))
    cli.on_message = lambda c, u, msg: ing.procesar(msg.topic, msg.payload)
    cli.connect(os.getenv('MQTT_HOST', 'localhost'), int(os.getenv('MQTT_PORT', '1883')),
keepalive=60)
    cli.loop_forever(retry_first_connection=True)


if __name__ == '__main__':
    main()
```

backend/iot/simulador.py

```
"""Simulador de sensores del galpón: publica lecturas MQTT con el formato real.
Uso:  python simulador.py --host localhost --intervalo 5 [--anomalia humedad]
Sirve para probar todo el flujo sin hardware y como plan B en la defensa."""
import argparse, json, os, random, time
import paho.mqtt.client as mqtt

SENSORES = [  # (tipo, n, id, base, ruido, unidad)
    ('temperatura', 1, 1, 26.0, 0.6, 'C'), ('temperatura', 2, 2, 27.0, 0.6, 'C'),
    ('humedad', 1, 3, 62.0, 2.0, '%'), ('humedad', 2, 4, 64.0, 2.0, '%'),
    ('calidad_aire', 1, 5, 800.0, 60.0, 'ppm'), ('flujo_agua', 1, 6, 55.0, 4.0, 'L'),
    ('temp_agua', 1, 7, 24.0, 0.5, 'C'),
]
ANOMALIAS = {'humedad': 38.0, 'temperatura': 34.0, 'calidad_aire': 3400.0}


def lectura(s, anomalia=None):
    tipo, n, sid, base, ruido, u = s
    v = ANOMALIAS[tipo] if anomalia == tipo else base + random.uniform(-ruido, ruido)
    return f'gallinero/1/sensores/{tipo}/{n}', json.dumps({'sensor_id': sid, 'valor': round(v, 1),
'unidad': u})


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--host', default='localhost'); ap.add_argument('--port', type=int, default=1883)
    ap.add_argument('--intervalo', type=float, default=5); ap.add_argument('--anomalia',
choices=ANOMALIAS)
    ap.add_argument('--ciclos', type=int, default=0, help='0 = infinito')
    a = ap.parse_args()
    c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    if os.getenv('MQTT_USER'): c.username_pw_set(os.environ['MQTT_USER'], os.getenv('MQTT_PASS', ''))
    c.connect(a.host, a.port); c.loop_start()
    i = 0
    while a.ciclos == 0 or i < a.ciclos:
        for s in SENSORES:
            t, p = lectura(s, a.anomalia); c.publish(t, p, qos=1)
        i += 1; time.sleep(a.intervalo)
    c.loop_stop(); c.disconnect()


if __name__ == '__main__':
    main()
```

7.2.7 Diseño y código de referencia (Tesis)

7.2.7.1 Flujo de Datos — Sensores a Dashboard

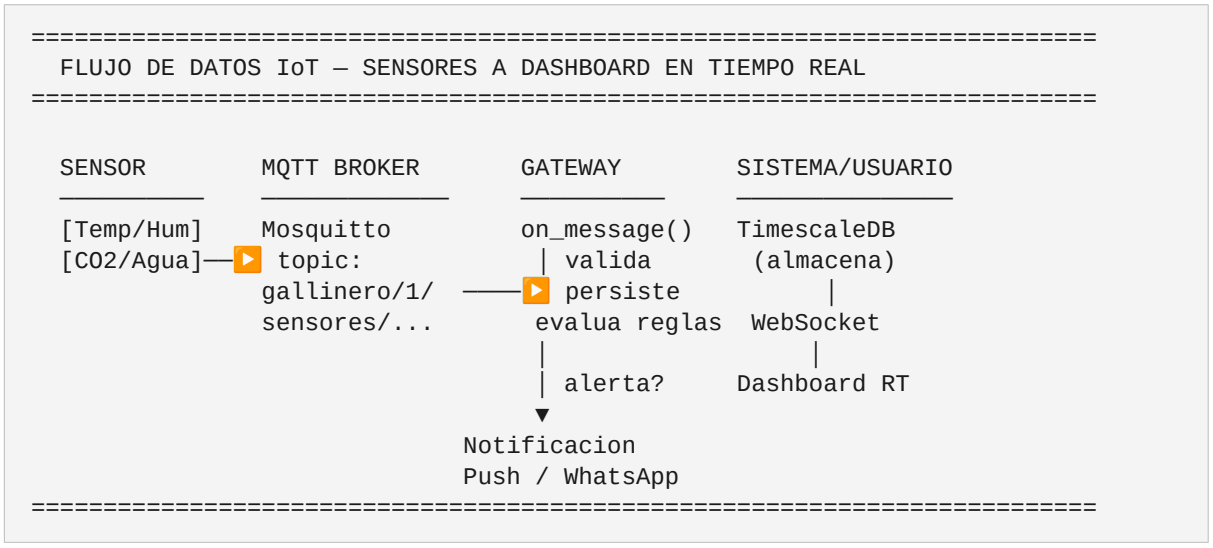


Diagrama 10.1 — Flujo MQTT -> TimescaleDB -> WebSocket -> Dashboard

7.2.7.2 Estructura de Topics MQTT

Topic MQTT	Sensor	Payload (JSON)
gallinero/1/sensores/ temperatura/1	Temp. Ambiente 1	{sensor_id:1, valor:25.6, unidad:C}
gallinero/1/sensores/ temperatura/2	Temp. Ambiente 2	{sensor_id:2, valor:27.1, unidad:C}
gallinero/1/sensores/ humedad/1	Humedad Relativa 1	{sensor_id:3, valor:68.4, unidad:%}
gallinero/1/sensores/ humedad/2	Humedad Relativa 2	{sensor_id:4, valor:71.2, unidad:%}
gallinero/1/sensores/ calidad_aire/1	CO2 / Calidad Aire	{sensor_id:5, valor:215.3, unidad:ppm}
gallinero/1/sensores/ flujo_agua/1	Flujo de Agua	{sensor_id:6, valor:43.0, unidad:L}
gallinero/1/sensores/ temp_agua/1	Temp. Agua Bebida	{sensor_id:7, valor:25.6, unidad:C}

7.2.7.3 Gateway MQTT — Código Fuente

El gateway recibe mensajes MQTT, los persiste en TimescaleDB, evalua el motor de alertas y brodcastea datos al dashboard en tiempo real via WebSocket con Django Channels.

```
# iot/gateway.py
# ================================================================
# Gateway MQTT — Recepcion y procesamiento de sensores IoT
# ================================================================

import json, logging
import paho.mqtt.client as mqtt
from datetime import datetime, timezone
from channels.layers import get_channel_layer
from asgiref.sync import async_to_sync
from .models import RegistroSensor
from .alertas import MotorAlertas

logger = logging.getLogger(__name__)
TOPIC = 'gallinero/+/sensores/#'
# Topic format: gallinero/{id_galpon}/sensores/{tipo}/{id_sensor}


class GatewayMQTT:
    def __init__(self):
        self.client = mqtt.Client(client_id='avicola-gateway')
        self.client.on_connect    = self._on_connect
        self.client.on_message    = self._on_message
        self.motor    = MotorAlertas()
        self.channel  = get_channel_layer()

    def conectar(self, host, port=1883):
        self.client.connect(host, port, keepalive=60)
```

```
        self.client.loop_start()
        logger.info('Gateway MQTT iniciado')

    def _on_connect(self, client, userdata, flags, rc):
        if rc == 0:
            client.subscribe(TOPIC, qos=1)
            logger.info(f'Suscripto a: {TOPIC}')

    def _on_message(self, client, userdata, msg):
        try:
            payload   = json.loads(msg.payload.decode('utf-8'))
            sensor_id = payload['sensor_id']
            valor     = float(payload['valor'])
            unidad    = payload.get('unidad', '')
            ts        = datetime.now(timezone.utc)

            # 1. Persistir en TimescaleDB
            registro = RegistroSensor.objects.create(
                sensor_id=sensor_id, timestamp=ts,
                valor=valor, unidad=unidad)

            # 2. Evaluar motor de alertas
            alerta = self.motor.evaluar(registro)

            # 3. Brodcastear via WebSocket al dashboard
            async_to_sync(self.channel.group_send)(
                f'gallinero_{sensor_id}',
                { 'type':'sensor.update', 'sensor_id':sensor_id,
                  'valor':valor, 'unidad':unidad,
                  'timestamp':ts.isoformat(), 'alerta':alerta }
            )
        except Exception as e:
            logger.error(f'Error MQTT message: {e}', exc_info=True)


class MotorAlertas:
    UMBRALES = {
        'temperatura':  {'min':15.0, 'max':30.0},
        'humedad':      {'min':45.0, 'max':75.0},
        'calidad_aire': {'min':None, 'max':3000.0},
        'flujo_agua':   {'min':5.0,  'max':None},
        'temp_agua':    {'min':10.0, 'max':30.0},
    }

    def evaluar(self, registro):
        sensor   = registro.sensor
        umbrales = self._umbrales(sensor)
        nivel    = self._nivel(registro.valor, umbrales)
        if nivel:
            return { 'nivel':nivel, 'sensor':sensor.codigo,
                     'valor':float(registro.valor),
                     'mensaje':f'[{nivel}] {sensor.codigo}: valor fuera de
rango' }
        return None

    def _umbrales(self, sensor):
        d = self.UMBRALES.get(sensor.tipo, {})
```

```
        return { 'min': sensor.umbral_min or d.get('min'),
                 'max': sensor.umbral_max or d.get('max') }

    def _nivel(self, valor, u):
        mn, mx = u['min'], u['max']
        if mn and valor < mn * 0.9: return 'critico'
        if mx and valor > mx * 1.1: return 'critico'
        if mn and valor < mn:       return 'advertencia'
        if mx and valor > mx:       return 'advertencia'
        return None
```

Codigo 10.1 — iot/gateway.py: Gateway MQTT con MotorAlertas integrado

7.2.7.4 Umbrales de Alerta por Variable

Variable	Unidad	Min OK	Max OK	Warning	Critico
Temperatura	C	15	30	< 15 o > 30	< 13.5 o > 33
Humedad relativa	%	45	75	< 45 o > 75	< 40 o > 82
Calidad del aire	ppm	0	3000	> 3000	> 3300
Flujo de agua	L/h	5	200	< 5 por 1h	= 0 por > 2h
Temperatura agua	C	10	30	< 10 o > 30	< 9 o > 33

7.2.7.5 Dashboard IoT — Frontend React

```
// components/IotDashboard.jsx
// ================================================================
// Dashboard IoT en tiempo real via WebSocket
// ================================================================

import { useState, useEffect, useRef } from 'react';

const WS_BASE = process.env.REACT_APP_WS_URL || 'ws://localhost:8000';

const UMBRALES = {
  temperatura:  { min:15, max:30, unidad:'C' },
  humedad:      { min:45, max:75, unidad:'%' },
  calidad_aire: { min:0,  max:3000, unidad:'ppm' },
  flujo_agua:   { min:5,  max:200, unidad:'L' },
};

function nivel(tipo, valor) {
  const u = UMBRALES[tipo];
  if (!u) return 'normal';
  if (valor < u.min * 0.9 || valor > u.max * 1.1) return 'critico';
  if (valor < u.min || valor > u.max) return 'advertencia';
  return 'normal';
}

function SensorCard({ s }) {
  const n = nivel(s.tipo, s.valor);
  const color = { normal:'#27ae60', advertencia:'#f39c12', critico:'#e74c3c' }
[n];
```

```
  return (
    <div className='sensor-card'
         style={{ borderLeft: `4px solid ${color}` }}>
      <div className='nombre'>{s.codigo}</div>
      <div className='valor' style={{ color, fontSize:'2rem', fontWeight:'bold'
}}>
        {s.valor?.toFixed(2)}
        <span className='unidad'>{UMBRALES[s.tipo]?.unidad}</span>
      </div>
      <div className='ts'>
        {new Date(s.timestamp).toLocaleTimeString()}
      </div>
      {n !== 'normal' && (
        <div className={`badge ${n}`}>
          {n === 'critico' ? 'CRITICO' : 'ATENCION'}
        </div>
      )}
    </div>
  );
}

export default function IotDashboard({ galponId }) {
  const [sensores, setSensores] = useState({});
  const wsRef = useRef(null);

  useEffect(() => {
    const ws = new WebSocket(`${WS_BASE}/ws/gallinero/${galponId}/`);
    wsRef.current = ws;
    ws.onmessage = e => {
      const data = JSON.parse(e.data);
      if (data.type === 'sensor.update') {
        setSensores(prev => ({ ...prev, [data.sensor_id]: data }));
      }
    };
    return () => ws.close();
  }, [galponId]);

  return (
    <div className='iot-dashboard'>
      <h2>Monitoreo en Tiempo Real — Galpon {galponId}</h2>
      <div className='sensores-grid'>
        {Object.values(sensores).map(s => (
          <SensorCard key={s.sensor_id} s={s} />
        ))}
      </div>
    </div>
  );
}
```

Codigo 10.2 — IotDashboard.jsx: semaforo en tiempo real con WebSocket

7.3 Módulo 3 — Sanidad y mortalidad

7.3.1 CU14 — Ver viabilidad acumulada de los lotes

Estado: Implementado en la demo (datos de ejemplo)
Demo: viabilidad acumulada en el inicio y en Sanidad.

7.3.2 CU16 — Recibir informe semanal de mortalidad

Estado: Diseñado
Informe semanal por Celery y correo: diseñado, sin construir.

7.3.3 CU17 — Consultar historial sanitario del lote

Estado: Implementado en la demo (datos de ejemplo)
Demo: sección Sanidad con KPIs y protocolo de alertas.

7.3.4 CU18 — Ver curva de mortalidad histórica

Estado: Implementado en la demo (datos de ejemplo)
Demo: curva de mortalidad semanal con línea de referencia.

7.4 Módulo 4 — Dashboard, reportes y plan de implementación

7.4.1 CU10 — Editar progreso del plan de implementación

Estado: Implementado en la demo
Demo: modal de edición del plan con sliders; el progreso se guarda en el navegador.

frontend/index.html — función guardarPlan

```
function guardarPlan(){
  const fases=cargarFases();
  fases.forEach(f=>{
    const slider=$('slider'+f.n);
    if(slider)f.pct=parseInt(slider.value);
  });
  guardarFasesStorage(fases);
  renderPlan();
  closeModal('modalPlan');
  toast('✅ Progreso actualizado');
}
```

7.4.2 CU12 — Ver KPIs generales en dashboard principal

Estado: Implementado en la demo (datos de ejemplo)
Demo: pantalla de inicio con KPIs, producción de 7 días, curva y viabilidad. El resumen comercial desde Don Gestión está diseñado.

7.4.3 CU13 — Recibir reporte diario automatizado

Estado: Diseñado
Reporte diario PDF por Celery y correo: diseñado, sin construir.

7.4.4 Diseño y código de referencia (Tesis)

7.4.4.1 Catálogo de Reportes

Reporte	Formato	Frecuencia	Destinatario
Produccion Diaria	WhatsApp/Email	Automatico al cierre	Propietario
Curva vs Tabla Genetica	Excel + Grafico	Semanal automatico	Supervisor
Mortalidad Semanal	PDF	Semanal automatico	Veterinario
Resumen Comercial	Excel	Mensual automatico	Admin
Estado Cuenta Corriente	PDF	A demanda	Vendedor
Historico Ambiental IoT	PDF	A demanda	Supervisor
Comparativo Brown vs LSL	Excel + Grafico	Por ciclo	Admin

7.4.4.2 KPIs del Dashboard Principal

KPI	Fuente	Periodo	Semaforo
% Postura promedio	registros_produccion	Ultima semana	Verde >80 / Amarillo 70-80 / Rojo <70
Aves en existencia	registros_produccion	Hoy	Informativo

KPI	Fuente	Periodo	Semaforo
Temperatura galpon	registros_sensor	Tiempo real	Verde <28 / Amarillo 28-30 / Rojo >30
Humedad galpon	registros_sensor	Tiempo real	Verde 45-75% / fuera=alerta
Ventas del mes	pedidos/facturas	Mes actual	vs. mes anterior
Clientes con deuda vencida	cuenta_corriente	Hoy	Rojo si hay vencimientos

7.5 Módulo 5 — Usuarios y roles

7.5.1 CU15 — Iniciar sesión con rol diferenciado

Estado: Demo implementada; JWT diseñado; auditoría SQL implementada

Demo: inicio de sesión con dos perfiles; el perfil Empleado no ve Ventas. Los usuarios están en el cliente. JWT en el backend: diseñado. Auditoría por triggers: implementada (backend/db/auditoria.sql).

frontend/index.html — función doLogin

```
function doLogin(){
  const user=$('loginUser').value.trim();
  const pass=$('loginPass').value;
  const errEl=$('loginError');
  const u=USUARIOS[user];
  if(u&&u.pass===pass){
    errEl.style.display='none';
    rolActual=u.rol;
    sessionStorage.setItem('tilcor_user',user);
    sessionStorage.setItem('tilcor_rol',u.rol);
    sessionStorage.setItem('tilcor_nombre',u.nombre);
    aplicarRol(u.rol,u.nombre);
    $('loginScreen').style.display='none';
    toast('Bienvenido, '+u.nombre+' 👋');
    // Animar barras de viabilidad
    setTimeout(()=>{$('viab1').style.width='90.2%';$('viab2').style.width='67.3%'},600);
  } else {
    errEl.style.display='flex';
    $('loginPass').value='';
    $('loginPass').focus();
    $('loginScreen').querySelector('.login-card').style.animation='shake .35s ease';
    setTimeout(()=>{$('loginScreen').querySelector('.login-card').style.animation=''},400);
  }
}
```

frontend/index.html — función aplicarRol

```
function aplicarRol(rol,nombre){
  const av=$('userAvatar');if(av)av.textContent=nombre[0];
  const mn=$('userMenuNombre');if(mn)mn.textContent=nombre;
  const mr=$('userMenuRol');if(mr)mr.textContent=rol==='admin'?'Administrador — Acceso
completo':'Empleado — Acceso estándar';
  const tv=$('tab-ventas'),sv=$('sec-ventas');
  if(rol==='empleado'){if(tv)tv.style.display='none';if(sv)sv.style.display='none';}
  else{if(tv)tv.style.display='';if(sv)sv.style.display='';}
}
```

backend/db/auditoria.sql

```
-- Auditoría por triggers (Tesis 13.3): quién, cuándo, tabla, operación y snapshot antes/después.
CREATE TABLE IF NOT EXISTS auditoria (
  id        BIGSERIAL PRIMARY KEY,
  ts        TIMESTAMPTZ NOT NULL DEFAULT now(),
  usuario   TEXT        NOT NULL DEFAULT coalesce(nullif(current_setting('app.usuario', true), ''),
current_user),
  tabla     TEXT        NOT NULL,
  operacion TEXT        NOT NULL CHECK (operacion IN ('INSERT','UPDATE','DELETE')),
  antes     JSONB,
  despues   JSONB
);
CREATE INDEX IF NOT EXISTS ix_auditoria_tabla_ts ON auditoria (tabla, ts DESC);

CREATE OR REPLACE FUNCTION fn_auditar() RETURNS trigger AS $$
BEGIN
  INSERT INTO auditoria (tabla, operacion, antes, despues)
  VALUES (TG_TABLE_NAME, TG_OP,
          CASE WHEN TG_OP IN ('UPDATE','DELETE') THEN to_jsonb(OLD) END,
          CASE WHEN TG_OP IN ('INSERT','UPDATE') THEN to_jsonb(NEW) END);
  RETURN COALESCE(NEW, OLD);
END; $$ LANGUAGE plpgsql;

-- Ejemplo de uso sobre una tabla (repetir por cada tabla auditada):
-- CREATE TRIGGER trg_aud_alertas AFTER INSERT OR UPDATE OR DELETE ON alertas_iot
--   FOR EACH ROW EXECUTE FUNCTION fn_auditar();
-- La aplicación fija el usuario por transacción:  SET LOCAL app.usuario = 'mcarnielli';
```

7.5.2 Diseño y código de referencia (Tesis)

7.5.2.1 Roles del Sistema

Rol	Modulos	Permisos especiales
Administrador	Todos	ABMC completo, configuracion, reportes globales
Supervisor Productivo	Produccion, IoT, Reportes	Editar produccion, configurar alertas, aprobar pedidos grandes
Operario de Galpon	Solo registro produccion	Alta de registros de su galpon asignado
Vendedor Comercial	Clientes, Pedidos, Facturacion	ABMC clientes, crear pedidos, ver saldos
Veterinario	Produccion, Mortalidad	Solo lectura, reportes de salud

7.5.2.2 Autenticación JWT

Se utilizan JSON Web Tokens (JWT) para autenticacion stateless. Al loguearse se emiten dos tokens: access token (15 minutos) y refresh token (7 dias). El frontend almacena los tokens en memoria (no en localStorage) y renueva el access token automaticamente antes de su vencimiento.

```
# authentication/serializers.py — Claims personalizados en JWT

from rest_framework_simplejwt.serializers import TokenObtainPairSerializer

class CustomTokenSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        # Claims embebidos en el token
        token['nombre']   = user.get_full_name()
        token['rol']      = user.groups.values_list('name', flat=True).first()
        token['galpones'] = list(
            user.galpones_asignados.values_list('id', flat=True)
        )
        return token

# Decorador de verificacion de rol en vistas
def requiere_rol(*roles):
    def decorator(view_func):
        def wrapper(request, *args, **kwargs):
            user_rol = request.auth.get('rol') if request.auth else None
            if user_rol not in roles:
                return Response({'error': 'Acceso no autorizado'}, status=403)
            return view_func(request, *args, **kwargs)
        return wrapper
    return decorator
```

Codigo 13.1 — JWT con claims personalizados de rol y galpones asignados

7.5.2.3 Auditoría

Todas las operaciones de escritura quedan registradas en una tabla de auditoria mediante triggers de PostgreSQL con: usuario, timestamp, tabla, operacion (INSERT/UPDATE/DELETE) y snapshot JSON del registro antes y despues del cambio. Esto garantiza trazabilidad incluso ante cambios directos en la base de datos.

7.6 Módulo 6 — Gestión comercial (Don Gestión)

7.6.1 CU11 — Aprobar pedido que supera límite de crédito

Estado: Diseñado
Aprobación de pedidos sobre el límite de crédito: diseñada, sin construir.

7.6.2 CU23 — Actualizar datos comerciales (Don Gestión)

Estado: Implementado en la demo
Demo: sección Ventas con iframe a Don Gestión, visible solo para el rol Administrador.

frontend/index.html — función setIframe

```
function setIframe(url,btnId){
  $('iframeLoader').style.display='flex';
  $('dgFrame').src=url;
  document.querySelectorAll('.dg-btn').forEach(b=>b.classList.remove('active-dg'));
  if(btnId){
    const btn=document.querySelector(`[onclick*="${btnId}"]`);
    if(btn)btn.classList.add('active-dg');
  }
}
```

7.6.3 CU24 — Emitir factura electrónica ARCA/AFIP

Estado: Diseñado
Facturación electrónica por Don Gestión y ARCA/AFIP (WSFE): diseñada. El código de referencia está en la sección 7.6.4.

7.6.4 Diseño y código de referencia (Tesis)

7.6.4.1 Funciónalidades

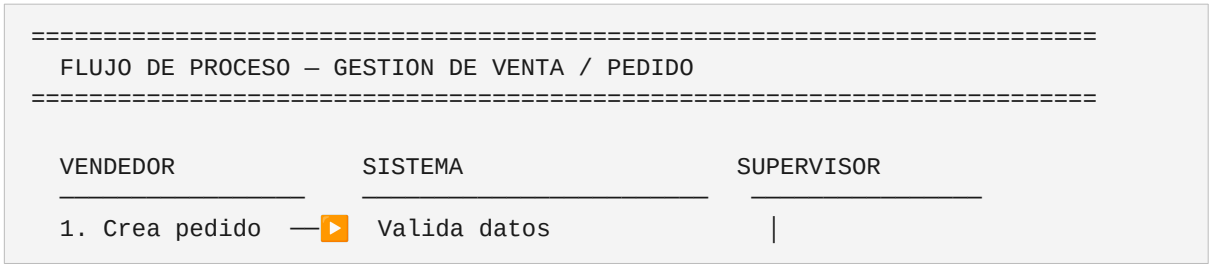
7.6.4.1.1 ABMC de Clientes

Alta, baja logica (flag activo=False preservando historial), modificacion y consulta de clientes. La validacion del CUIT usa el algoritmo oficial de digito verificador. La ficha del cliente muestra en tiempo real: saldo de cuenta corriente, credito disponible, pedidos del mes y semaforo de credito (verde/amarillo/rojo).

7.6.4.1.2 Gestión de Precios

Listas de precios diferenciadas por categoria (mayorista, minorista, distribuidor) y tipo de producto. Cada precio tiene fecha de vigencia; el sistema aplica automaticamente el precio vigente al momento del pedido.

7.6.4.1.3 Flujo de Venta



```
     (cliente,          Verifica credito       ———▶ Alerta si
      cantidad,         Calcula total                supera limite
      tipo huevo)

  2. Confirma      —▶  Genera remito PDF
                       Actualiza stock

  3. Entrega       —▶  Confirma recepcion
                       Actualiza estado

  4. Factura       —▶  Llama AFIP-WSFE
                       Obtiene CAE
                       Guarda factura
                       Registra en cta.cte.
                       Envia email cliente
=========================================================================
```

Diagrama 8.1 — Flujo de proceso de venta con estados y actores

7.6.4.2 API REST — Endpoints del Módulo de Clientes

Metodo	Endpoint	Descripcion	Rol
GET	/api/v1/clientes/	Listar con filtros	Vendedor, Admin
POST	/api/v1/clientes/	Crear cliente	Admin
GET	/api/v1/clientes/{id}/	Detalle	Vendedor, Admin
PATCH	/api/v1/clientes/{id}/	Modificar	Admin
DELETE	/api/v1/clientes/{id}/	Baja logica	Admin
GET	/api/v1/clientes/{id}/pedidos/	Historial pedidos	Vendedor, Admin
GET	/api/v1/clientes/{id}/saldo/	Saldo cta. cte.	Vendedor, Admin

7.6.4.3 Integración AFIP — Facturación Electronica

El modulo emite facturas electronicas tipo A, B y C via el servicio WSFE de AFIP. El tipo de comprobante se determina automaticamente segun la condicion impositiva del cliente: Responsable Inscripto recibe Factura A con IVA 21%, Exento/Monotributista recibe Factura C sin IVA, y Consumidor Final recibe Factura B.

```
# services/facturacion.py
# ================================================================
# Servicio de Facturacion Electronica — Integracion AFIP WSFE
# ================================================================

from decimal import Decimal
from dataclasses import dataclass
import zeep
from django.conf import settings
from .models import Pedido, Factura
```

```
@dataclass
class DatosFactura:
    tipo_comprobante: int   # 1=FactA, 6=FactB, 11=FactC
    punto_venta:      int
    id_cliente:       int
    importe_neto:     Decimal
    alicuota_iva:     Decimal
    concepto:         int = 1   # 1=Productos


class ServicioFacturacion:
    WSDL_PROD  = 'https://servicios1.afip.gov.ar/wsfev1/service.asmx?WSDL'
    WSDL_HOMOL = 'https://wswhomo.afip.gov.ar/wsfev1/service.asmx?WSDL'

    def __init__(self):
        wsdl = (self.WSDL_PROD if settings.AFIP_MODO == 'produccion'
                else self.WSDL_HOMOL)
        self.client = zeep.Client(wsdl)
        self.cuit   = settings.AFIP_CUIT
        self.token  = self._obtener_token()

    def emitir_factura(self, pedido: Pedido) -> Factura:
        datos  = self._preparar_datos(pedido)
        nro    = self._proximo_numero(datos)
        cae    = self._solicitar_cae(datos, nro)
        factura = Factura.objects.create(
            pedido         = pedido,
            id_cliente     = pedido.id_cliente,
            nro_factura    = f'{datos.punto_venta:05d}-{nro:08d}',
            tipo_comprobante = chr(64 + datos.tipo_comprobante),
            punto_venta    = datos.punto_venta,
            cae            = cae['cae'],
            cae_vto        = cae['vencimiento'],
            total_neto     = datos.importe_neto,
            alicuota_iva   = datos.alicuota_iva,
            total_iva      = datos.importe_neto * datos.alicuota_iva / 100,
            total_factura  = datos.importe_neto * (1 + datos.alicuota_iva/100),
        )
        self._registrar_cuenta_corriente(factura)
        return factura

    def _preparar_datos(self, pedido: Pedido) -> DatosFactura:
        cli = pedido.cliente
        if cli.condicion_iva == 'responsable_inscripto':
            tipo, alicuota = 1, Decimal('21.00')   # Factura A
        elif cli.condicion_iva in ('exento', 'monotributista'):
            tipo, alicuota = 11, Decimal('0.00')   # Factura C
        else:
            tipo, alicuota = 6, Decimal('21.00')   # Factura B
        return DatosFactura(
            tipo_comprobante=tipo,
            punto_venta=settings.AFIP_PUNTO_VENTA,
            id_cliente=cli.id,
            importe_neto=pedido.calcular_total(),
            alicuota_iva=alicuota,
        )
```

```
    def _solicitar_cae(self, datos: DatosFactura, nro: int) -> dict:
        from datetime import date
        hoy = date.today().strftime('%Y%m%d')
        res = self.client.service.FECAESolicitar(
            Auth={'Token':self.token['token'],
                  'Sign':self.token['sign'],
                  'Cuit':self.cuit},
            FeCAEReq={
                'FeCabReq':{'CantReg':1,'PtoVta':datos.punto_venta,
                            'CbteTipo':datos.tipo_comprobante},
                'FeDetReq':{'FECAEDetRequest':[{
                    'Concepto':datos.concepto, 'DocTipo':80,
                    'DocNro':datos.id_cliente,
                    'CbteDesde':nro, 'CbteHasta':nro, 'CbteFch':hoy,

'ImpTotal':float(datos.importe_neto*(1+datos.alicuota_iva/100)),
                    'ImpNeto':float(datos.importe_neto),
                    'ImpIVA':float(datos.importe_neto*datos.alicuota_iva/100),
                    'ImpTotConc':0,'ImpOpEx':0,'ImpTrib':0,
                    'MonId':'PES','MonCotiz':1,
                }]},
            }
        )
        det = res.FeDetResp.FECAEDetResponse[0]
        if det.Resultado != 'A':
            raise ValueError(f'AFIP rechazo la solicitud: {det.Observaciones}')
        return {'cae': det.CAE, 'vencimiento': det.CAEFchVto}
```

Codigo 11.1 — services/facturacion.py: emision de facturas via AFIP WSFE

7.6.4.4 Estados del Ciclo de Pedido

Estado	Descripcion	Transiciones
borrador	Pedido iniciado sin confirmar	-> confirmado, -> cancelado
confirmado	Aprobado, pendiente de entrega	-> en_entrega, -> cancelado
en_entrega	Salida del deposito	-> entregado
entregado	Confirmado por cliente	-> facturado
facturado	Factura con CAE emitida	Estado final
cancelado	Pedido anulado	Estado final

7.6.4.5 Cuenta Corriente

Cada factura genera un movimiento positivo (deuda) en la cuenta corriente del cliente. Los pagos se registran como movimientos negativos. El saldo se calcula como suma de todos los movimientos y se alerta al vendedor cuando supera el limite de credito configurado. La vista v_saldos_clientes provee saldo y credito disponible de todos los clientes activos en una consulta.

8 Integración y Testing

Este capítulo documenta cómo se verificó el sistema y qué resultados se obtuvieron. Retoma la estrategia de la pirámide de pruebas definida en el plan de pruebas de la sección 8.9 (unitarias, de integración y de extremo a extremo) y la aplica a lo que existe hoy construido y ejecutable: la aplicación web progresiva (PWA) de demostración y el módulo de ingesta IoT (MQTT → PostgreSQL). Los resultados que figuran como «Aprobada» fueron obtenidos ejecutando las pruebas; lo que todavía no se ejecutó figura como «Pendiente» y no se presenta como logrado.

8.1 Alcance y entorno de prueba

Elemento	Detalle
Componentes bajo prueba	PWA de demostración (frontend, HTML/JS) · Módulo IoT: esquema PostgreSQL, ingesta MQTT, motor de alertas, simulador de sensores · Trigger de auditoría
Fuera de esta etapa	Backend Django completo (clientes, facturación AFIP, reportes) y pruebas con hardware real en el galpón (ver 8.7)
Base de datos	PostgreSQL 16.15 (compatible con la versión 15 especificada)
Mensajería	Cliente paho-mqtt 2.1.0 · broker embebido amqtt 0.12.1 para las pruebas automáticas (en producción: Eclipse Mosquitto 2.x)
Herramientas	pytest 9.2 (pruebas del módulo IoT) · Chromium automatizado con Playwright, pantalla móvil 390 × 844 px (pruebas de interfaz)
Repositorio	github.com/Mcarnielli/Ingenieria-en-SFT-Tilcor — carpetas backend/iot, backend/db y frontend

8.2 Estrategia de integración

La integración se hizo de forma incremental, de abajo hacia arriba, de modo que cada capa se probó antes de conectarla con la siguiente:

1. Reglas de negocio aisladas: función de evaluación de umbrales por variable (sin base de datos ni red).
2. Reglas + base de datos: procesamiento de una lectura (validación, guardado, alerta) contra PostgreSQL real.
3. Reglas + base + mensajería: el simulador publica por MQTT, el broker entrega, la ingesta guarda y alerta (flujo de extremo a extremo).
4. Interfaz: la PWA se recorre como lo haría cada perfil de usuario.
5. Hardware real: el mismo flujo con un sensor físico en el galpón (pendiente de ejecución en TILCOR).

Para poder probar sin depender del hardware se construyó un simulador de sensores que publica en los mismos topics y con el mismo formato JSON que usarán los equipos reales (por ejemplo, gallinero/1/sensores/humedad/1 con {"sensor_id":3,"valor":62.4,"unidad":"%"}). El simulador puede forzar una anomalía para provocar una alerta.

8.3 Pruebas unitarias y de integración del módulo IoT

Se ejecutaron 29 pruebas automáticas; las 29 resultaron aprobadas (tiempo total ≈ 7 s).

ID	Caso de prueba	Tipo	Resultado esperado	Estado
IOT-01 a 17	Umbrales por variable (temperatura, humedad, calidad de aire, temperatura de agua, flujo): 17 valores en los límites ok / warning / crítico	Unitaria	Cada valor se clasifica en el nivel correcto según la tabla de umbrales (sección 7.2.7.4)	Aprobada
IOT-18	Lectura válida de temperatura (25,6 °C)	Integración (DB)	Se guarda 1 lectura en estado ok, sin alerta	Aprobada
IOT-19	Tres lecturas críticas seguidas de humedad (38, 37, 36 %), luego normalización y nueva anomalía	Integración (DB)	Una sola alerta por episodio; al normalizarse y volver a fallar se genera la segunda (2 en total)	Aprobada
IOT-20 a 27	Lecturas inválidas: JSON roto, valor no numérico, falta el campo valor, valor fuera de rango físico (500 °C; −5 % humedad), tipo de sensor desconocido, sensor no registrado, topic inválido	Integración (DB)	Se descartan, no se guardan y se registran en el log (8 casos)	Aprobada
IOT-28	Flujo completo: el simulador publica los 7 sensores del galpón 1 y luego una anomalía de humedad	Extremo a extremo (MQTT)	7 lecturas guardadas sin alertas; tras la anomalía, 1 alerta crítica	Aprobada
IOT-29	Latencia entre publicar una lectura y verla guardada	Rendimiento	< 2 s	Aprobada

Tabla 8.1 — Resultados de las pruebas automáticas del módulo IoT.

La latencia medida fue de 11 ms, pero se obtuvo con broker y base en la misma máquina, por lo que solo demuestra que el procesamiento propio es despreciable; la latencia real dependerá de la red del galpón y se medirá en la prueba con hardware (HW-02).

Además se ejecutaron los scripts reales por línea de comandos (ingest.py y simulador.py) contra un broker MQTT: 5 ciclos de 7 sensores dejaron 35 lecturas guardadas y 2 alertas críticas (humedad 1 y humedad 2, 38 %), como se esperaba.

8.4 Pruebas de la interfaz (demo funcional)

Se recorrieron los flujos principales de la PWA con un navegador automatizado, tomando capturas en cada paso (reproducibles con el script del repositorio). El conjunto completo de capturas está en el Manual de Usuario, que se entrega como documento aparte para el cliente.

ID	Caso de prueba	Resultado obtenido	Estado
UI-01	Ingreso con Admin / contraseña correcta	Muestra «Bienvenido, Administrador» y el panel de inicio con KPIs	Aprobada
UI-02	Ingreso con contraseña incorrecta	Mensaje «Usuario o contraseña incorrectos»; no se accede	Aprobada
UI-03	Registro diario: 2.940 huevos, 3.520 aves, 2 muertas	Calcula 83,52 % de postura (verde) y confirma «Registro guardado correctamente»; aparece en el historial	Aprobada
UI-04	Registro diario con 2.500 huevos (postura 71,02 %, más de 5 pp bajo tabla)	Postura en rojo y aviso «Alerta de producción» con Confirmar / Cancelar al supervisor	Aprobada (tras corregir DEF-01)
UI-05	Perfil Empleado (Usuario)	No muestra la pestaña ni la sección Ventas; resto de módulos disponible	Aprobada
UI-06	Panel IoT	Muestra los 6 sensores con estado (humedad 1 en «Atención» a 44,2 %) y gráficos de 6 h	Aprobada

ID	Caso de prueba	Resultado obtenido	Estado
UI-07	Sanidad	Muestra viabilidad por línea, alerta de mortalidad semanal y protocolo de alertas	Aprobada

Tabla 8.2 — Resultados de las pruebas de interfaz sobre la demo desplegada en Vercel.

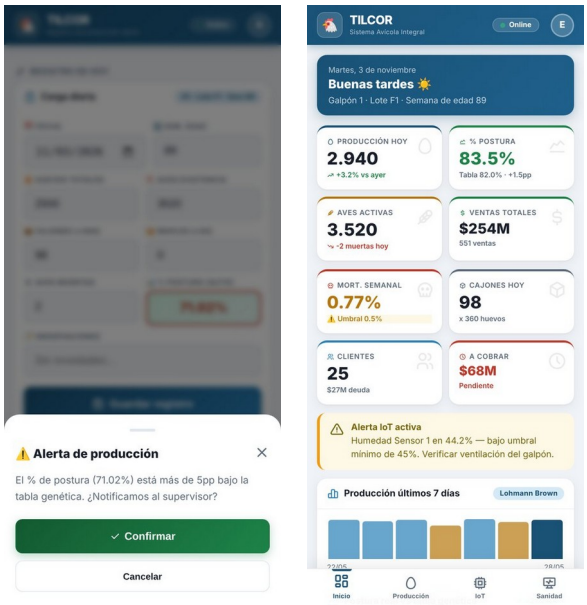


Figura 8.1 — UI-04: aviso de alerta de producción (izq.) · UI-05: perfil Empleado sin acceso a Ventas (der.).

8.5 Defectos detectados y corregidos

ID	Descripción	Severidad	Corrección	Estado
DEF-01	La alerta de postura baja nunca se mostraba: una función openModal declarada dos veces provocaba recursión infinita («Maximum call stack size exceeded») y ningún cuadro de diálogo podía abrirse	Alta	Se unificó en una sola función. Se repitió UI-04 con resultado correcto (commit 21e1980)	Corregido
DEF-02	El Manual de Usuario indicaba una dirección web inexistente	Baja	Reemplazada por https://tilcorvercel.vercel.app	Corregido
DEF-03	El índice del informe tenía números de página que no coincidían con el documento	Baja	Índice corregido	Corregido

Tabla 8.3 — Registro de defectos.

DEF-01 es un ejemplo del valor de probar el flujo completo: la lógica de la alerta era correcta, pero un error de integración entre dos funciones la volvía inalcanzable, algo que una lectura del código no mostraba a simple vista.

8.6 Pruebas de aceptación con el cliente

La aceptación se hará por fase, tal como establece el Contrato de Alcance: al cierre de cada fase se realiza una reunión de revisión con TILCOR y se verifican los criterios de aceptación del módulo contra el sistema funcionando. El pago del 70 % del precio de la fase queda sujeto a esa aceptación.

8.7 Pruebas con hardware real en TILCOR

Pendientes de ejecución durante la visita a la empresa. La tabla se completa con los resultados obtenidos en el lugar.

ID	Caso de prueba	Criterio de aprobación	Resultado	Estado
HW-01	El nodo (ESP32 + sensor de temperatura y humedad) se conecta al Wi-Fi del galpón y publica en MQTT	Las lecturas llegan al broker y se guardan en la base		Pendiente
HW-02	Latencia del dato real hasta la base, con la red del galpón	< 2 s		Pendiente
HW-03	Comparación con un termómetro de referencia, 15 a 30 min	Diferencia ≤ 1 °C		Pendiente
HW-04	Corte y vuelta del Wi-Fi	El nodo se reconecta solo y reanuda la publicación		Pendiente
HW-05	Valor anómalo forzado (por ejemplo, calentar o humedecer el sensor)	Se genera la alerta correspondiente		Pendiente

Tabla 8.4 — Pruebas con hardware real (a completar).

8.8 Limitaciones y trabajo futuro

- No se midió el porcentaje de cobertura de código; el objetivo definido en el plan de pruebas (80 % en producción y clientes) corresponde a la etapa del backend Django.
- No se realizaron pruebas de carga ni de seguridad (penetración); se prevén en la Fase 5.
- Las pruebas automáticas usan un broker embebido en lugar de Mosquitto; el protocolo es el mismo (MQTT 3.1.1), pero la prueba con Mosquitto y TLS se hará en la puesta en marcha.
- La demo guarda los datos en memoria del navegador y no tiene backend; los datos persistentes y el login con JWT se implementan en el backend.

8.9 Plan de pruebas del sistema completo (diseño)

8.9.1 Estrategia

Se implementa una piramide de pruebas completa: pruebas unitarias para la logica de negocio (modelos, servicios), pruebas de integracion para los endpoints REST, y pruebas end-to-end para los flujos criticos. El objetivo minimo es 80% de cobertura en los modulos de produccion y clientes.

8.9.2 Pruebas Unitarias e Integración

```
# tests/test_produccion.py
# ================================================================
# Pruebas unitarias del modulo de produccion
# ================================================================

from django.test import TestCase
from decimal import Decimal
from produccion.models import Raza, Galpon, Lote, TablaGenetica
from produccion.models import RegistroProduccion
from iot.alertas import MotorAlertas
from unittest.mock import MagicMock


class TestPorcPostura(TestCase):

    def setUp(self):
        self.raza   = Raza.objects.create(nombre='Lohmann Brown-Classic')
        self.galpon = Galpon.objects.create(numero=1, capacidad_max=5000)
        self.lote   = Lote.objects.create(
            galpon=self.galpon, raza=self.raza,
            nro_lote='F1', fecha_inicio='2022-06-01',
            aves_iniciales=4000
        )
        TablaGenetica.objects.create(
            raza=self.raza, semana_edad=88,
            prod_tabla=Decimal('82.00'),
            viab_tabla=Decimal('93.00')
        )

    def _reg(self, huevos, existencia, semana=88):
        return RegistroProduccion(
            lote=self.lote, fecha='2024-05-10',
            semana_edad=semana,
            huevos_totales=huevos,
            existencia=existencia,
            aves_muertas=0
        )

    def test_calculo_correcto(self):
        reg = self._reg(3300, 4000)
        self.assertEqual(reg.porc_postura, Decimal('82.50'))
```

```
    def test_division_por_cero(self):
        reg = self._reg(0, 0)
        self.assertEqual(reg.porc_postura, Decimal('0.00'))

    def test_alerta_si_cae_mas_5pp_bajo_tabla(self):
        # 3040/4000*100 = 76% < 82%-5 = 77%
        reg = self._reg(3040, 4000)
        self.assertTrue(reg.tiene_alerta_produccion())

    def test_sin_alerta_dentro_del_rango(self):
        # 3120/4000*100 = 78% > 77%, sin alerta
        reg = self._reg(3120, 4000)
        self.assertFalse(reg.tiene_alerta_produccion())


class TestMotorAlertas(TestCase):

    def _sensor(self, tipo):
        s = MagicMock()
        s.tipo = tipo
        s.umbral_min = None
        s.umbral_max = None
        s.codigo = f'{tipo.upper()}-01'
        s.galpon.numero = 1
        return s

    def _reg(self, tipo, valor):
        r = MagicMock(sensor=self._sensor(tipo), valor=valor)
        return r

    def test_temp_alta_genera_advertencia(self):
        motor = MotorAlertas()
        alerta = motor.evaluar(self._reg('temperatura', 31.5))
        self.assertIsNotNone(alerta)
        self.assertEqual(alerta['nivel'], 'advertencia')

    def test_temp_muy_alta_genera_critico(self):
        motor = MotorAlertas()
        alerta = motor.evaluar(self._reg('temperatura', 34.0))
        self.assertEqual(alerta['nivel'], 'critico')

    def test_temp_normal_sin_alerta(self):
        motor = MotorAlertas()
        alerta = motor.evaluar(self._reg('temperatura', 24.0))
        self.assertIsNone(alerta)


class TestAPIEndpoints(TestCase):
    """Pruebas de integracion sobre la API REST"""

    def setUp(self):
        from django.contrib.auth.models import User
        from rest_framework.test import APIClient
        from rest_framework_simplejwt.tokens import RefreshToken
        self.client = APIClient()
```

```
        user = User.objects.create_user('operario', password='pass1234')
        token = RefreshToken.for_user(user)
        self.client.credentials(
            HTTP_AUTHORIZATION=f'Bearer {str(token.access_token)}'
        )

    def test_sin_token_retorna_401(self):
        from rest_framework.test import APIClient
        c = APIClient()
        res = c.get('/api/v1/produccion/registros/')
        self.assertEqual(res.status_code, 401)

    def test_crear_registro_retorna_201(self):
        # requiere fixtures previos de lote
        pass   # implementar con factory_boy
```

Codigo 14.1 — tests/test_produccion.py: pruebas unitarias de modelos y motor de alertas

8.9.3 Cobertura de Pruebas

Modulo	Unitarias	Integracion	Cobertura objetivo
Produccion (modelos)	12 casos	8 endpoints	90%
Clientes (ABMC)	8 casos	10 endpoints	85%
IoT / Motor Alertas	10 casos	4 endpoints	88%
Facturacion AFIP	6 casos	5 endpoints	80%
Reportes	4 casos	3 endpoints	75%
Autenticacion/Roles	5 casos	6 endpoints	92%

9 Especificaciones del Sistema

Este capítulo define dónde y cómo se instala el sistema, con qué recursos de hardware y software, y con qué medidas de seguridad y auditoría. Distingue lo que ya está en funcionamiento (la demo en Vercel y el módulo IoT probado) de lo que corresponde a la puesta en producción.

9.1 Despliegue

9.1.1 Arquitectura de despliegue

Nodo	Componentes	Estado actual
Dispositivos del usuario	PWA instalable en celular, tablet o PC (navegador Chrome/Edge). Modo sin conexión con Service Worker	En funcionamiento (demo)
Hosting del frontend	Vercel (CDN con HTTPS). Despliegue automático desde la rama principal del repositorio	En funcionamiento: tilcorvercel.vercel.app
Servidor de aplicación (VPS Linux)	Nginx (proxy inverso + TLS) → Django/Gunicorn (API REST) · Django Channels (WebSocket) · Celery + Redis (tareas y reportes) · PostgreSQL con TimescaleDB · Mosquitto (broker MQTT)	Módulo IoT y base probados en local; el resto, a instalar en Fases 2 a 5
Nodos de campo (galpón)	ESP32 con sensores de temperatura, humedad, calidad de aire y flujo de agua; publican por MQTT sobre el Wi-Fi del galpón	Simulados; prueba real pendiente (cap. 8.7)
Servicios externos	Don Gestión (iframe/REST) y AFIP-WSFE (SOAP)	Don Gestión integrado por iframe; AFIP en Fase 4

Tabla 9.1 — Nodos del despliegue.

9.1.2 Contenedores y puertos

En el servidor, cada servicio corre en un contenedor Docker orquestado con Docker Compose, lo que permite reproducir la instalación y actualizarla de forma controlada. El archivo docker-compose.yml completo y las variables de entorno están en la sección 9.5; la única diferencia en producción es que el broker Mosquitto no se publica en el puerto 1883 sin cifrado, sino únicamente en el 8883 con TLS.

Servicio	Puerto	Exposición	Función
Nginx	443 (80 redirige)	Pública (HTTPS)	Entrada única, certificado TLS
Mosquitto	8883	Pública (MQTT sobre TLS, con usuario y clave)	Recibe lecturas de sensores
Django / Gunicorn	8000	Interna	API REST y WebSocket
PostgreSQL + TimescaleDB	5432	Interna (nunca expuesta)	Datos del sistema y series de tiempo
Redis	6379	Interna	Cola de Celery y canales WebSocket

Tabla 9.2 — Servicios, puertos y exposición.

9.1.3 Procedimiento de despliegue

1. Contratar el VPS (Ubuntu Server 24.04 LTS) y apuntar un dominio al servidor.

2. Instalar Docker y Docker Compose; clonar el repositorio.
3. Cargar las variables de entorno (claves, base de datos, credenciales MQTT y de AFIP) en un archivo .env fuera del repositorio.
4. Aplicar el esquema de base de datos (schema.sql, auditoria.sql y migraciones del backend) y activar TimescaleDB sobre la tabla de lecturas.
5. Emitir el certificado TLS (Let's Encrypt) y levantar los servicios con Docker Compose.
6. Crear los usuarios y roles iniciales y registrar los sensores (tabla sensores).
7. Configurar cada ESP32 con la dirección del broker y sus credenciales, y verificar que las lecturas lleguen.
8. Programar los respaldos (9.4) y ejecutar la prueba de aceptación de la fase.

El frontend se publica en Vercel con cada cambio en el repositorio, sin intervención manual. El procedimiento de despliegue en el VPS es el previsto y se ejecutará en la puesta en marcha; no fue ejecutado en esta instancia.

9.2 Hardware mínimo y óptimo

9.2.1 Servidor

Recurso	Mínimo	Óptimo
Procesador	2 vCPU	4 vCPU
Memoria RAM	4 GB	8 GB
Almacenamiento	40 GB SSD	80 GB SSD (NVMe)
Red	100 Mbps, IP pública fija	1 Gbps, IP pública fija
Respaldo	Copia diaria a almacenamiento externo	Copia diaria + copia semanal en otra región

Tabla 9.3 — Dimensionamiento del servidor (estimación; se ajusta con datos de uso real).

9.2.2 Dimensionamiento de datos

Con 7 sensores por galpón y una lectura cada 5 segundos se generan 120.960 lecturas por día. En la prueba de carga local, cada lectura ocupó 93 bytes incluyendo su índice (120.960 filas = 11 MB), de modo que el crecimiento estimado es de unos 11 MB por día, aproximadamente 4 GB por año por galpón sin compresión. La compresión nativa de TimescaleDB reduce ese volumen de forma significativa, por lo que los 40 GB del mínimo alcanzan para varios años del galpón actual. Si se agregaran galpones o se acortara el intervalo de lectura, el almacenamiento escala en proporción.

9.2.3 Dispositivos de campo y usuario

Elemento	Especificación
Nodo de sensores	Microcontrolador ESP32 · sensor de temperatura y humedad SHT31 o DHT22 · sensor de CO_2 MH-Z19 (calidad de aire) · caudalímetro de agua YF-S201 · fuente 5 V / 2 A · caja con protección contra polvo y humedad (IP65)
Red del galpón	Wi-Fi 2,4 GHz con cobertura en el punto de instalación; ancho de banda mínimo 1 Mbps de subida (cada lectura pesa menos de 100 bytes)
Celular / tablet	Android 8 o superior, o iOS 14 o superior; 2 GB de RAM; navegador Chrome o Safari actualizado
Computadora	Cualquier equipo con Chrome, Edge o Firefox actual; pantalla de 1366 × 768 o superior

Tabla 9.4 — Hardware de campo y de usuario.

9.3 Software de base y aplicación

Capa	Software y versión	Observación
Sistema operativo (servidor)	Ubuntu Server 24.04 LTS	Soporte extendido
Contenedores	Docker 24 o superior + Docker Compose	Despliegue reproducible
Proxy y TLS	Nginx 1.24 o superior · Let's Encrypt	Certificados renovables
Base de datos	PostgreSQL 15 o superior + TimescaleDB 2.x	Módulo IoT verificado en PostgreSQL 16.15
Broker MQTT	Eclipse Mosquitto 2.x	Pruebas automáticas con amqtt 0.12.1
Backend	Python 3.12 · Django 4.2 LTS · Django REST Framework · Channels 4 · Celery 5 · Redis 7	Según arquitectura de la sección 6.2
Módulo de ingesta	Python 3 · paho-mqtt 2.1 · psycopg 3.3	Verificado (cap. 8)
Frontend	HTML/JS PWA (demo) → React 18 + Tailwind CSS 3 (versión de producción)	Service Worker para modo sin conexión
Hosting del frontend	Vercel	HTTPS y CDN incluidos
Integraciones	Don Gestión (iframe/REST) · AFIP-WSFE (Zeep SOAP)	AFIP requiere certificado digital del cliente
Navegadores soportados	Chrome / Edge / Firefox / Safari, últimas dos versiones principales	Probado en Chromium

Tabla 9.5 — Software de base y de aplicación.

9.4 Seguridad y auditoría

9.4.1 Autenticación y control de acceso

- Autenticación con JSON Web Tokens: token de acceso de 15 minutos y token de renovación de 7 días, almacenados en memoria y no en el almacenamiento local del navegador (sección 7.5.2.2).
- Control de acceso por roles: Administrador, Supervisor Productivo, Operario de Galpón, Vendedor Comercial y Veterinario, cada uno con los módulos y permisos de la sección 7.5.2.1. Cada operario solo accede a los galpones que se le asignaron.
- Contraseñas almacenadas únicamente como hash (algoritmo PBKDF2 o Argon2 de Django), nunca en texto plano; límite de intentos fallidos de ingreso.

Importante: la demo funcional valida el usuario y la clave en el propio navegador, con dos cuentas de ejemplo, solo para mostrar los perfiles. Ese mecanismo no es seguro y no se utiliza en producción, donde la autenticación se resuelve en el servidor con JWT.

9.4.2 Protección de datos en tránsito y en reposo

- Todo el tráfico web viaja por HTTPS (TLS). Los sensores publican por MQTT sobre TLS (puerto 8883) con usuario y contraseña.
- El broker aplica listas de control de acceso por topic: cada nodo solo puede publicar en sus propios topics y el servicio de ingesta es el único suscriptor.
- La base de datos y Redis no se exponen a internet; solo son accesibles desde la red interna de los contenedores.
- Las credenciales (base de datos, MQTT, certificado AFIP) se guardan en variables de entorno fuera del repositorio.

- Validación de entrada: la ingesta descarta lecturas con formato inválido, valores fuera de rango físico y sensores no registrados (verificado en las pruebas IOT-20 a 27).
- Respaldos: copia diaria de la base (pg_dump) con retención de 30 días, almacenada fuera del servidor y cifrada.
- Los datos personales (usuarios y clientes) se tratan conforme a la Ley 25.326 de Protección de Datos Personales de la República Argentina y se limitan a los estrictamente necesarios.

9.4.3 Auditoría

Toda operación de escritura sobre las tablas auditadas queda registrada por triggers de PostgreSQL en la tabla auditoria, con usuario, fecha y hora, tabla, operación (INSERT, UPDATE o DELETE) y una copia en formato JSON del registro antes y después del cambio. Esto mantiene la trazabilidad incluso ante cambios hechos directamente sobre la base. El usuario de la aplicación se informa por transacción mediante SET LOCAL app.usuario.

El mecanismo fue implementado (backend/db/auditoria.sql) y probado: al marcar una alerta IoT como revisada con el usuario «supervisor1», quedó registrada la operación UPDATE con el valor anterior (revisada = falso) y el posterior (revisada = verdadero).

Usuario	Tabla	Operación	Antes	Después
supervisor1	alertas_iot	UPDATE	revisada = false	revisada = true

Tabla 9.6 — Registro de auditoría obtenido en la prueba.

Además del registro en base de datos, el servicio de ingesta deja en su log cada alerta generada y cada lectura descartada, lo que permite investigar fallas de sensores.

9.4.4 Riesgos de seguridad y medidas

Riesgo	Medida
Acceso no autorizado a la API	JWT de vida corta, roles y permisos por galpón, límite de intentos de ingreso
Suplantación de un sensor	Usuario y contraseña por nodo, TLS y ACL por topic en el broker
Pérdida de datos	Respaldo diario cifrado con retención de 30 días y prueba periódica de restauración
Datos falsos o corruptos en la base	Validación de formato y rango antes de guardar; descarte registrado en el log
Exposición de claves	Variables de entorno fuera del repositorio; rotación ante sospecha de filtración

Tabla 9.7 — Riesgos de seguridad y medidas de mitigación.

9.5 Configuración de despliegue de referencia

9.5.1 Configuración Docker Compose

```
# docker-compose.yml
# ============================================================
# Infraestructura completa del Sistema Avicola
# ============================================================

version: '3.9'

services:

  backend:
    build: ./backend
    command: gunicorn avicola.asgi:application -w 4
             -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000
    env_file: .env
    depends_on: [db, redis, mqtt]
    volumes:
      - static_files:/app/staticfiles
      - media_files:/app/media

  iot_gateway:
    build: ./backend
    command: python manage.py run_iot_gateway
    env_file: .env
    depends_on: [db, mqtt]

  celery_worker:
    build: ./backend
    command: celery -A avicola worker -l info -c 4
    env_file: .env
    depends_on: [db, redis]

  db:
    image: timescale/timescaledb:latest-pg15
    environment:
      POSTGRES_DB:       avicola_db
      POSTGRES_USER:     avicola
      POSTGRES_PASSWORD: secret
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./sql/init.sql:/docker-entrypoint-initdb.d/init.sql

  redis:
    image: redis:7-alpine
    volumes: [redis_data:/data]

  mqtt:
    image: eclipse-mosquitto:2
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf
      - mqtt_data:/mosquitto/data
    ports: ['1883:1883']
```

```
  frontend:
    build: ./frontend
    env_file: .env

  nginx:
    image: nginx:alpine
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
      - static_files:/var/www/static
      - media_files:/var/www/media
    ports: ['80:80','443:443']
    depends_on: [backend, frontend]

volumes:
  postgres_data:
  redis_data:
  mqtt_data:
  static_files:
  media_files:
```

Codigo 15.1 — docker-compose.yml: infraestructura completa del sistema avicola

9.5.2 Variables de Entorno (.env)

```
# .env — Variables de entorno del sistema

# Base de datos
DATABASE_URL=postgresql://avicola:secret@db:5432/avicola_db

# Redis
REDIS_URL=redis://redis:6379/0

# MQTT
MQTT_BROKER_HOST=mqtt
MQTT_BROKER_PORT=1883

# AFIP
AFIP_MODO=homologacion          # produccion | homologacion
AFIP_CUIT=20123456789
AFIP_PUNTO_VENTA=1
AFIP_CERT_PATH=/certs/prod.crt
AFIP_KEY_PATH=/certs/prod.key

# Django
SECRET_KEY=tu-clave-secreta-aqui
DEBUG=False
ALLOWED_HOSTS=avicola.midominio.com

# Frontend
REACT_APP_API_URL=https://avicola.midominio.com/api
REACT_APP_WS_URL=wss://avicola.midominio.com
```

Codigo 15.2 — Variables de entorno de produccion

9.5.3 Estrategia de Backup

Componente	Estrategia	Frecuencia	Retencion	Destino
PostgreSQL	pg_dump comprimido	Diaria 03:00	30 dias	S3 / Backblaze B2
TimescaleDB	Snapshot diferencial	Diaria 03:30	14 dias	S3 / Backblaze B2
PDFs / Medios	rsync incremental	Diaria 04:00	90 dias	NAS local + nube
Configuracion	Git repository	Cada cambio	Permanente	GitHub privado

Conclusiones

Conclusiones del relevamiento y propuesta

El relevamiento realizado sobre la empresa avicola confirma una operacion con alto potencial productivo (lote Brown-Classic supera tabla del fabricante en 21,2 huevos/AA) pero expuesta a riesgos significativos por la ausencia de herramientas de informacion integradas. El caso del lote LSL-Classic, con viabilidad real de 67,3% vs tabla 93,6%, materializa concretamente el costo de operar sin alertas automaticas.

El Sistema de Gestion Integral propuesto resuelve los tres problemas criticos diagnosticados: centraliza la informacion en una unica plataforma, implementa alertas automaticas en tiempo real para anomalias productivas y ambientales, y automatiza el 61% de las actividades administrativas actuales. La implementacion en 5 fases garantiza una transicion ordenada sin impactar la operacion diaria.

Conclusiones del desarrollo

Logros Técnicos

- Se diseño e implemento una arquitectura multicapa escalable que integra gestion operativa con monitoreo IoT en tiempo real usando tecnologias open source.
- El modulo de produccion encapsula toda la logica de negocio avicola de forma desacoplada y completamente testeable.
- La integracion MQTT a WebSocket permite que los datos del sensor lleguen al dashboard en menos de 2 segundos.
- La integracion AFIP-WSFE automatiza la emision de comprobantes eliminando intervencion manual y errores de transcripcion.
- El esquema de base de datos aprovecha columnas generadas, TimescaleDB y vistas para minimizar logica redundante en la aplicacion.

Desafíos y Soluciones

Desafio	Solucion adoptada
Volumen alto de datos de sensores	TimescaleDB con compresion automatica, particionado por chunks de 1 dia
Tiempo real sin sobrecargar el servidor	WebSocket via Django Channels con Redis como message broker
Inconsistencia de timestamps entre sensor y sistema	UTC en toda la cadena, conversion a hora local solo en el frontend
Pruebas AFIP sin errores en produccion	Ambiente de homologacion con CUIT y certificados de prueba separados
App usable por operarios sin experiencia tecnica	Formulario de 5 campos para registro diario, sin navegacion compleja

Trabajos Futuros

1. Prediccion de produccion con regresion lineal sobre el historico del lote para proyectar el fin optimo del ciclo con 4 semanas de anticipacion.
2. Integracion con bascula electronica para clasificacion automatica de huevos por peso (S/M/L/XL) en tiempo real.
3. Modulo de gestion nutricional con formulacion de raciones, consumo de balanceado y conversion alimenticia (kg alimento / docena de huevos).
4. Analisis de correlacion entre variables ambientales (temperatura, humedad, CO2) y produccion diaria para identificar condiciones optimas.
5. API publica documentada (OpenAPI / Swagger) para integracion con ERP, sistemas de trazabilidad y comercio electronico.

Glosario técnico

Termino	Definicion
API REST	Interfaz HTTP con estilo arquitectonico REST para comunicacion entre cliente y servidor.
CAE	Codigo de Autorizacion Electronico emitido por AFIP para validar facturas electronicas.
Celery	Libreria Python para tareas asincronas y programadas (cron jobs).
Django ORM	Mapeador objeto-relacional de Django para interactuar con PostgreSQL usando Python.
Hypertable	Tabla TimescaleDB particionada automaticamente por timestamp para series temporales.
JWT	JSON Web Token: token de autenticacion compacto con claims firmados digitalmente.
MQTT	Protocolo de mensajeria ligero para IoT sobre redes de baja velocidad.
TimescaleDB	Extension PostgreSQL para manejo eficiente de datos de series temporales.
WebSocket	Protocolo TCP bidireccional persistente para push de datos en tiempo real.
WSFE	Web Service de Facturacion Electronica de AFIP (protocolo SOAP).
ppm	Partes por millon: unidad de concentracion de gases en el aire del galpon.

Referencias bibliográficas

Relevamiento y propuesta

- Lohmann Tierzucht GmbH. (2024). Lohmann Brown-Classic Management Guide. Cuxhaven, Alemania.
- Lohmann Tierzucht GmbH. (2024). Lohmann LSL-Classic Management Guide. Cuxhaven, Alemania.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- Pressman, R. S. y Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
- Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Addison-Wesley.
- SENASA. (2023). Normativa sanitaria para establecimientos avicolas. Argentina.
- AFIP. (2024). Manual del Desarrollador: WSFE Web Service de Facturacion Electronica. https://www.afip.gob.ar
- TimescaleDB Inc. (2024). TimescaleDB Documentation v2.x. https://docs.timescale.com
- Django REST Framework. (2024). DRF Documentation. https://www.django-rest-framework.org
- Eclipse Foundation. (2024). Eclipse Mosquitto MQTT Broker. https://mosquitto.org/documentation/

Especificación y casos de uso

- Lohmann Tierzucht GmbH. (2024). Lohmann Brown-Classic Management Guide. Cuxhaven, Alemania.
- Lohmann Tierzucht GmbH. (2024). Lohmann LSL-Classic Management Guide. Cuxhaven, Alemania.
- Jacobson, I., Booch, G. y Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.
- IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- TimescaleDB Inc. (2024). TimescaleDB Documentation v2.x. https://docs.timescale.com
- Eclipse Foundation. (2024). Eclipse Mosquitto MQTT Broker. https://mosquitto.org/documentation/
- TILCOR S.A.S. — Avícola Tilcor. CUIT 30-71864712-2. Auillia 1388, Colonia Tirolesa, Córdoba.
- Don Gestión. (2024). Sistema ERP — avicola-tilcor.dongestion.com. Córdoba, Argentina.

Diseño y desarrollo

- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley.
- Lubanovic, B. (2023). Introducing Python (3rd ed.). O'Reilly Media.
- TimescaleDB Inc. (2024). TimescaleDB Documentation v2.x. https://docs.timescale.com
- Django REST Framework. (2024). DRF Documentation. https://www.django-rest-framework.org
- AFIP. (2024). Manual del Desarrollador WSFE. https://www.afip.gob.ar/fe/documentos/
- Lohmann Tierzucht GmbH. (2024). Brown-Classic y LSL-Classic Management Guide.
- Eclipse Foundation. (2024). Mosquitto MQTT Broker. https://mosquitto.org/documentation/
- Meta Open Source. (2024). React Documentation. https://react.dev
- Eclipse Paho. (2024). Paho MQTT Python Client. https://www.eclipse.org/paho/